ANALISIS DAN IMPLEMENTASI METODE *TRANSFER LEARNING* UNTUK IDENTIFIKASI VIDEO *DEEPFAKE* MENGGUNAKAN MODEL *EFFICIENTNET*

TUGAS AKHIR

Muhammad Dhaoed Arrazaq

2101020107

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNIK ELEKTRO DAN INFORMATIKA
FAKULTAS TEKNIK DAN TEKNOLOGI KEMARITIMAN
UNIVERSITAS MARITIM RAJA ALI HAJI
TANJUNGPINANG
2026

ANALISIS DAN IMPLEMENTASI METODE *TRANSFER LEARNING* UNTUK IDENTIFIKASI VIDEO *DEEPFAKE* MENGGUNAKAN MODEL *EFFICIENTNET*

Tugas Akhir

Skema Skripsi

Tugas Akhir diajukan sebagai salah satu persyaratan memperoleh gelar Sarjana Teknik pada Program Studi Teknik Informatika

Muhammad Dhaoed Arrazaq

2101020107

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNIK ELEKTRO DAN INFORMATIKA
FAKULTAS TEKNIK DAN TEKNOLOGI KEMARITIMAN
UNIVERSITAS MARITIM RAJA ALI HAJI
TANJUNGPINANG

2026

PERNYATAAN MENGENAI TUGAS AKHIR DAN SUMBER INFORMASI SERTA PELIMPAHAN HAK CIPTA

Dengan ini saya menyatakan bahwa tugas akhir berjudul **"Analisis Dan Implementasi Metode *Transfer Learning* Untuk Identifikasi Video *Deepfake* Menggunakan Model *EfficientNet*"** adalah benar karya saya dengan arahan dari komisi pembimbing dan belum diajukan dalam bentuk apa pun kepada perguruan tinggi mana pun. Sumber informasi yang berasal atau dikutip dari karya yang diterbitkan maupun tidak diterbitkan dari penulis lain telah disebutkan dalam teks dan dicantumkan dalam Daftar Pustaka di bagian akhir skripsi ini.

Apabila di kemudian hari terbukti pernyataan saya tidak benar dan melanggar peraturan yang berlaku dalam karya tulis dan hak intelektual, maka saya bersedia ijazah yang telah saya terima untuk ditarik kembali oleh Universitas Maritim Raja Ali Haji dan menerima sanksi lainnya sesuai dengan peraturan yang berlaku. Dengan ini saya melimpahkan hak cipta dari karya tulis saya kepada Universitas Maritim Raja Ali Haji.

Tanjungpinang, 08 Desember 2025

Muhammad Dhaoed Arrazaq
NIM 2101020107

© Hak Cipta Milik Universitas Maritim Raja Ali Haji, Tahun 2021

Hak Cipta Dilindungi Undang- Undang

Dilarang mengutip sebagian atau seluruh karya tulis ini tanpa mencantumkan atau menyebutkan sumbernya. Pengutipan hanya untuk kepentingan pendidikan, penelitian, penulisan karya ilmiah, penyusunan laporan, penulisan kritik, atau tinjauan suatu masalah; dan pengutipan tersebut tidak merugikan kepentingan Universitas Maritim Raja Ali Haji.

Dilarang mengumumkan dan memperbanyak sebagian atau seluruh karya tulis ini dalam bentuk apapun tanpa izin Universitas Maritim Raja Ali Haji.

HALAMAN PERSETUJUAN

Judul	:	Analisis Dan Implementasi Metode *Transfer Learning* Untuk Identifikasi Video *Deepfake* Menggunakan Model *EfficientNet*
Nama	:	Muhammad Dhaoed Arrazaq
NIM	:	2101020107
Program Studi	:	Teknik Informatika
Tanggal Persetujuan	:	08 Desember 2025

Disetujui oleh,

Komisi Pembimbing

Hendra Kurniawan S. Kom., M.Sc.Eng., Ph. D.
(Ketua)

Fortia Magfira M.Kom
(Anggota)

HALAMAN PENGESAHAN

Judul	:	Analisis Dan Implementasi Metode *Transfer Learning* Untuk Identifikasi Video *Deepfake* Menggunakan Model *EfficientNet*
Nama	:	Muhammad Dhaoed Arrazaq
NIM	:	2101020107

Telah berhasil dipertahankan di hadapan Komisi Penguji sebagai bagian persyaratan yang diperlukan dalam memperoleh gelar Sarjana Teknik pada Program Studi Teknik Informatika, Jurusan Teknik Elektro dan Informatika, Fakultas Teknik dan Teknologi Kemaritiman, Universitas Maritim Raja Ali Haji.

KOMISI PENGUJI

Ketua	:	Ferdi Chahyadi, S.Kom., M.Cs	(.....)
Anggota I	:	Novrizal Fattah Fahmitra, S.Kom., M.Kom	(.....)
Anggota II	:	Rifaldi Herikson, S.Kom., M.Kom	(.....)
Anggota III	:	Hendra Kurniawan, S. Kom., M.Sc.Eng., Ph. D.	(.....)
Anggota IV	:	Fortia Magfira, M. Kom	(.....)

Mengetahui,

Ketua Jurusan Teknik Informatika dan Elektro

Hollanda Arief Kusuma, S.IK., M.Si

NIP. 198904012019031016

Tanggal Ujian: 19 Desember 2025 Tanggal Lulus:

ABSTRAK

MUHAMMAD DHAOED ARRAZAQ. Analisis dan Implementasi Metode *Transfer Learning* untuk Identifikasi Video *Deepfake* Menggunakan Model *EfficientNet*. Dibimbing oleh HENDRA KURNIAWAN dan FORTIA MAGFIRA.

Manipulasi media digital yang didorong oleh perkembangan *Generative Adversarial Networks* (GANs) dan *Autoencoders* telah melahirkan fenomena *Deepfake*, sebuah teknik sintesis citra wajah hiper-realistis yang berpotensi mendestabilisasi kepercayaan publik, keamanan informasi, dan reputasi individu. Tantangan fundamental dalam ranah forensik digital saat ini adalah menciptakan mekanisme deteksi yang mampu menyeimbangkan akurasi tinggi terhadap artefak manipulasi wajah yang semakin halus dengan efisiensi komputasi yang memadai untuk penerapan skala luas. Penelitian ini bertujuan untuk merancang, mengimplementasikan, dan mengevaluasi arsitektur sistem deteksi *Deepfake* yang tangguh dengan mendayagunakan model *Convolutional Neural Network* (CNN) *EfficientNet-B0* melalui pendekatan *transfer learning*. Penelitian ini menerapkan strategi pelatihan dua fase—ekstraksi fitur dan *fine-tuning*—pada dataset *FaceForensics++* serta mengintegrasikan logika *Consensus Voting* untuk mengagregasi prediksi spasial menjadi keputusan temporal pada tingkat video.

Berdasarkan serangkaian pengujian empiris yang komprehensif, penelitian ini menetapkan model yang dilatih dengan dataset terkurasi sebanyak 300 video sebagai konfigurasi optimal. Model usulan berhasil mencapai tingkat Akurasi sebesar 87.18%, dengan nilai Presisi 88.5%, Recall 75.8%, dan F1-Score 81.6% pada dataset pengujian. Analisis *Confusion Matrix* mengungkapkan bahwa sistem memiliki tingkat spesifisitas yang tinggi, meminimalisir risiko kesalahan klasifikasi pada konten asli (*False Positive*), yang merupakan aspek krusial dalam pembuktian forensik. Selain itu, pengujian performa menunjukkan bahwa meskipun proses pelatihan membutuhkan sumber daya komputasi tinggi berbasis GPU, proses inferensi model dapat berjalan efisien pada perangkat komputasi standar, menegaskan kelayakan sistem untuk diimplementasikan dalam skenario dunia nyata (*in the wild*).

Kata kunci: *Deepfake, EfficientNet, Transfer Learning,* Forensik Digital, *Convolutional Neural Network,* Deteksi Wajah*, Consensus Voting.*

ABSTRACT

MUHAMMAD DHAOED ARRAZAQ. Analysis and Implementation of *Transfer Learning* Method for *Deepfake* Video Identification Using *EfficientNet* Model. Supervised by HENDRA KURNIAWAN and FORTIA MAGFIRA.

Digital media manipulation technologies, particularly the *Deepfake* phenomenon driven by advancements in Generative Adversarial Networks (GANs) and Autoencoders, has precipitated a crisis of trust in digital information integrity. A fundamental challenge in modern digital forensics is developing detection mechanisms that balance high accuracy against increasingly subtle facial manipulation artifacts with sufficient computational efficiency for large-scale deployment. This study aims to design, implement, and evaluate a robust *Deepfake* detection system architecture by leveraging the *EfficientNet-B0* Convolutional Neural Network (CNN) model through a *transfer learning* approach. The research employs a two-phase training strategy—*Feature Extraction* and *fine-tuning*—on the FaceForensics++ dataset and integrates Consensus Voting logic to aggregate spatial predictions into video-level temporal decisions.

Based on comprehensive empirical testing, this study identifies the model trained on a curated dataset of 300 videos as the optimal configuration. The proposed model achieved an Accuracy of 87.18%, with a Precision of 88.5%, Recall of 75.8%, and F1-Score of 81.6% on the test dataset. *Confusion Matrix* analysis reveals that the system demonstrates a high level of specificity, minimizing the risk of misclassifying authentic content (False Positives), which is a critical aspect of forensic verification. Furthermore, performance testing indicates that while the training process requires high computational GPU resources, the model's inference process operates efficiently on standard computing devices, confirming the system's feasibility for implementation in real-world ("in the wild") scenarios.

Keywords: *Deepfake, EfficientNet, Transfer Learning, Digital Forensics, Convolutional Neural Network, Face Detection, Consensus Voting.*

PRAKATA

Puji dan syukur Penulis panjatkan kepada Allah *subhanahu wa ta'ala atas segala* karunia-Nya sehingga laporan tugas akhir ini berhasil diselesaikan. Tema yang dipilih dalam tugas akhir yang akan dilaksanakan pada bulan Desember 2025 ini ialah Identifikasi, dengan judul **"Analisis Dan Implementasi Metode *Transfer Learning* Untuk Identifikasi Video *Deepfake* Menggunakan *EfficientNet*".**

Terima kasih penulis ucapkan kepada Bapak Hendra Kurniawan, S.Kom., M.Sc.Eng, Ph.D, dan Ibu Fortia Magfira. selaku Dosen Pembimbing karena telah menyediakan waktu, tenaga, dan pikiran selama membimbing penulis dengan memberikan arahan dan masukan selama proses penyusunan tugas akhir ini. Dosen-dosen penguji yang telah memberikan saran serta masukan berharga untuk penyempurnaan penelitian. Kedua orang tua penulis, Bapak Dadang Kurniawan dan Ibu Nurlaila yang selalu tanpa henti memberikan doa dan dorongan motivasi kepada penulis, saudara, dan seluruh keluarga. Disamping itu, penghargaan penulis sampaikan kepada Ibu Martaleli Bettiza, S.Si., M.Sc. dan Bapak Hollanda Arief Kusuma, S.IK., M.Si_, beserta seluruh dosen dan staf Fakultas Teknik dan Teknologi Kemaritiman (FTTK) yang telah membantu selama proses penyusunan Tugas akhir ini. Terakhir sahabat dan rekan-rekan Teknik Informatika, atas segala bantuan, dukungan, dan solidaritasnya.

Semoga laporan tugas akhir ini dapat bermanfaat untuk banyak orang.

Tanjungpinang, 08 Desember 2025

Muhammad Dhaoed Arrazaq

DAFTAR ISI

HALAMAN PERSETUJUAN iii
HALAMAN PENGESAHAN iv
ABSTRAK v
ABSTRACT vi
PRAKATA vii
DAFTAR ISI viii
DAFTAR TABEL xi
DAFTAR GAMBAR xiii
DAFTAR LAMPIRAN xv
BAB I PENDAHULUAN 1
A. Latar Belakang 1
B. Perumusan Masalah 5
C. Batasan Penelitian 5
D. Tujuan Penelitian 6
E. Manfaat Penelitian 6
BAB II KAJIAN LITERATUR 7
A. Tinjauan Pustaka 7
B. Landasan Teori 9
1. Deepfake 9
2. DeepLearning 13
3. Transfer learning 14
4. CNN 17
5. EfficientNet 18
6. Faceforensics++ 27

7. Python 28
8. Google Colab 31
9. Confusion matrix 31
BAB III METODE PENELITIAN 33
A. Sub Waktu Penelitian 33
B. Alat dan Instrumen Penelitian 33
C. Jenis Penelitian 35
D. Studi Pustaka 35
E. Prosedur Pelaksanaan Penelitian 36
F. Pengumpulan Data 38
E. Perancangan Sistem 39
1. Preprocessing Data 40
2. Perancangan Arsitektur *EfficientNet* 44
3. Pelatihan Sistem 49
F. Analisis Data 56
BAB IV HASIL DAN PEMBAHASAN 61
A. Lingkungan Uji Coba 61
B. Persiapan dan Pra-pemrosesan Data 63
1. Persiapan Data (Data Preparation) 63
2. Penyaringan Data (Data Filtering) 63
3. Pembersihan Data (Data Cleaning) 63
4. Pra-proses Data (Data Pre-processing) 64
C. Implementasi Sistem 72
1. Implementasi Arsitektur *EfficientNet-B0* dan *Transfer Learning* 72
2. Logika Sistem 75
D. Pelatihan Model (*Training Model*) 77

E. Analisis Hasil Pelatihan 86
F. Perbandingan Lingkungan Uji Coba 87
G. Evaluasi Model 90
1. Evaluasi Kinerja Model 90
2. Perbandingan dan Pemilihan Model Terbaik 93
H. Tampilan User Interface 96
I. Hasil Pengujian Model 101
1. Pengujian Menggunakan Data Uji 101
2. Pengujian Batas Minimal Deteksi Wajah (*Boundary Testing*) 104
3. Pengujian Menggunakan Logika Sistem 107
4. Pengujian Menggunakan Data Baru (*In the Wild*) 108
5. Pengujian Pengaruh Resolusi dan Frame Rate Pada Model 111
J. Analisis dan Perbandingan Model 115
BAB V SIMPULAN DAN SARAN 117
A. Kesimpulan 117
B. Saran 117
DAFTAR PUSTAKA 119
LAMPIRAN 128

DAFTAR TABEL

Tabel 1. Struktur lengkap *EfficientNet-B0* 20
Tabel 2. *Compound Scale* Pada *EfficientNet-B0* 22
Tabel 3. *Confusion Matrix* 31
Tabel 4. Spesifikasi Perangkat yang digunakan 34
Tabel 5. Dataset yang akan digunakan: 41
Tabel 6. Spesifikasi Lingkungan Uji Coba 61
Tabel 7. *Hyperparameter* yang digunakan 77
Tabel 8. Ringkasan Hasil Pelatihan Fase 1 (300 Data) 78
Tabel 9. Ringkasan Hasil Pelatihan Fase 2 (300 Data) 80
Tabel 10. Ringkasan Hasil Pelatihan Fase 1 (400 Data) 82
Tabel 11. Ringkasan Hasil Pelatihan Fase 2 (400 Data) 83
Tabel 12. Ringkasan Hasil Pelatihan Fase 1 84
Tabel 13. Ringkasan Hasil Pelatihan Fase 2 85
Tabel 14. Analisis Waktu Pelatihan 88
Tabel 15. Metrik Evaluasi (300 Data) 91
Tabel 16. Metrik Evaluasi (400 Data) 92
Tabel 17. Metrik Evaluasi 93
Tabel 18. Perbandingan Metrik Evaluasi Model 93
Tabel 19. Hasil Pengujian Dataset (Asli) 102
Tabel 20. Hasil Pengujian Dataset (*Deepfake)* 103
Tabel 21. Hasil Pengujian Parameter *Minimal Face Size* (Asli) 105
Tabel 22. Hasil Pengujian Parameter *Minimal Face Size* (*Deepfake*) 105
Tabel 23. Hasil Pengujian Parameter (*In the Wild* - Asli) 106
Tabel 24. Hasil Pengujian Parameter (*In the Wild – Deepfake*) 106
Tabel 25. Hasil Pengujian Logika *NoFace* 107
Tabel 26.Hasil Pengujian Logika *FaceLimit* 108
Tabel 27. Hasil Pengujian Data (*In the Wild* – Asli) 109
Tabel 28. Hasil Pengujian Data Baru (*In the Wild - Deepfake)* 110
Tabel 29. Hasil Pengujian FPS pada data (Asli) 111
Tabel 30. Hasil Pengujian FPS pada data (*Deepfake)* 112
Tabel 31. Hasil Pengujian Pengaruh Resolusi (Asli) 113

Tabel 32. Hasil Pengujian Pengaruh Resolusi (*Deepfake)* 114
Tabel 33.Hasil Pengujian pada Parameter MFS 10x10 (Asli)..... 136
Tabel 34.Hasil Pengujian pada Parameter MFS 5x5 (Asli)..... 137
Tabel 35.Hasil Pengujian pada Parameter MFS 0x0 (Asli)..... 138
Tabel 36. Hasil Pengujian pada Parameter MFS 10x10 (*Deepfake*) 139
Tabel 37.Hasil Pengujian pada Parameter MFS 5x5 (*Deepfake*) 140
Tabel 38.Hasil Pengujian pada Parameter MFS 0x0 (Asli)..... 141

DAFTAR GAMBAR

Gambar 1. Mekanisme Pembuatan Konten *Deepfake* 9
Gambar 2. Perkembangan teknologi *Deepfake* dari tahun ke tahun..... 10
Gambar 3. Struktur GANs 11
Gambar 4. Alur Kerja GAN..... 11
Gambar 5. Proses mendeteksi manipulasi wajah dengan CNN dan RNN 13
Gambar 6. Ilustrasi metode tradisional ML vs *Transfer learning* 14
Gambar 7. Skema *neural network* dengan *transfer learning* 15
Gambar 8. Arsitektur LSTM dengan opsi *transfer learning* 15
Gambar 9. Alur *Fine-tuning* 17
Gambar 10. Arsitektur CNN..... 18
Gambar 11. (a) Arsitektur CNN Tradisional, (b) *Pre-Trained* CNN 18
Gambar 12. Arsitektur *EfficientNet* 19
Gambar 13. Struktur *Compound Scaling*..... 21
Gambar 14. Skema *Squeeze-and-Excitation Blocks* 23
Gambar 15. Model *Pre-Trained* 25
Gambar 16 . Halaman utama *Faceforensics++* 27
Gambar 17. Sample wajah dari Dataset *FaceForensics++*..... 28
Gambar 18. Alur Metode Penelitian..... 36
Gambar 19. Dataset Video Asli 39
Gambar 20. Dataset Video *Deepfake*..... 39
Gambar 21. Flowchart Preprocessing Data 40
Gambar 22. Ektraksi *Frame* 43
Gambar 23. Konsep Perancangan Model 45
Gambar 24. *Flowchart* Perancangan Sistem 51
Gambar 25. *Use Case Diagram*..... 52
Gambar 26. *Sequence Diagram*..... 53
Gambar 27. *User Interface* Halaman Utama 54
Gambar 28. *User Interface Sidebar* Halaman Utama..... 54
Gambar 29. *User Interface* setelah Upload File 55
Gambar 30. *User Interface* setelah menampilkan Hasil Ouput..... 56
Gambar 31. Input pada *Channel Red*..... 57

Gambar 32. Input *Channel Red* setelah di*rescaling* 57
Gambar 33. Hasil Normalisasi *Channel Red* 58
Gambar 34. Kurva Fase 1 – *Initial Training* (300 Data) 79
Gambar 35.Kurva Fase 2 – *Fine-tuning* (300 Data) 80
Gambar 36. Kurva Fase 1 – *Initial Training* (400 Data) 82
Gambar 37. Kurva Fase 2 – *Fine-tuning* (400 Data) 83
Gambar 38. Kurva Fase 1 – *Initial Training* pada perangkat *local* 85
Gambar 39. Kurva Fase 2 – *Fine-tuning* pada perangkat local 86
Gambar 40. *Confusion Matrix* (300 Data) 90
Gambar 41. *Confusion Matrix* (400 Data) 91
Gambar 42. *Confusion Matrix* pada Perangkat *Local* 92
Gambar 43. Halaman Dashboard 96
Gambar 44. Halaman Utama Sistem 97
Gambar 45. Halaman Utama Sistem (*Sidebar*) 97
Gambar 46. Halaman Utama Sistem – Video Terunggah 98
Gambar 47. Halaman Utama Sistem - Hasil Klasifikasi – Asli 98
Gambar 48. Halaman Utama Sistem - Hasil Klasifikasi – *Deepfake* 99
Gambar 49. Halaman Utama Sistem – Terlalu Banyak Wajah 99
Gambar 50. Halaman Utama Sistem – Wajah Tidak Terdeteksi 100

DAFTAR LAMPIRAN

Lampiran 1. Model *Summary* 128
Lampiran 2. Link ke Dataset yang digunakan 135
Lampiran 3. Ringkasan Parameter yang digunakan 135
Lampiran 4. Tabel Pengujian Parameter *Minimal Face Size* 136

BAB I
PENDAHULUAN

A. Latar Belakang

Seperti namanya, ”*deepfake*” secara etimologi adalah gabungan dari “*deep learning*” dan “palsu” yang melibatkan pengubahan video secara digital untuk menciptakan representasi hiper-realistis dari individu yang seolah-olah mengatakan dan melakukan hal-hal yang sebenarnya tidak pernah terjadi [1]. Istilah ini umumnya digunakan untuk merujuk pada manipulasi media yang sudah ada (gambar, video, dan/atau audio) atau pembangkitan media baru (sintetis) menggunakan pendekatan berbasis *deep learning*. Data *deepfake* yang paling sering dibahas meliputi gambar wajah palsu, rekaman suara palsu, dan video palsu yang menggabungkan gambar palsu dan rekaman suara palsu. Meskipun kata “palsu” dalam istilah ini menunjukkan media yang dimanipulasi atau disintesis, terdapat banyak aplikasi yang tidak berbahaya dari teknologi *deepfake*, misalnya untuk hiburan dan seni kreatif. Dengan pertimbangan ini, istilah lain “deep synthesis” telah diusulkan sebagai alternatif yang lebih netral [2]. Namun, istilah baru ini belum banyak diketahui secara luas.

Istilah *deepfake* pertama kali muncul pada tahun 2017 dan menjadi sangat populer setelah kemunculannya. Semua bermula dari salah satu pengguna aplikasi *Reddit* dengan *username* /u/deepfakes, pencipta subreddit /r/deepfakes yang kemudian diblokir, mendedikasikannya untuk video porno sintetis, dan repositori kode open-source GitHub untuk sistem *deepfake* yang banyak direplikasi dan di-fork di mana pengguna tersebut mengembangkan GAN menggunakan TensorFlow, perangkat lunak mesin pencari gratis dari Google untuk menempelkan wajah seseorang ke tubuh wanita lain dalam film pornografi [3]. Munculnya *deepfake* semacam itu kemudian kembali menjadi perbincangan publik yang hangat ketika sebuah video dari Mark Zuckerberg muncul pada Juni 2019 yang membahas kekuatan Facebook untuk mengendalikan semua data penggunanya yang ternyata adalah video yang diedit secara masif, padahal video aslinya telah diunggah

sebelumnya oleh Zuckerberg sendiri pada September 2017 ketika dia membahas campur tangan Facebook dalam pemilihan umum di Rusia [4].

Sejauh ini penggunaan teknologi *deepfake* AI generatif yang paling umum adalah untuk membuat gambar intim dan pornografi non-konsensual. Sebuah tinjauan *deepfake* online tahun 2019 menemukan bahwa 96% adalah pornografi [5]. Analisis tahun 2023 tentang *deepfake* non-konsensual menemukan bahwa setidaknya 244.625 video telah ditambahkan ke situs web teratas yang disiapkan untuk menghosting video porno *deepfake* dalam tujuh tahun sebelumnya, 113.000 di antaranya ditambahkan pada tahun 2023, menandai peningkatan 54% dibandingkan tahun sebelumnya [6]. Pada tahun 2025, Motif penyalahgunaan *deepfake* juga semakin bervariasi, dengan kategori terbesar pada konten eksplisit non-konsensual (32%), penipuan finansial (23%), manipulasi politik (14%), dan misinformasi (13%). Serangan tidak hanya menargetkan figur publik, tetapi juga masyarakat umum, termasuk perempuan, pelajar, dan anak di bawah umur. Secara global, insiden paling banyak terjadi di Amerika Utara (38%), diikuti Asia (27%) dan Eropa (21%). Bahkan 63% kasus melibatkan elemen lintas negara, sehingga menambah kompleksitas dalam proses penegakan hukum dan pencegahan. [7].

Perkembangan teknologi *deepfake* yang menjamur dapat menimbulkan ancaman epistemik, seperti menggiring opini masyarakat untuk lebih mudah mempercayai akan informasi yang mereka konsumsi yang sebenarnya tidak nyata dan tidak pernah terjadi [8]. Menjamurnya teknologi seperti *deepfake* dapat menimbulkan dampak negatif , yakni munculnya tindak pidana baru [9]. Salah satu fenomena atau kasus nyata yang muncul akibat perkembangan teknologi ini risiko *"People Defense"*, di mana terdakwa mengklaim bukti yang didakwakan kepada tersangka adalah palsu, seperti pada kasus *People v. Foreman*. Kondisi tersebut berpotensi menimbulkan krisis kredibilitas bukti digital di pengadilan. [10]. Di Korea Selatan, kasus manipulasi wajah guru perempuan ke dalam konten pornografi meningkat drastis, dengan data menunjukkan lonjakan lebih dari 670% antara 2021–2024. Survei pada 2024 mencatat lebih dari 2.492 kasus manipulasi foto ilegal di lingkungan sekolah, melibatkan ratusan guru dan siswa sebagai korban [11]. Penyalahgunaan dalam dalam bentuk politik, konflik Rusia-Ukraina yang

sedang berlangsung, video *deepfake* telah muncul sebagai alat propaganda perang. Dalam satu kasus, sebuah video palsu Presiden Ukraina Volodymyr Zelenskyy muncul secara online, yang secara salah menyatakan penyerahan Ukraina [12].

Teknologi *deepfake* juga merambat sampai ke Indonesia. Fenomena penipuan berbasis *deepfake* di Indonesia menunjukkan eskalasi signifikan dengan peningkatan kasus sebesar 1550% antara tahun 2022 hingga 2023. Lonjakan ini menegaskan bahwa teknologi *deepfake* selain membawa inovasi, juga menghadirkan ancaman serius terhadap keamanan digital [13]. Kasus *deepfake* yang terjadi di Indonesia salah satunya adalah video manipulatif untuk menawarkan bantuan pemerintah palsu menggunakan wajah Presiden Prabowo Subianto dengan syarat transfer dana melalui WhatsAp sekitar 100 orang tersebar di 20 provinsi, Kerugian diperkirakan sekitar Rp 95 juta rupiah dalam kurun waktu empat bulan sebelum tertangkap [14]. Penyimpangan lainnya juga terjadi di Universitas Udayana (Unud), Bali, seorang mahasiswa berinisial SLKDP diduga manipulasi data pribadi mahasiswi menjadi konten vulgar menggunakan *deepfake* di Telegram, dengan mengambil tangkapan layar dari akun Instagram korban tanpa izin, selain melanggar privasi ini juga mempengaruhi mentalitas korban, menghancurkan hubungan mereka, dan menodai martabat korban [15]. Seperti halnya video *deepfake* lainnya menjelang pemilu Presiden 2029, diperkirakan akan terancam *deepfake* dan isu SARA [16]. Untuk mengantisipasi tantangan tersebut, penyelenggara pemilu memperkuat kolaborasi dengan Kementerian Komunikasi dan Digital (Komdigi) serta Badan Siber dan Sandi Negara (BSSN) [17].

Dengan besarnya dampak negatif yang dihasilkan teknologi *Deepfake*, berbagai penelitian telah diilakukan untuk mendeteksi *deepfake*, mulai dari analisis tekstur wajah, pola pergerakan mata dan bibir, hingga deteksi artefak visual yang tidak alami [18]. Deteksi video *deepfake* dapat diklasifikasikan menjadi tiga kategori berdasarkan pendekatannya: Metode *Feature-Based (Handcrafted Features), Deep learning-Based Model,* dan *Hybrid Methods (Hybrid Feature-Based* dan *Deep learning Approaches)* [19]. Penelitian untuk mendeteksi *deepfake* menggunakan *Convolutional Neural Network* (CNN) yang dioptimalkan dengan arsitektur *EfficientNet-B6*. Hasil penelitian menunjukkan bahwa model CNN

EfficientNet mampu mencapai akurasi sekitar 90%, dengan nilai precision, recall, dan F1-score secara real-time melalui platform seperti Flask [20]. Penelitian dengan metode serupa yang sama berfokus pada pengembangan sistem deteksi *deepfake* berbasis CNN. Penelitian yang dilakukan pada dataset publik Celeb-DF. Menunjukkan bahwa model mencapai performa tertinggi, dengan akurasi 93,2% dan AUC 0,93. Model CNN menawarkan keseimbangan yang praktis antara akurasi dan biaya komputasi, menjadikannya cocok untuk digunakan di dunia nyata, terutama pada perangkat dengan sumber daya terbatas [21]. Penelitian yang dilakukan oleh Anlin mengusulkan sistem deteksi *deepfake* berbasis *transfer learning* dengan memanfaatkan FaceNet512 untuk menghasilkan sampel wajah, kemudian diklasifikasikan menggunakan MobileNetV2. Hasil pengujian menunjukkan akurasi rata-rata mencapai 82,4% dengan nilai AUC sebesar 0,82. Pendekatan ini terbukti lebih efisien dibandingkan metode analisis *full-frame video* karena hanya berfokus pada fitur wajah, sehingga lebih cepat, ringan, dan tetap akurat [22].

Berdasarkan uraian pada latar belakang tersebut, penelitian ini mengusulkan sebuah sistem ”DEFEND” atau ”*Deepfake Efficient Detector*”. Penelitian ini bertujuan untuk mengukur kemampuan dan tingkat akurasi metode *transfer learning* pada arsitektur *EfficientNet* dalam mengklasifikasi dan mengenali video asli serta video *deepfake* yang banyak beredar di media sosial. Dataset *FaceForensics++* digunakan sebagai data latih dan data uji untuk memastikan model mampu mendeteksi manipulasi artefak visual secara akurat. Penelitian ini hadir dengan mengimplementasikan model *EfficientNet-B0* yang dirancang melalui teknik *compound scaling* untuk menyeimbangkan kedalaman, lebar, dan resolusi jaringan secara efisien. Penggunaan *EfficientNet* diharapkan mampu memberikan akurasi yang kompetitif namun dengan beban komputasi (FLOPs) yang jauh lebih rendah dibandingkan model konvensional. Hasil penelitian diharapkan dapat berkontribusi pada peningkatan akurasi deteksi *deepfake* serta membantu upaya meminimalisir mitigasi penyebaran konten manipulatif yang dapat menimbulkan dampak negatif yang di masyarakat.

B. Perumusan Masalah

Berdasarkan latar belakang penelitian yang telah diuraikan di atas, maka dapat disimpulkan bahwa rumusan masalah penelitian ini adalah untuk mengetahui tingkat akurasi yang dihasilkan metode *transfer learning* model *EffientNet* dalam mengklasifikasikan keaslian wajah pada video.

C. Batasan Penelitian

Dalam penelitian ini, penulis telah menetapkan beberapa batasan terkait penelitian ini sebagai berikut :

1. Penellitian ini berfokus pada metode *transfer learning* dengan menggunakan arsitektur *EfficientNet-B0* dilatih *(pre-trained)* pada dataset *ImageNet*.
2. Analisis deteksi difokuskan hanya pada area wajah (*Face Region)* dan diekstraksi menjadi *frames*/video. Penelitian ini tidak menganalisis komponen audio (suara), gestur, atau bagian tubuh lainnya untuk menentukan keaslian video. Dengan format input berupa video ".mp4". Durasi video rata-rata 10-50 detik, menggunakan berbagai variasi *frame fate* (seperti 24, 25, 30, 60, hingga 120 fps). Proses pengambilan sampel *frames* dilakukan sebanyak 20 *frames* per video sebagai representasi visual dari seluruh durasi video.
3. Dataset yang digunakan pada penelitian ini berasal dari *FaceForensics*++, https://niessnerlab.org/projects/roessler2018faceforensics.html, dengan rasio 224x224 piksel, yang dibagi dibagi menjadi data latih (70%), data validasi (20%), dan data uji (10%). Dataset ini berjumlah 300 video (terdiri dari 150 video "Asli", "0" dan 150 video "*Deepfake*", "1"). Model hanya bisa melakukan analisis pada video, jika terdapat minimal 1 wajah dan maksimal 3 wajah pada video yang di input/diunggah.
4. Model dikembangkan menggunakan dataset FaceForensics++, namun untuk menguji keandalan sistem "DEFEND", dilakukan pengujian *Trial and Error* menggunakan video dari internet (*In the wild*). Pengujian ini mencakup perbandingan berbagai resolusi video (seperti 360p, 480p, 720p, hingga 1080p) untuk menganalisis sejauh mana kualitas resolusi memengaruhi tingkat akurasi model dalam mengidentifikasi artefak manipulasi wajah.

5. *Output* akhir dari yang diharapkan dari model ini adalah mampu memberikan klasifikasi biner ”0” atau ”1”, dalam bentuk probabilitas keyakinan ”Asli” atau ”*Deepfake*” untuk hasil akhir dari identifikasi.

D. Tujuan Penelitian

Berdasarkan latar belakang penelitian yang telah diuraikan di atas, maka dapat disimpulkan bahwa tujuan penelitian ini adalah untuk mengetahui tingkat akurasi yang dihasilkan metode *transfer learning* model *EffientNet* dalam mengklasifikasikan keaslian wajah pada video.

E. Manfaat Penelitian

Manfaat dari penelitian yang penulis lakukan adalah sebagai berikut :

1. Bagi penulis, penelitian dan penulisan karya ilmiah ini memberikan pengalaman dan pengetahuan baru yang berharga, khususnya dalam memahami dan menerapkan metode *transfer learning* Model Arsitektur *EfficientNet* dalam mengidentifikasi keaslian video. Penelitian ini menawarkan solusi praktis dalam mengklasifikasikan unsur *Deepfake* guna menghindari terjadinya kesalahpahaman dalam penyebaran informasi.
2. Bagi perguruan tinggi dan akademisi, Penelitian ini diharapkan dapat memberikan kontribusi dalam penggunaan metode *transfer learning* model *EfficientNet* sebagai Algoritma yang mengidentifikasi Keaslian Video Asli dan Video *Deepfake* dapat digunakan sebagai acuan untuk penelitian yang akan datang di masa depan serta memperbaharui dan melengkapi penelitian sebelumnya yang sudah ada sebelumnya.
3. Bagi Masyarakat secara umum, penelitian ini diharapkan bisa meningkatkan kesadaran masyarakat terhadap ancaman penyalahgunaan dari teknologi *Deepfake* dan memberikan upaya mengenali penyebaran informasi palsu yang berbasis video di media sosial. Hasil dari penelitian ini jika memungkinkan akan dipublikasikan dalam bentuk, artikel ilmiah yang dapat diakses secara terbuka dalam bentuk jurnal untuk meningkatkan literasi digital masyarakat terhadap risiko konten *Deepfake* yang disalahgunakan.

BAB II
KAJIAN LITERATUR

A. Tinjauan Pustaka

Penelitian di bidang forensik media digital telah berkembang pesat sebagai respons terhadap kemajuan teknologi sintesis citra. Fokus utama dalam domain ini bergeser dari metode berbasis fitur manual (*hand-crafted features*) menuju pendekatan berbasis pembelajaran mendalam (*Deep learning*) yang mampu mempelajari representasi fitur yang kompleks secara otomatis[23].

Pada tahap awal perkembangannya, deteksi video manipulasi banyak mengandalkan analisis inkonsistensi fisik yang terlihat mata atau dapat dihitung secara geometris. Penelitian ini menyoroti pentingnya artefak visual seperti frekuensi kedipan mata yang tidak wajar, ketidaksinkronan pergerakan bibir, serta inkonsistensi bayangan pada wajah. Namun, seiring dengan perbaikan algoritma generatif yang mampu memimik perilaku biologis ini, metode berbasis fitur fisik menjadi kurang efektif [24].

Generasi berikutnya dari sistem deteksi beralih ke analisis artefak pada level piksel dan sinyal. Penelitian yang dilakukan oleh Marra dan Rossler menunjukkan bahwa CNN dapat mendeteksi residu yang ditinggalkan oleh proses *upsampling* dan *warping* pada wajah yang dimanipulasi. *FaceForensics++* menjadi standar de facto dalam mengevaluasi metode ini, menyediakan ribuan video dengan berbagai teknik manipulasi [25][26].

Pemilihan arsitektur jaringan saraf menjadi faktor determinan dalam kinerja deteksi. Arsitektur Xception, yang diperkenalkan oleh Chollet, telah menjadi salah satu standar industri dalam deteksi *Deepfake*. Penelitian Xception, dengan penggunaan *depthwise separable convolutions*, mampu mencapai akurasi sangat tinggi (mencapai 87.7% hingga 96.2% tergantung dataset) karena kemampuannya memisahkan pemetaan korelasi lintas-kanal dan korelasi spasial [27]. Namun, model-model besar seperti Xception dan ResNet sering kali memiliki jumlah parameter yang masif, yang berimplikasi pada kebutuhan sumber daya komputasi yang besar dan waktu inferensi yang lebih lama. Di sinilah arsitektur *EfficientNet*

menawarkan paradigma baru. Tan dan Le memperkenalkan *EfficientNet* yang menggunakan *compound scaling* untuk menyeimbangkan kedalaman, lebar, dan resolusi jaringan[24]. Studi oleh Muntina yang menerapkan *EfficientNet-B6* menunjukkan akurasi 90% dengan presisi tinggi [21]. Lebih lanjut, penelitian lain menegaskan bahwa varian *EfficientNet-B0* yang lebih ringan pun mampu memberikan kinerja kompetitif untuk klasifikasi citra medis kompleks, yang memiliki karakteristik serupa dengan deteksi artefak *deepfake* (fitur mikroskopis pada tekstur) [28]. Penelitian menggunakan arsitektur *EfficientNet-B0* memberikan keseimbangan yang optimal antara performa deteksi dan efisiensi komputasi, yang menjadi variabel kunci dalam implementasi sistem deteksi *real-time* [29].

Meskipun CNN murni sangat kuat dalam mengekstraksi fitur spasial per-*frame*, video memiliki dimensi temporal yang kaya informasi. Petmezas et al. (2025) mengusulkan arsitektur hibrida CNN-LSTM (*Long Short-Term Memory*) untuk menangkap inkonsistensi antar-*frame*, seperti *flickering* pada area wajah. Hasil penelitian menunjukkan AUC sebesar 90.82% pada dataset VoxCeleb2.1 Serupa dengan itu, menggunakan Bi-LSTM untuk meningkatkan deteksi pada video *FaceForensics++*. Meskipun pendekatan ini menawarkan akurasi tinggi, kompleksitas komputasinya meningkat secara eksponensial, sering kali membuatnya kurang praktis untuk aplikasi *real-time* pada perangkat dengan sumber daya terbatas[30].

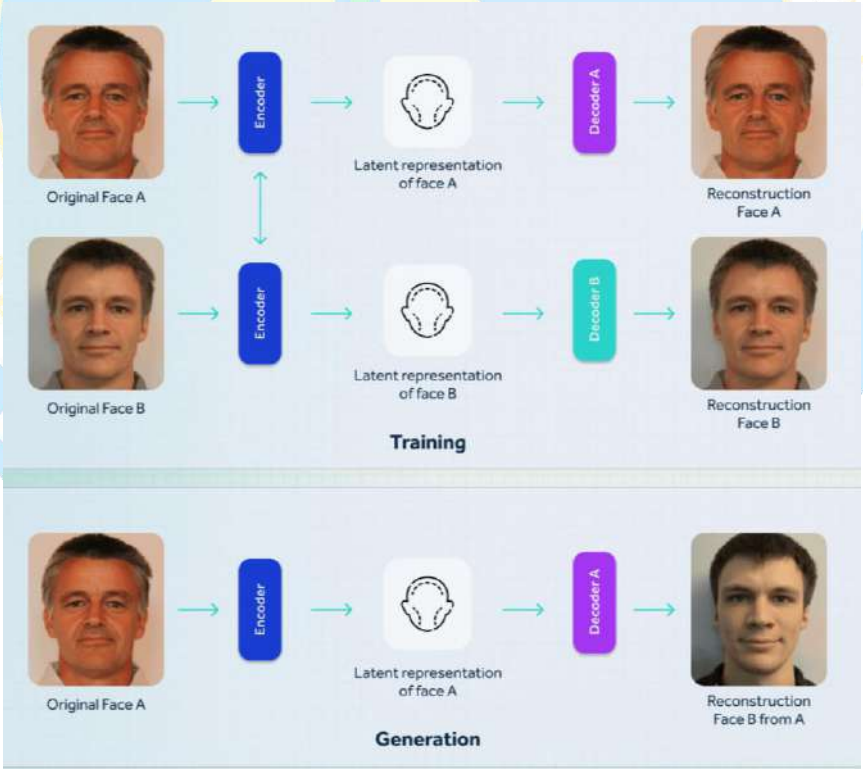
Berdasarkan tinjauan di atas, meskipun metode deteksi *deepfake* berbasis *deep learning* telah mencapai tingkat akurasi yang signifikan, sebagian besar penelitian saat ini masih menggunakan arsitektur model yang kompleks dan membutuhkan sumber daya komputasi tinggi, seperti *Xception* dan *ResNet-50*. Hal ini menyebabkan kesulitan dalam implementasi sistem secara *real-time* pada perangkat dengan spesifikasi terbatas. Penelitian ini hadir untuk mengisi celah tersebut dengan mengimplementasikan model *EfficientNet-B0* yang dirancang melalui teknik *compound scaling* untuk menyeimbangkan kedalaman, lebar, dan resolusi jaringan secara efisien. Penggunaan *EfficientNet* diharapkan mampu memberikan akurasi yang kompetitif namun dengan beban komputasi (FLOPs) yang jauh lebih rendah dibandingkan model konvensional.

B. Landasan Teori

1. Deepfake

Deepfake sebuah istilah yang merupakan etimologi dari ”*Deep learning”* dan ”*Fake”* atau palsu. *Deepfake* merupakan sebuah teknologi yang menggunakan *artificial intelligence* untuk memanipulasi suara, foto, maupun video dan menghasilkan video hiper-realistis yang dimanipulasi secara digital untuk menampilkan orang melakukan dan mengatakan sesuatu yang sebenarnya tidak pernah terjadi [33].

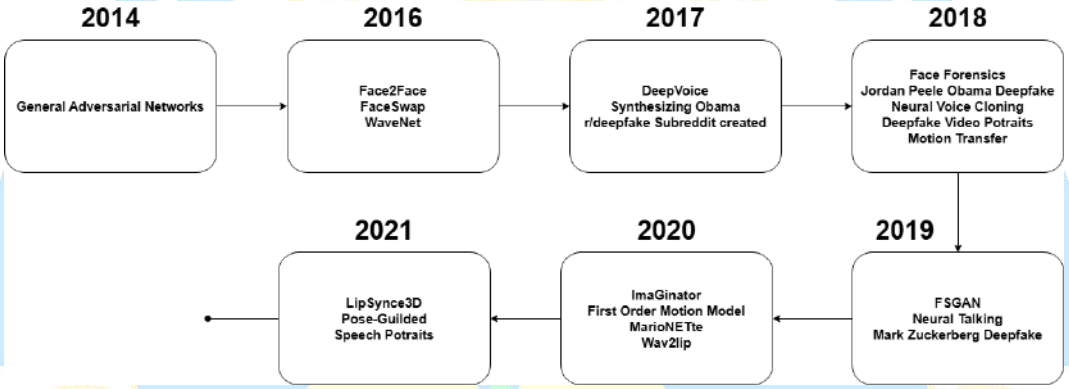
Deepfakes bergantung pada jaringan saraf yang menganalisis kumpulan data yang besar untuk belajar meniru ekspresi wajah, perilaku, suara, dan intonasi seseorang. Prosesnya melibatkan pemberian rekaman dua orang ke dalam algoritma *deep learning* untuk melatihnya menukar wajah [34]. Dengan kata lain, *deepfake*s menggunakan teknologi pemetaan wajah dan AI yang menukar wajah seseorang dalam video dengan wajah orang lain [35].



Gambar 1. Mekanisme Pembuatan Konten *Deepfake*

Sumber : ([36])

*Deepfake*s mulai dikenal publik pada tahun 2017 ketika seorang pengguna Reddit memposting video yang menampilkan selebriti dalam situasi seksual yang memalukan [3].*Deepfake*s sulit dideteksi, karena menggunakan rekaman asli, dapat memiliki audio yang terdengar autentik, dan Gambar hiper-realistis yang dihasilkan oleh AI serta video yang dimanipulasi, dikombinasikan dengan pengiriman cepat untuk platform media sosial, memberikan penipuan, propaganda, dan disinformasi kekuatan baru yang tidak menyenangkan [37]. Dengan demikian, banyak orang mengira bahwa video yang mereka lihat adalah nyata.

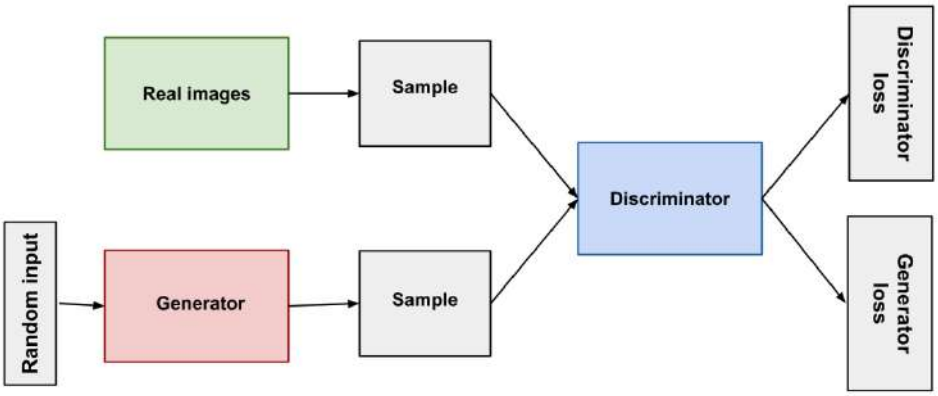


Gambar 2. Perkembangan teknologi *Deepfake* dari tahun ke tahun

a. GANs

Deepfake adalah produk dari *Generative Adversarial Networks* (GANs). *Generative Adversarial Networks* (GANs) merupakan kelas jaringan saraf dalam yang kuat yang semakin banyak digunakan untuk membuat konten *deepfake*, seperti gambar dan video palsu [38].

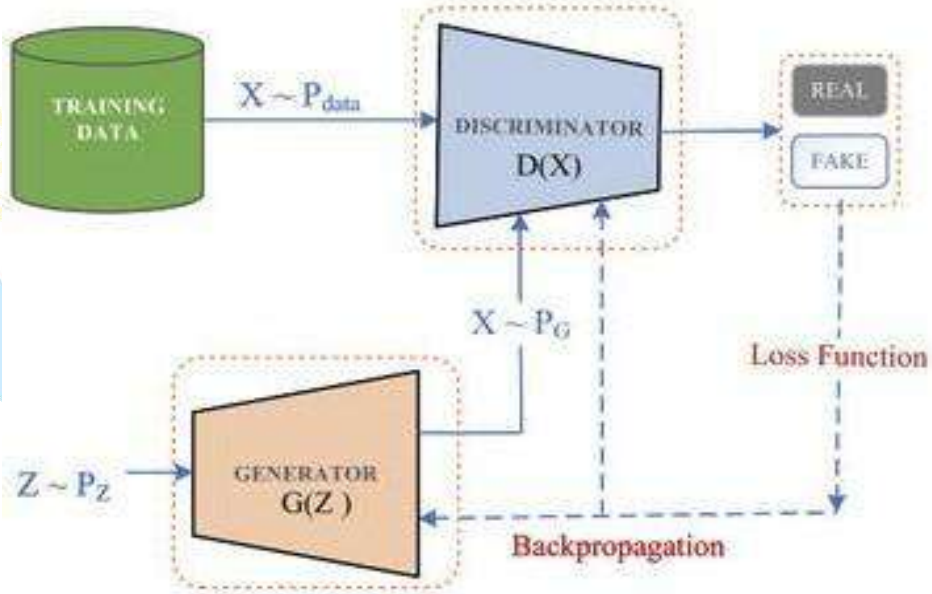
Arsitektur GAN yang khas mencakup dua komponen utama: generator dan discriminator. Generator membuat sampel data baru, sedangkan discriminator menilai terhadap data nyata untuk membedakan keasliannya. Sepanjang proses pelatihan, generator berusaha menipu discriminator, yang pada gilirannya beradaptasi untuk lebih baik dalam mengidentifikasi data palsu. Dengan cara itu, kedua komponen ini saling mendorong untuk meningkat [39].



Gambar 3. Struktur GANs

Sumber : ([40])

Diskriminator dan generator adalah jaringan saraf. Input diskriminator terhubung langsung ke output generator, dan klasifikasi diskriminator memberikan sinyal yang digunakan generator untuk memperbarui bobotnya melalui backpropagation [40]. Alur kerja dari algortima GANs dapat dilihat pada gambar (4) dibawah ini:



Gambar 4. Alur Kerja GAN

Sumber : ([38])

Optimisasi matematis *minmax* (G*) dari *neural network* G dan D dapat diukur dengan persamaan (1) dibawah ini.

$$G^* \in \arg\min\max V\ (G, D)$$
$$= \arg\min\max\ E_{x \sim P_{data}(X)}\ [\log\ (D(X))] + E_{Z \sim P_Z(Z)}\ [\log(D(G(Z)))] \quad (1)$$

Keterangan :

G = Generator

D = Discriminator

x = Sampel data

z = Vektor noise acak

Ex = Ekspektasi Log Probabilitas jika data asli

Ez = Ekspektasi Log Probabilitas jika data palsu

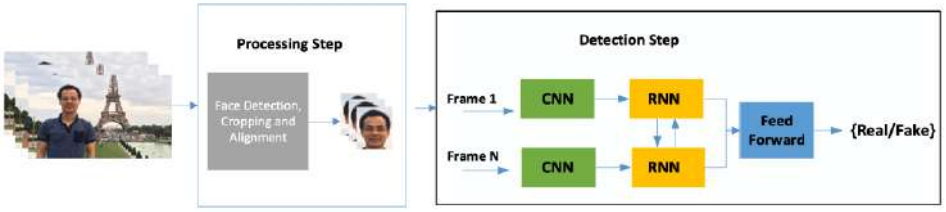
Dimana Z adalah input untuk *generator* G(Z) dengan probabilitas distribusi Pz dan *return* X dengan distribusi probabilitas Pg. *Discriminator* D(X) mengestimasi probabilitas bahwa X adalah dari distribusi pelatihan data Pdata. Akhir-akhir ini banyak macam GAN, seperti DCGAN, WGAN, PGGAN, BigGAN dan Style-GAN. Macam-macam GAN ini dibuat untuk mengimprovisasi *design, loss* dan teknik pelatihan [38].

b. Deteksi *Deepfake*

Dengan perkembangan Teknologi *deepfake,* penelitian untuk mendeteksi video dan gambar *deepfake*, telah menjadi semakin penting dan krusial. Untuk mendorong para peneliti, banyak organisasi seperti *United States Defense Advanced Research Projects Agency* (DARPA), Facebook Inc, dan Google meluncurkan inisiatif penelitian dalam upaya mendeteksi dan mencegah *deepfake* [41]. Penelitian terkait deteksi *deepfake* menunjukkan bahwa *Deep Neural Network* menghasilkan hasil yang luar biasa dalam mendeteksi berita palsu dan rumor di unggahan media sosial [42]. Deteksi video *deepfake* dapat diklasifikasikan menjadi tiga kategori berdasarkan pendekatannya: Metode *Feature-Based*

(Handcrafted Features), Deep learning-Based Model, dan *Hybrid Methods (Hybrid Feature-Based* dan *Deep learning Approaches)* [19].

Akibatnya, banyak pendekatan menggunakan *deep leraning*, termasuk *long short-term memory (LSTM), recurrent neural network (RNN)* dan bahkan penggunaan metodd *hybrid*, telah diusulkan untuk mendeteksi gambar dan video *deepfake*, mendorong lebih banyak penelitian di bidang ini. Studi ini menunjukkan bahwa *deep neural networks* dapat mendeteksi berita palsu dan rumor di postingan media sosial dengan sangat baik. Deteksi *deepfake* menggunakan metode pembelajaran mendalam seperti *recurrent neural network* (RNN), *Convolutional Neural Network* (CNN), dan *long short-term memory* (LSTM) [43].



Gambar 5. Proses mendeteksi manipulasi wajah dengan CNN dan RNN

Sumber : ([44])

2. DeepLearning

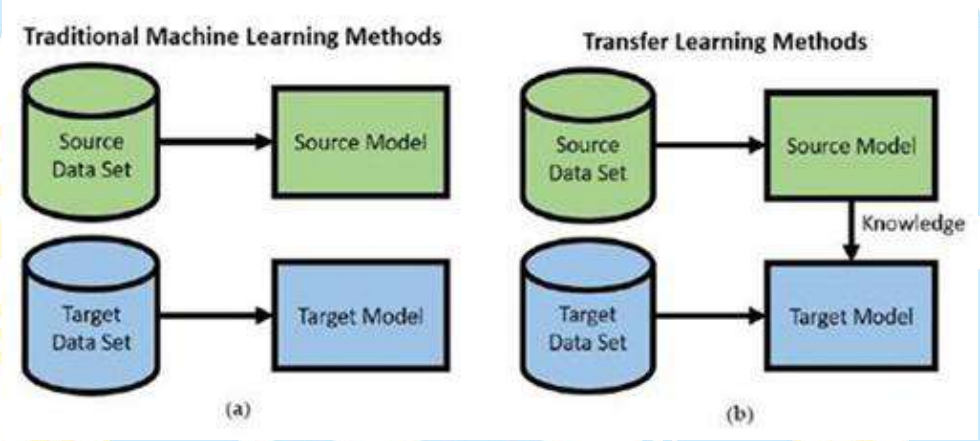
Deep learning adalah cabang komputasi dalam *machine learning* yang menggunakan arsitektur jaringan saraf dalam (*deep neural networks*) untuk memproses data dalam jumlah besar dan kompleks [45]. Di antara berbagai algoritma dari *Machine Learning*, *Deep learning* sangat umum digunakan. Nama lain dari *Deep learning* adalah Reinforcement learning. Disebabkan oleh pertumbuhan yang tidak terduga dalam kemampuan memperoleh data serta kemajuan luar biasa yang dicapai dalam teknologi perangkat keras, seperti *High Performance Computing* (HPC), semakin banyak penelitian baru di bidang pembelajaran mendalam dan terdistribusi [46].

Deep transfer learning berfokus pada transfer pengetahuan dari sumber ke target secara berurutan. Secara umum, pendekatan DTL dapat dikategorikan menjadi empat, yaitu *instance-based, feature-based, model-based,* dan *adversarial-*

based, dengan *model-based (*seperti *fine-tuning, freezing* CNN *layers,* dan *progressive learning)* menjadi yang paling banyak digunakan karena fleksibilitasnya dalam menyesuaikan parameter jaringan [44].

3. Transfer learning

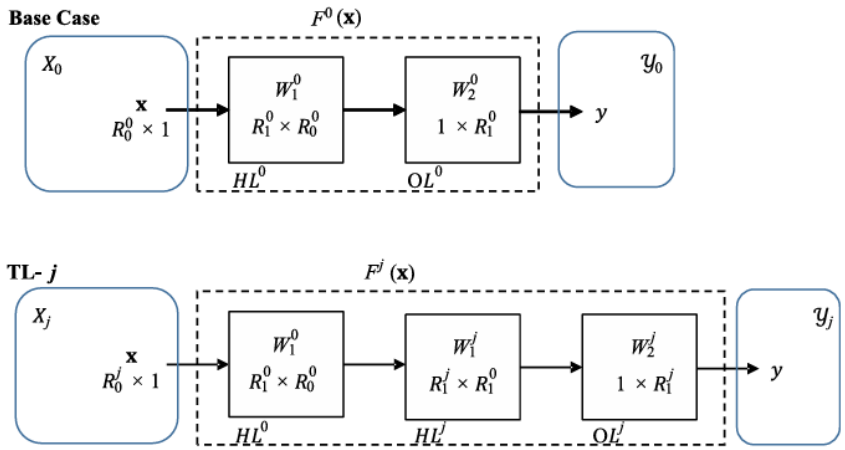
Tidak semua orang memiliki sumber daya yang mereka inginkan karena kelangkaan data, sulit diakses, harga tidak logis, dan sulit dikompilasi [48]. *Transfer learning* mengatasi perbedaan tersebut dengan membagikan pengetahuan antar sistem pembelajaran, sehingga menawarkan teori yang pada dasarnya berbasis pada perbedaan distribusi, dan dengan demikian, memberikan jalan menuju mayoritas kasus penggunaan [49].



Gambar 6. Ilustrasi metode tradisional ML vs *Transfer learning*

Sumber : ([50])

Sebagai hasilnya, *transfer learning* atau yang sering disebut sebagai transfer pengetahuan, yaitu sebuah teknik untuk memanfaatkan keahlian eksternal guna mentransfer pengetahuan sebagai proses pembelajaran antar tugas spesifik [48]. *Transfer learning* (TL) adalah konsep dalam pembelajaran mesin yang bertujuan untuk meningkatkan kemampuan model dalam melakukan generalisasi dengan menggunakan pengetahuan dari tugas atau domain yang terkait [50]. *Transfer learning* fokus pada domain dan tugas, dan mengabaikan perspektif yang ditawarkan secara eksplisit mempertimbangkan struktur dan perilaku sistem.



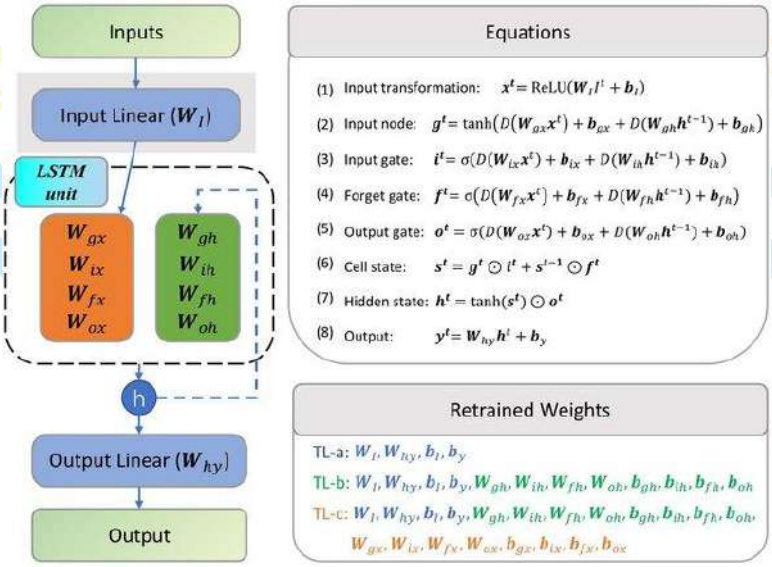
Gambar 7. Skema *neural network* dengan *transfer learning*

Sumber : ([52])

Respon dari model pengganti dapat dinyatakan sebagai:

$$F(x) = y_t = + \varepsilon,$$

di mana $y_t \in \Upsilon$, adalah output nyata, F(x) adalah pendekatan fungsi objektif pada $x \in \chi$ dan ε adalah kesalahan antara output nyata dan output yang diprediksi. HL^j: *hidden layers* dan OL^j: *ouput layers* dari urutan jth *transfer learning*. j-0 untuk *machine learning* tradisional (*base task*) [52].



Gambar 8. Arsitektur LSTM dengan opsi *transfer learning*

Sumber : ([53])

Arsitektur *Long Short-Term Memory* (LSTM) dapat dikembangkan dengan pendekatan *transfer learning* (TL) melalui beberapa opsi, yaitu TL-a, TL-b, dan TL-c. Ketiga opsi tersebut berbeda dalam jumlah bobot (*weights*) yang diizinkan untuk dituning ulang selama proses pelatihan dengan data target, setelah terlebih dahulu dilakukan *pre-training* menggunakan data sumber. Semakin banyak bobot yang dapat disesuaikan, semakin fleksibel model dalam beradaptasi dengan karakteristik tugas yang lebih spesifik [53].

a. *Feature Extraction*

Feature extraction (ekstraksi fitur) pada *EfficientNet* adalah proses menggunakan bagian dari arsitektur jaringan saraf konvolusional (CNN) *EfficientNet* yang telah dilatih sebelumnya (biasanya pada dataset ImageNet) untuk menghasilkan representasi fitur dari data gambar baru [70].

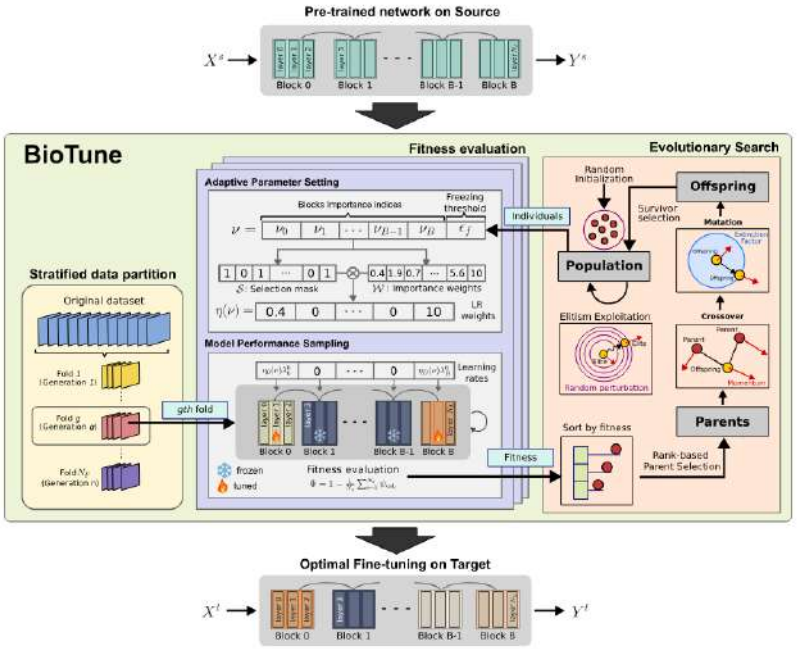
Feature extraction digunakan untuk mendapatkan fitur gambar dasar, seperti tepi dan gumpalan, dari lapisan awal jaringan, fitur ekstraksi fitur digunakan. Setelah tahap ini selesai, pemodelan dilakukan menggunakan arsitektur *EfficientNet-B0*. Setelah data gambar didapatkan, selanjutnya melakukan tahapan *feature extraction* menggunakan dataset ImageNet21k. Model dilatih sebelumnya menggunakan dataset ImageNet21k [71].

b. *Fine-Tuning*

Transfer learning dengan *EfficientNet* umumnya melibatkan penyesuaian model yang telah dilatih sebelumnya pada dataset baru, seringkali yang lebih kecil, untuk meningkatkan kinerja sambil mengurangi waktu pelatihan dan biaya komputasi. [65]. *Fine-tuning* sering melibatkan pemuatan bobot pra-latih ImageNet, penambahan lapisan atas yang spesifik tugas, dan kemudian melatih lapisan yang dipilih dengan penyesuaian hiperparameter yang cermat, termasuk laju pembelajaran dan optimizer seperti Adam [72].

Tiga lapisan terakhir dari *EfficientNet-B0* dimodifikasi untuk pembelajaran transfer agar mencapai hasil yang lebih baik dibandingkan model *EfficientNet-B0* asli. Sebuah jendela konvolusi 2D digunakan. Sebuah lapisan *global average pooling* (GAP) 2D, lapisan *dropout* dengan tingkat

0,5, dan lapisan dense dengan aktivasi *Softmax* diterapkan dalam arsitektur *EfficientNet-B0* yang telah disesuaikan [67]. Secara keseluruhan, model *EfficientNet* yang telah disesuaikan menawarkan keseimbangan antara akurasi, efisiensi komputasi, dan skalabilitas, menjadikannya cocok untuk penerapan real-time dalam aplikasi medis dan spesialis lainnya [73].

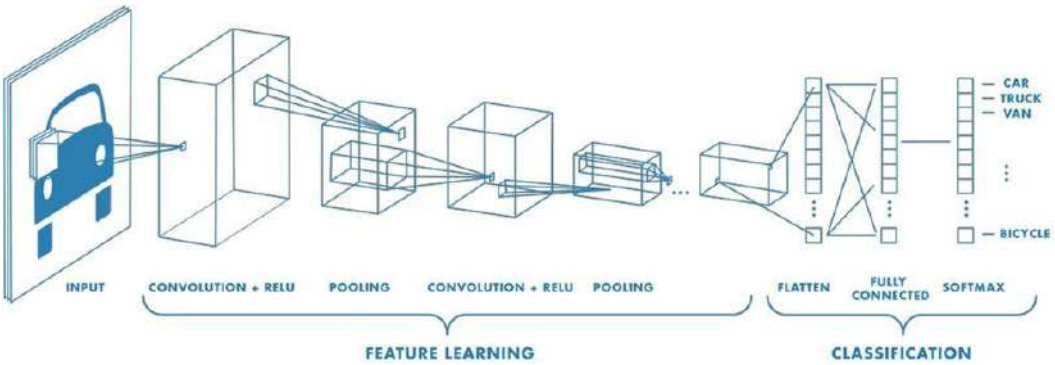


Gambar 9. Alur *Fine-tuning*

Sumber : ([74])

4. CNN

Convolutional Neural Networks (CNN) telah sepenuhnya mendominasi bidang *computer vision* dalam beberapa tahun terakhir [54]. CNN terdiri dari lapisan input, lapisan output, serta beberapa *layers* tersembunyi. *Layers* tersembunyi dari CNN biasanya terdiri dari, *convolutional layers, pooling layers, fully connected layers and normalisation layers (*ReLU). Lapisan tambahan dapat digunakan untuk model yang lebih kompleks [55].

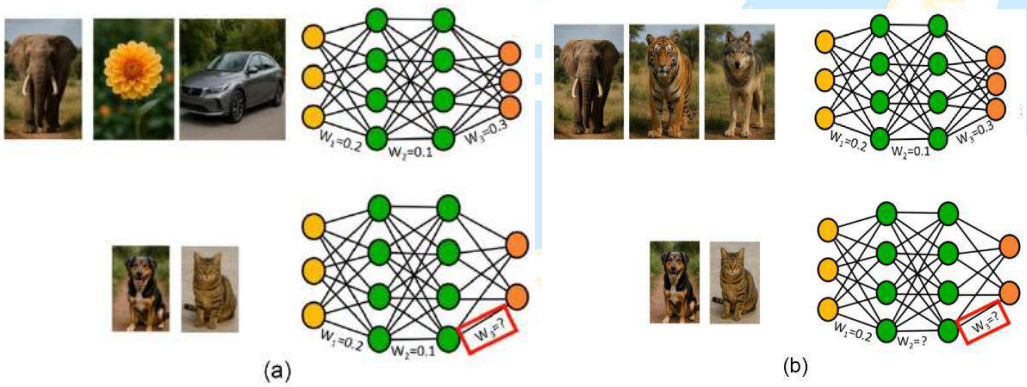


Gambar 10. Arsitektur CNN

Sumber : ([56])

TensorFlow memfasilitasi *transfer learning* dari representasi CNN yang telah dilatih sebelumnya. Dengan menggunakan *transfer learning*, topologi model CNN diteliti untuk tugas klasifikasi citra. Para peneliti harus menguji dan memodifikasi topologi jaringan serta atribut dataset untuk menentukan variabel-variabel yang memengaruhi akurasi klasifikasi [54].

Salah satu tujuanpre utama dari *transfer learning* CNN dari data yang telah dilatih ke dataset baru (*transfer knowledge*) dengan bentuk berbeda yang memiliki kesamaan fitur lokal maupun global dengan dataset pelatihan, selain itu juga membantu CNN dalam melakukan tugas klasifikasi [57].



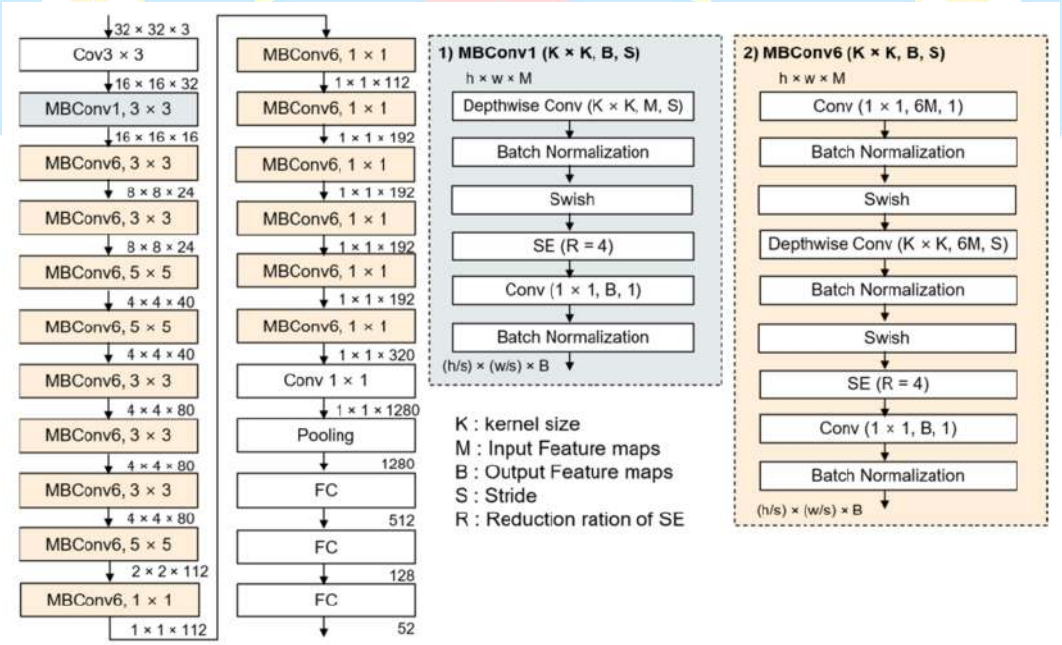
Gambar 11. (a) Arsitektur CNN Tradisional, (b) *Pre-Trained* CNN

5. EfficientNet

EfficientNet merupakan keluarga *Convolutional Neural Networks* (CNN) yang dirancang dengan metode *compound scaling* baru, *EfficientNet* memanfaatkan metode *compound scalin*g dengan mengelola peningkatan dimensi jaringan pada

untuk mencapai akurasi yang baik dengan beban komputasi yang rendah. Terdiri dari lebar (*width*), kedalaman (*depth*), dan resolusi (*resolution*) dimana lebar dimensi didasarkan pada jumlah kanal di semua lapisan, kedalaman dimensi didasarkan pada jumlah lapisan, dan resolusi dimensi didasarkan pada ukuran gambar yang digunakan, sehingga menghasilkan peningkatan akurasi dan efisiensi. mencapai akurasi *state-of-the-art* pada *ImageNet* dengan ukuran yang jauh lebih kecil dan kecepatan lebih tinggi, serta mampu ditransfer dengan baik ke dataset lain dengan jumlah parameter yang lebih sedikit [58].

EfficientNet adalah model jaringan baru yang dikembangkan oleh Google yang telah menjadi populer berkat fitur-fiturnya yang ringan, cepat, dan akurat. Model ini tidak hanya menggunakan *mobile inverted bottleneck convolution* (Macon) tetapi juga memperkenalkan mekanisme perhatian dari *Squeeze-and-Excitation Network* (SENet). *Neural architecture search* (NAS) *multi-objective* kemudian digunakan untuk merancang model *EfficientNet-B0* yang sangat efisien, dan *EfficientNet*-B1-B7 dikembangkan dengan menetapkan koefisien yang berbeda. Karena strukturnya serupa, penelitian ini hanya menggunakan *EfficientNet-B0* sebagai *EfficientNet* [59].



Gambar 12. Arsitektur *EfficientNet*

Sumber : ([60])

Arsitektur *EfficientNet* mengintegrasikan *inverted bottleneck residual blocks* dan modul *squeeze-and-excitation (SE)*, modul bangunan untuk CNN yang meningkatkan ketergantungan antar saluran dengan hampir tanpa biaya komputasi. CNN menggunakan filter konvolusinya untuk mengekstrak informasi hierarkis dari gambar. Lapisan bawah menemukan potongan konteks yang sederhana atau bentuk geometris kompleks lainnya. Mengekstrak apa pun yang diperlukan untuk menyelesaikan tugas secara efisien yang meningkatkan kinerja melalui perbaikan *feature recalibration* [63]. Struktur lengkap *EfficientNet-B0* dapat diringkas sebagai berikut:

Tabel 1. Struktur lengkap *EfficientNet-B0*

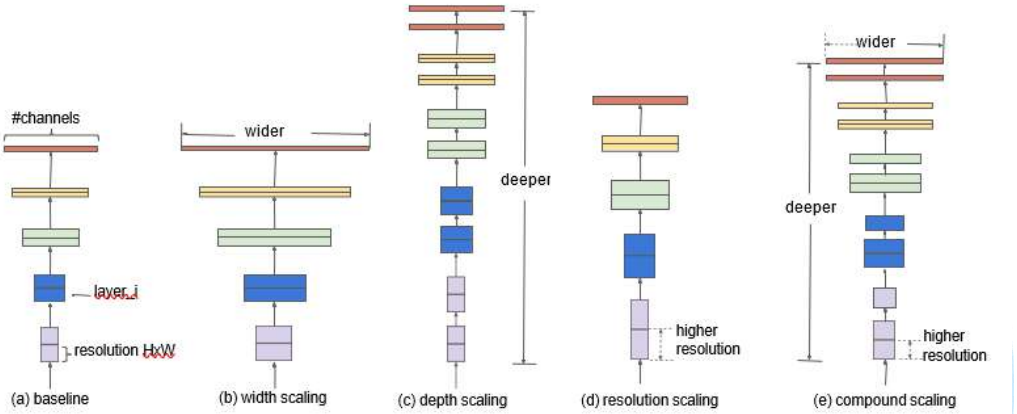
Tahap i	**Jenis Blok** F_i	**Ukuran Kernel**	**Resolusi Output** $H_i \times \hat{}Wi$	***Channels*** C_i	**Jumlah Blok** L_i
Stem	Conv	3x3	224x224	32	1
Stage 1	Conv2D + BN + Swish	3×3	112×112	16	1
Stage 2	MBConv1	3×3	112×112	24	2
Stage 3	MBConv6	5×5	56×56	40	2
Stage 4	MBConv6	3×3	28×28	80	3
Stage 5	MBConv6	5×5	14×14	112	3
Stage 6	MBConv6	5×5	14×14	192	4
Stage 7	MBConv6	3×3	7×7	320	1
Stage 8	Conv2D + Pool	-	7×7	1280	1
Head	FC	-	1×1	1000	1

a. Compound Scaling

Gagasan utama dari *EfficientNet* adalah memanfaatkan strategi penskalaan gabungan yang baru untuk secara seragam meningkatkan lebar, kedalaman, dan resolusi jaringan, dengan tujuan memperoleh hasil klasifikasi yang lebih baik ketika lebih banyak sumber daya tersedia. Strategi penskalaan

model ini ditampilkan pada Gambar (13), mengilustrasikan contoh jaringan dasar dengan lebar, kedalaman, dan resolusi yang lebih kecil. Di antara H dan W menunjukkan bentuk tensor input. *EfficientNet* menggunakan koefisien gabungan φ untuk secara konsisten menyesuaikan lebar, kedalaman, dan resolusi jaringan dengan cara sebagaimana dijelaskan oleh Tan dan Le [58].

Dimulai dengan *EfficientNet-B0* sebagai baseline dan menggunakan metode penskalaan gabungan dapat meningkatkan jaringan ke berbagai varian yang lebih besar dan unggul [62].



Gambar 13. Struktur *Compound Scaling*

Sumber : ([58])

Metode *compound scaling* adalah inti dari *EfficientNet* yang membedakannya dari ConvNets lain. Metode ini menskalakan tiga dimensi jaringan—kedalaman (*depth*), lebar (*width*), dan resolusi (*resolution*)—secara seragam dengan koefisien *compound* ϕ (phi) [58]. Formulasi matematisnya dapat diukur dengan persamaan dibawah ini.

$$\begin{aligned} \text{Depth,} \quad & d = \alpha^{\varphi} \\ \text{Width,} \quad & w = \beta^{\varphi} \\ \text{Resolution,} \quad & r = \gamma^{\varphi} \\ \textbf{s.t.} \quad & \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2 \\ & \alpha \geq 1, \beta \geq 1 \ \gamma \geq 1 \end{aligned} \tag{2}$$

dengan kendala $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$ di mana:

- α (alpha) = mengatur skala kedalaman (jumlah lapisan),
- β (beta) = mengatur skala lebar (jumlah kanal),
- γ (gamma) = mengatur skala resolusi (ukuran gambar input),
- ϕ (phi) = koefisien untuk mengontrol total sumber daya komputasi.
- d (depth) = kedalaman
- w (width) = lebar
- r (resolution) = resolusi.

Di sini kita hanya perlu menyesuaikan parameter ini untuk mencapai penskalaan model. Untuk operasi konvolusi, FLOPS (operasi floating point per detik) umumnya sebanding dengan d, w^2, r^2. Jika depth meningkat 2 kali lipat, maka kebutuhan komputasi juga meningkat 2 kali lipat, tetapi jika width dan resolution meningkat 2 kali lipat, maka kebutuhan komputasi meningkat 4 kali lipat [58].

Tabel 2. *Compound Scale* Pada *EfficientNet-B0*

Model	*Width_scale*	*Depth_scale*	*Resolution_scale*
EfficientNet-B0	1.00	1.00	1.00

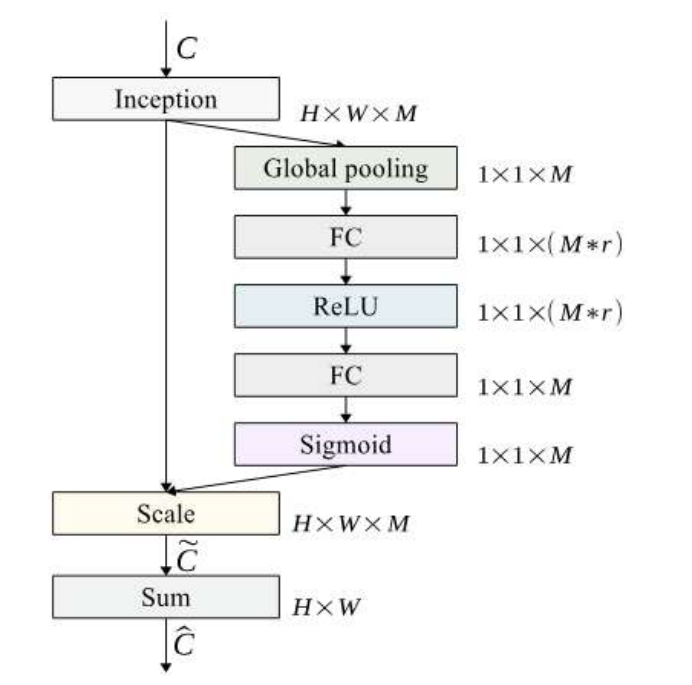
Untuk model baseline, parameternya $\varphi_0 = 1$. Ketika parameter baru digunakan untuk penskalaan model, FLOPS model akan menjadi $(\alpha \cdot \beta^2 \cdot \lambda^2)^\varphi$ kali lipat dari FLOPS model baseline. Karena batasan $\alpha \cdot \beta^2 \cdot \lambda^2 \approx 2$, FLOPS akan meningkat kira-kira sebesar 2^φ [58].

b. Mobile Inverted Bottleneck Convolution (MBConv)

Mobile Inverted Bottleneck Convolution disingkat MBConv adalah dasar arsiktur diperoleh melalui pencarian arsitektur *neural network*. Struktur modulnya mirip dengan konvolusi terpisah berdasarkan *depthwise* [65]. MBConv pertama-tama melakukan konvolusi 1 × 1 *point-by-point* pada masukan dan berdasarkan rasio perluasan (expand ratio) untuk mengubah dimensi saluran keluaran (misalnya, ketika rasio perluasan adalah 3, dimensi saluran akan meningkat 3 kali. Namun, jika rasio perluasan adalah 1, maka langsung melewati konvolusi 1 × 1 *point-by-point* serta normalisasi batch dan

fungsi aktivasi berikutnya). Kemudian dilanjutkan dengan konvolusi depthwise k × k. Kemudian memulihkan dimensi saluran asli di akhir konvolusi 1x1 *point-by-point*. Terakhir, dilakukan drop connect dan *skip connection* pada masukan. Ketika modul SE dilampirkan, operasi akan dilakukan setelah konvolusi depthwise k × k.

c. *Squeeze and Excitation (Global Average Pooling & Fully Connected)*



Gambar 14. Skema *Squeeze-and-Excitation Blocks*

Sumber : ([63])

Squeeze-and-Excitation (SE) Optimization adalah teknik yang diintegrasikan dalam *EfficientNet* untuk meningkatkan representasi fitur melalui pemodelan interdependensi antar saluran. Keuntungan utama dari optimasi SE adalah kemampuannya untuk meningkatkan sensitivitas model terhadap fitur-fitur yang berkorelasi dengan kelas target [64]. *EfficientNet* dengan optimasi SE mampu membedakan antara video asli dan *Deepfake* dengan lebih baik karena kemampuannya untuk fokus pada perbedaan halus dalam tekstur, warna, dan pola gerakan yang sering menjadi penanda manipulasi. Penggunaan optimasi SE dalam arsitektur deteksi *Deepfake* telah

terbukti meningkatkan akurasi deteksi sambil mempertahankan efisiensi komputasi [54]. *Squeeze-and-Excitation* (SE) Optimization bekerja dengan 2 tahap utama: *Squeeze* (penyempitan ruang informasi) dan *Excitation* (pemberian bobot adaptif(*Weight)* pada *channel*).

$$GAP = \frac{1}{H.W} + \sum_{i=1}^{H} \sum_{j=1}^{W} X_c\ (i,j) \tag{3}$$

Keterangan :

- H : Tinggi
- W : Lebar
- C : Jumlah *Channel*
- GAP : Nilai Presentasi *Global Channel*

Pada tahapan *squeeze* setiap *channel* dipadatkan menjadi satu nilai dengan *global average pooling*.

$$s = \sigma\ (W_2 \cdot\ \delta\ (W_1 \cdot\ z)) \tag{4}$$

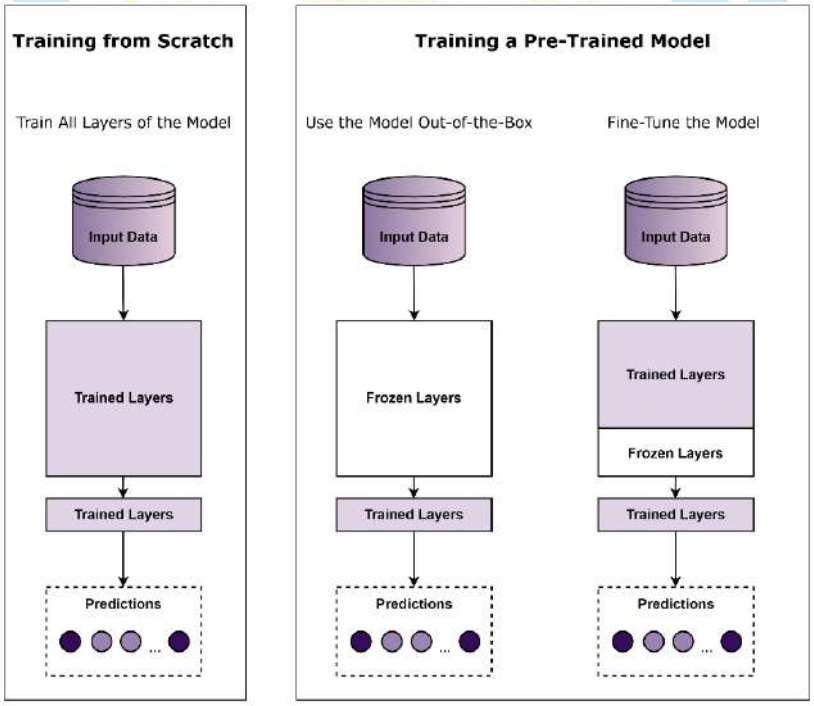
Keterangan :

- $W_1 \in R\frac{c}{r}\ x\ C$: Matriks reduksi
- $W_2 \in R\,C\ x\ \frac{c}{r}$: Matriks ekspansi
- r : Ratio
- δ : Aktivasi ReLU
- σ : Sigmoid
- $s \in R^C$: Vektor Skala (Weight) tiap *channel*

Selanjutnya tahap *Excitation* menghasilkan bobot adaptif (*weight)* antar *channel*.

d. Pre-Trained

Transfer learning dengan model *EfficientNet* yang telah dilatih sebelumnya melibatkan penggunaan bobot yang dipelajari dari dataset besar seperti ImageNet untuk meningkatkan kinerja pada dataset target baru yang seringkali lebih kecil [65]. Pendekatan ini biasanya dimulai dengan memuat bobot *EfficientNet* yang telah dilatih sebelumnya, kemudian menyesuaikan model dengan menambahkan lapisan atas baru atau head klasifikasi baru yang sepenuhnya terhubung yang disesuaikan dengan tugas yang lebih spesifik.



Gambar 15. Model *Pre-Trained*

Sumber : ([66])

Penyesuaian model *EfficientNet* yang telah dilatih sebelumnya, telah menunjukkan akurasi tinggi dan ketahanan dalam tugas pemrosesan citra, sambil tetap efisien secara komputasi dan ringan [67]. Penambahan data dan penyesuaian hiperparameter yang cermat lebih lanjut meningkatkan generalisasi dan kinerja model selama *transfer learning* [68]. Arsitektur yang *scaleable* dan bobot yang telah dilatih sebelumnya dari *EfficientNet* menjadikannya tulang punggung yang kuat untuk *transfer learning* di

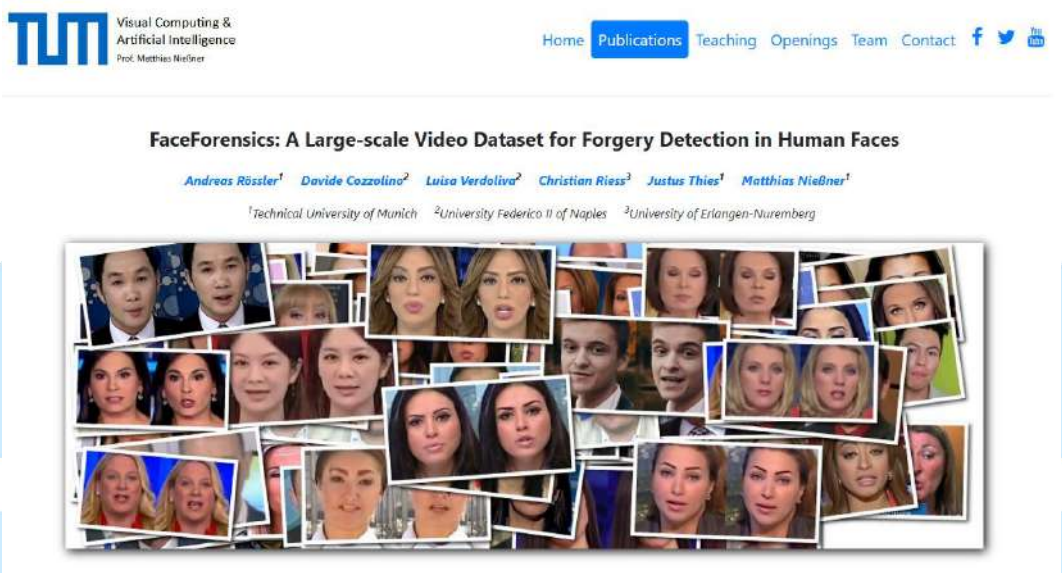
berbagai domain, termasuk pemantauan jarak jauh dan klasifikasi gambar maritim, di mana ia outperform arsitektur lain dengan parameter yang lebih sedikit dan pelatihan yang lebih cepat [69].

e. Head Classifier

Transfer learning dengan *EfficientNet* sering melibatkan penggantian kepala klasifikasi asli dengan kepala klasifikasi baru yang spesifik untuk tugas tertentu guna menyesuaikan model dengan domain target. Pendekatan umum adalah linear probing, di mana tulang punggung *EfficientNet* yang telah dilatih sebelumnya dibekukan dan hanya kepala klasifikasi baru yang dilatih, yang secara komputasi efisien tetapi mungkin berkinerja lebih rendah dibandingkan dengan *fine-tuning* penuh [75]. Metode yang lebih canggih seperti memanfaatkan fitur dari lapisan-lapisan model yang telah dilatih sebelumnya untuk melatih kepala klasifikasi baru, mencapai kinerja yang setara dengan *fine-tuning* penuh sambil secara signifikan mengurangi biaya pelatihan dan meningkatkan generalisasi di luar distribusi [75]. Penelitian lain menggunakan varian *EfficientNet* dengan kepala klasifikasi baru yang ditambahkan telah menunjukkan akurasi tinggi, seringkali ditingkatkan dengan metode ensembel atau mekanisme penolakan untuk meningkatkan keandalan [76]. Menyesuaikan kepala klasifikasi dengan lapisan padat tambahan, *dropout*, atau mekanisme perhatian dapat lebih meningkatkan kinerja dan mengurangi *overfitting*, seperti yang ditunjukkan dalam aplikasi seperti klasifikasi buah dan penentuan tingkat tumor otak [77]. Secara keseluruhan, merancang kepala klasifikasi baru di atas ekstraktor fitur *EfficientNet* merupakan strategi *transfer learning* yang fleksibel dan efektif yang menyeimbangkan akurasi, efisiensi, dan adaptabilitas untuk tugas-tugas yang beragam [78].

6. Faceforensics++

FaceForensics++ adalah dataset berskala besar yang banyak digunakan untuk menguji dan mengembangkan system pendeteksi gambar dan video wajah yang dimanipulasi, terutama *deepfake*. Dataset ini telah menjadi landasan utama untuk mengevaluasi dan melatih algoritma deteksi *deepfake*, mendorong kemajuan dalam penelitian [79].



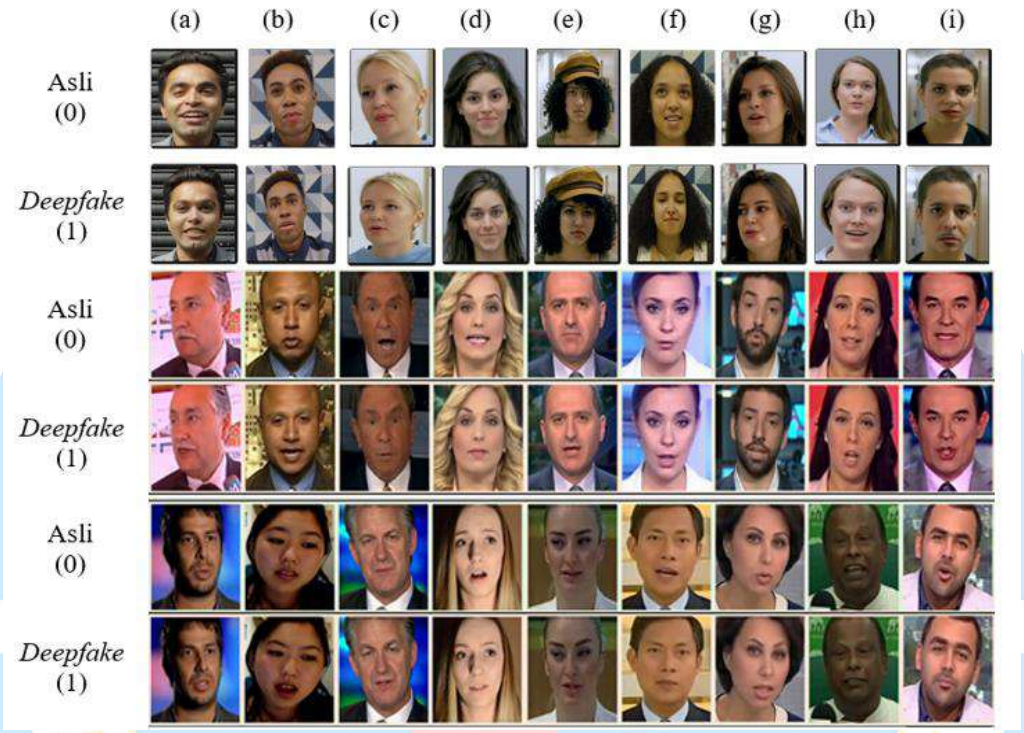
Gambar 16 . Halaman utama *Faceforensics++*

Sumber : ([80])

FaceForensics++ berisi lebih dari 1.000 video YouTube asli dan lebih dari 1,8 juta gambar yang dimanipulasi, dihasilkan menggunakan empat teknik pemalsuan wajah yang terkenal: *Deepfakes*, Face2Face, FaceSwap, dan NeuralTextures [81]. Dataset ini mencakup fps yang cenderung beragam untuk mensimulasikan skenario dunia nyata dan menantang model deteksi dengan kualitas video yang bervariasi. Skala dan keragamannya menjadikannya standar untuk melatih dan mengevaluasi sistem deteksi *deepfake* [82].

Model-model terbaru berfokus pada pengurangan kompleksitas komputasi untuk aplikasi real-time dan penerapan pada perangkat dengan sumber daya terbatas, mencapai akurasi tinggi dengan persyaratan memori dan pemrosesan yang lebih rendah [83]. *FaceForensics++* telah menjadi standar emas dalam penelitian

deteksi *deepfake*, memfasilitasi kemajuan cepat dalam pengembangan model dan pengujian kinerja. Meskipun akurasi deteksi tinggi dalam dataset, tantangan yang terus berlanjut meliputi peningkatan generalisasi antar-dataset, ketahanan terhadap kompresi, dan efisiensi untuk implementasi di dunia nyata [84].



Gambar 17. Sample wajah dari Dataset *FaceForensics++*

7. Python

Python, yang dikembangkan oleh Guido van Rossum dan diperkenalkan pada tahun 1991, adalah bahasa pemrograman tingkat tinggi yang inovatif yang menekankan keterbacaan kode dan efisiensi pengembang. Sintaksnya yang elegan dan pengetikan dinamis mempermudah pengembangan perangkat lunak, menjadikannya pilihan ideal baik untuk pemula maupun ahli. Python adalah bahasa pemrograman yang serbaguna, tingkat tinggi, dan dinamis yang dihargai karena aplikasi tujuan umumnya [85].

Machine learning dan *Deep learning* adalah masa depan, dan merupakan topik yang sering dipakai, sama halnya dengan Python. Akhir-akhir ini, setiap kali istilah Pembelajaran Mesin muncul, Python adalah solusi utama bagi para pengembang, hal ini menjelaskan popularitas Python serta betapa kuatnya bahasa

ini [86]. Python ideal untuk AI, *machine learning* dan *deep learning* karena *tools, frameworks, packages, modules* dan *libraries* yang tersedia secara mudah serta karena seseorang selalu dapat membuatnya sendiri untuk digunakan [86]. Berikut ini adalah contoh *libraries* dan *framework* yang akan diterapkan pada penelitian ini:

a. MTCNN

Multi-task Cascaded Convolutional Networks (MTCNN), *library* Python yang dikenal karena kemampuannya melakukan deteksi wajah, lokalisasi *landmark*, dan penyelarasan wajah secara bersamaan, dievaluasi melalui serangkaian eksperimen [30]. MTCNN dinilai karena kinerjanya, berdasarkan parameter utama, termasuk kemiripan kosinus, tingkat kepercayaan deteksi, waktu pemrosesan, dan metrik akurasi seperti *true positive* dan *false positive*. Hasilnya menunjukkan bahwa MTCNN unggul dalam mengenali wajah frontal dengan akurasi tinggi dan waktu pemrosesan yang cepat, mencapai tingkat kepercayaan deteksi yang sangat baik dan tingkat *false positive* yang rendah [87].

b. OpenCV

OpenCV (*Open Source Computer Vision*), *library* yang awalnya dikembangkan oleh *Intel*, menawarkan serangkaian fungsi pemrograman yang kaya untuk analisis gambar tingkat rendah maupun tinggi. *Library* ini tersedia secara gratis dan kompatibel dengan bahasa seperti C++, Python, dan Java. Arsitektur modular OpenCV terdiri dari modul inti, pemrosesan gambar (imgproc), video, dan pembelajaran mesin [88].

c. PyTorch

PyTorch merupakan pustaka *deep learning open source* yang banyak digunakan karena menggabungkan kemudahan pemakaian dengan kinerja komputasi yang tinggi. Gaya pemrograman dinamis yang bersifat Python serta dukungan kuat terhadap GPU menjadikannya pilihan utama bagi peneliti maupun praktisi dalam membangun, melatih, dan mengimplementasikan jaringan saraf tiruan [84].

PyTorch dirancang untuk mengeksekusi operator secara asinkron di GPU dengan memanfaatkan mekanisme aliran CUDA untuk mengantri pemanggilan kernel CUDA ke FIFO perangkat keras GPU. Hal ini memungkinkan sistem untuk menumpuk eksekusi kode Python di CPU dengan operator tensor di GPU [90]. Secara keseluruhan, PyTorch menonjol berkat model pemrograman yang dinamis dan ramah pengguna, dukungan GPU yang kuat, serta ekosistem yang berkembang pesat. Pustaka ini sangat populer dalam penelitian dan semakin banyak diadopsi dalam lingkungan produksi, karena menawarkan keseimbangan antara fleksibilitas, kinerja, dan skalabilitas [91].

d. Streamlit

Streamlit merupakan kerangka kerja (*framework*) Python sumber terbuka yang dirancang untuk mempermudah pembangunan aplikasi web interaktif, khususnya dalam bidang *data science*, *machine learning*, dan analitik. *Frame*work ini menawarkan antarmuka pemrograman yang intuitif sehingga pengguna dapat membuat *dashboard* maupun alat berbasis data dengan sedikit kode, tanpa memerlukan keterampilan pengembangan *front-end* [92]. Keunggulan utama Streamlit terletak pada kemudahan penggunaan, di mana pengembangan aplikasi web dapat dilakukan hanya dengan beberapa baris kode Python. Hal ini menjadikannya ideal bagi peneliti, pendidik, maupun analis yang tidak memiliki latar belakang pemrograman web [93].

Dari sisi aksesibilitas, aplikasi dapat dijalankan secara lokal, di-*deploy* ke *cloud*, atau untuk meningkatkan reproduktibilitas dan jangkauan [94]. Secara keseluruhan, Streamlit berperan dalam mendemokratisasi pengembangan aplikasi web berbasis data dengan memberikan kemudahan bagi pengguna untuk membangun, membagikan, dan mengimplementasikan alat interaktif. Kesederhanaan, integrasi dengan ekosistem Python, serta cakupan penerapan yang luas menjadikannya pilihan utama untuk prototyping cepat dan berbagi wawasan lintas disiplin [95].

8. Google Colab

Google Colaboratory (Colab) merupakan platform berbasis *cloud* yang disediakan secara gratis untuk menulis dan mengeksekusi kode Python dalam bentuk *Jupyter Notebook*. Colab menyediakan akses ke perangkat keras berperforma tinggi seperti GPU dan TPU, sehingga banyak digunakan dalam bidang *machine learning*, *deep learning*, analisis data, maupun pendidikan karena sifatnya yang mudah diakses dan ramah pengguna.

Keunggulan utama Colab meliputi akses gratis ke GPU (misalnya Tesla P100, T4, K80) dan TPU yang memungkinkan percepatan komputasi tanpa investasi perangkat keras lokal [96]. Namun, Colab memiliki keterbatasan yang perlu diperhatikan. Sesi gratis dibatasi waktu (hingga 12 jam untuk pengguna gratis dan 24 jam untuk Colab Pro), kapasitas RAM terbatas (12–25GB), serta dapat terputus jika tidak aktif. Sumber daya yang tersedia juga tidak selalu konsisten [97].

Google Colab berperan penting dalam mendemokratisasi akses komputasi berperforma tinggi. Dengan lingkungan berbasis peramban, dukungan perangkat keras, serta fitur kolaboratif, Colab menjadi alat yang bernilai bagi mahasiswa, pendidik, dan peneliti. Meski demikian, pengguna tetap perlu memperhatikan keterbatasan sumber daya dan skalabilitasnya [98].

9. Confusion matrix

Confusion Matrix adalah metode untuk mengukur kinerja model klasifikasi dalam machine learning dengan menganalisis hasil prediksi dan hasil aktual. *Confusion Matrix* dari empat kategori, yaitu *True Positive (TP), True Negative (TN), False Positive (FP),* dan *False Negative (FN)* yang ditampilkan dalam bentuk tabel 3 pada dibawah ini.

Tabel 3. *Confusion Matrix*

Aktual	Prediksi	
	Positif	Negatif
Positif	TP	FP
Negatif	FN	TN

Keterangan:

- *True Negative* (TN) hasil prediksi negatif dan data aktualnya negatif.
- *True Positive* (TP) hasil prediksi positif dan data aktualnya positif.
- *False Negative* (*FN*) hasil prediksi negatif dan data aktualnya positif
- *False Positive* (FP) hasil prediksi positif dan data aktualnya negatif

a. Accuracy

Accuracy merupakan cara untuk mengukur sejauh mana data diklasifikasikan sesuai seperti seharusnya. *Accuracy* dapat diukur dengan persamaan dibawah ini.

$$Accuracy = \frac{TPtotal}{TP + TN + FP + FN} \quad (5)$$

b. Precision

Precision merujuk pada seberapa baik model membuat prediksi yang benar untuk kelas positif dari total prediksi positif yang dilakukan. *Precision* dapat diukur dengan persamaan dibawah ini.

$$Precision = \frac{TP}{TP + FP} \quad (6)$$

c. Recall

Recall digunakan untuk menggambarkan seberapa baik suatu model dapat mengidentifikasi kelas positif dengan benar. *Recall* dapat diukur dengan persamaan dibawah ini.

$$Recall = \frac{TP}{TP + FN} \quad (7)$$

d. F1-Score

F1-Score digunakan untuk menangani potensi ketidakseimbangan kelas dalam dataset, memastikan model tidak bias terhadap kelas mayoritas (video asli). Kedua metrik ini melengkapi *confusion matrix* dengan memberikan perspektif yang lebih luas tentang performa model, sesuai dengan kebutuhan untuk mengevaluasi keandalan prediksi dalam berbagai skenario. *F1-Score* diukur berdasarkan rata-rata dari precision dan recall. *F1-Score* dapat diukur dengan persamaan dibawah ini.

$$F1\ Score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (8)$$

BAB III
METODE PENELITIAN

A. Sub Waktu Penelitian

Penelitian ini dilaksanakan selama lima bulan, mulai dari bulan Maret hingga Juli tahun 2025. Pada periode tersebut, kegiatan penelitian yang dilakukan adalah pengumpulan data dalam bentuk pengambilan (*retrieval)* memanfaatkan dataset yang sudah tersedia secara publik dan terverifikasi dan data *crawling*. Selanjutnya, proses penelitian mulai dari tahap studi literatur, pengumpulan dataset video (*FaceForensics++*), *preprocessing* data (ekstraksi wajah menggunakan MTCNN), hingga perancangan dan pengujian sistem (menggunakan Streamlit), sehingga model mampu mengidentifikasi video dan memberikan *output* dalam bentuk probabilitas keyakinan ”Asli” atau ”*Deepfake*” dan memberikan tingkat akurasi sebagai hasil pemprosesan.

B. Alat dan Instrumen Penelitian

Untuk tahap yang membutuhkan komputasi intensif, yaitu proses pelatihan dan *fine-tuning* model *EfficientNet-B0*, penelitian memanfaatkan platform *cloud computing* Google Colaboratory (Google Colab) dengan akselerasi *Graphics Processing Unit* (GPU).

Untuk tahap yang membutuhkan komputasi intensif, yaitu proses pelatihan dan *fine-tuning* model *EfficientNet-B0*. Dalam penelitian ini, peneliti akan membandingkan performa model pada dua perangkat dengan spesifikasi berbeda, yaitu perangkat Low-end dan High-end. Penelitian ini, akan memanfaatkan platform *cloud computing* Google Colaboratory (Google Colab) dan instrumen fisik berupa laptop untuk menganalisis performa (efisiensi model). Spesifikasi perangkat yang digunakan untuk pengujian ialah sebagai berikut:

Tabel 4. Spesifikasi Perangkat yang digunakan

Perangkat	Processor	Memory	GPU	Sistem Operasi	Keterangan
Low-End (Asus VivoBook 15)	11th Gen Intel Core i5-1135G7 @ 2.40 GHz – 4.20 GHz.	8.00 GB RAM	Intel Iris Xe Graphics.	64-bit Windows 11.	Perangkat ini diklasifikasikan sebagai low-end karena keterbatasan RAM, yang membatasi kemampuannya untuk pelatihan model secara efisien.
High-End (Google Colab)	CPU server-grade (Intel Xeon atau AMD EPYC)	12-13 GB RAM (gratis). 25-27 GB RAM (berbayar).	NVIDIA Tesla T4 (gratis). NVIDIA A100/V100 (berbayar).	Linux	Perangkat ini diklasifikasikan sebagai high-end karena dukungan GPU dedicated dan kapasitas memori yang memadai untuk pelatihan model *Deep learning* secara efisien.

Perbandingan ini bertujuan untuk memverifikasi bahwa *EfficientNet-B0* dapat diimplementasikan secara efisien pada perangkat low-end tanpa kehilangan akurasi signifikan dibandingkan dengan perangkat high-end berbasis *cloud*. Hasilnya akan mendukung klaim bahwa model ini cocok untuk aplikasi deteksi *deepfake* di berbagai platform, termasuk perangkat dengan sumber daya terbatas.

C. Jenis Penelitian

Jenis penelitian yang digunakan dalam Tugas Akhir ini adalah Penelitian Kuantitatif dengan pendekatan Penelitian Terapan (*Applied Research*). Penelitian kuantitatif, digunakan karena penelitian berfokus pada pengumpulan dan analisis data yang bersifat numerik. Seluruh proses, mulai dari pembagian dataset (70% latih, 20% validasi, 10% uji), parameter pelatihan (*batch size, epoch, learning rate*), hingga evaluasi akhir model, diukur dan dianalisis menggunakan metrik statistik yang objektif (Akurasi, Presisi, Recall, dan F1-Score) untuk menguji hipotesis penelitian.

Penelitian Terapan (*Applied Research*) karena tujuan utama penelitian adalah untuk memecahkan masalah praktis dan spesifik di dunia nyata, yaitu mendeteksi video *deepfake*. Penelitian ini tidak hanya mengamati fenomena, tetapi secara aktif merancang, membangun, dan mengimplementasikan sebuah sistem baru (menggunakan *preprocessing* MTCNN, pelatihan *EfficientNet-B0*, dan aplikasi Streamlit) serta menguji efektivitas dari solusi yang diusulkan.

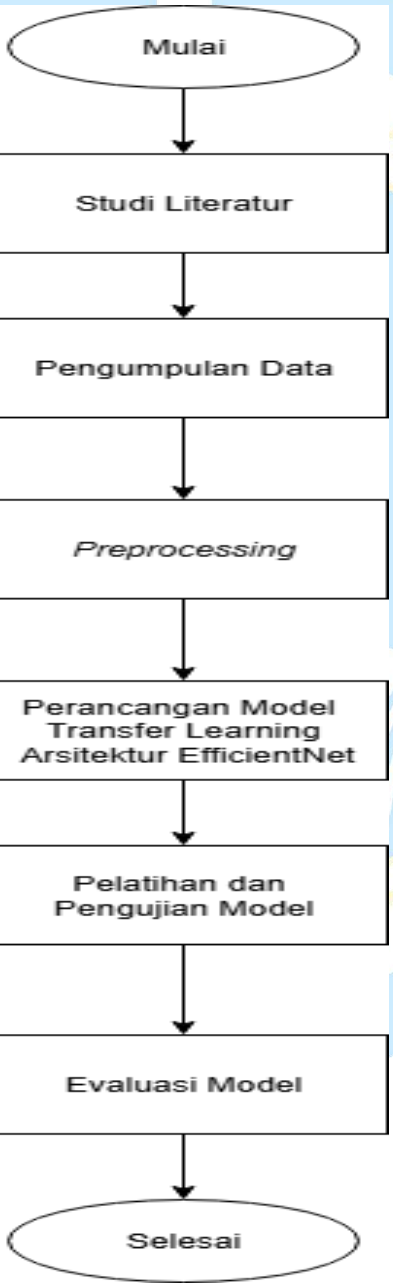
D. Studi Pustaka

Studi pustaka dilakukan untuk memperoleh informasi berupa landasan teoritis terkait penelitian terdahulu dari para ahli maupun penulis yang berkaitan dengan deteksi dan klasifikasi video *Deepfake*. Fokus utama studi ini adalah untuk memperoleh informasi mengenai implementasi *Convolutional Neural Network* (CNN) dengan metode *transfer learning*, khususnya menggunakan arsitektur model *EfficientNet-B0* dalam penelitian terkait.

Studi pustaka dilakukan dengan mengumpulkan jurnal penelitian nasional dan jurnal penelitian internasional sebagai sumber data dan referensi. Penelusuran bahan referensi juga difokuskan pada topik penelitian mengenai penerapan berbagai arsitektur CNN dalam upaya mendeteksi dan mengklasifikasi artefak manipulasi wajah digital dalam kasus penelitian ini *deepfake* pada dataset *benchmark* seperti *FaceForensics++*.

E. Prosedur Pelaksanaan Penelitian

Metodologi penelitian digunakan sebagai acuan dan pedoman peneliti dalam melaksanakan penelitian ini agar mendapatkan hasil sesuai dengan tujuan penelitian. Metodologi penelitian mencakup pengumpulan data hingga terbentuknya hasil akhir berupa model terbaik.



Gambar 18. Alur Metode Penelitian

Berikut adalah penjelasan rinci untuk setiap tahapan alur penelitian yang dilakukan:

1. Studi Literatur, tahap ini dimulai dengan melakukan tinjauan pustaka (studi literatur) secara komprehensif terkait deteksi *deepfake*. Kajian difokuskan pada pemahaman arsitektur, efektivitas metode *transfer learning*, dan analisis arsitektur spesifik *EfficientNet-B0*. Tujuan dari tahap ini adalah untuk menyesuaikan penelitian terkini dan mengidentifikasi celah dari penelitian, seperti perlunya augmentasi data yang lebih tangguh seperti kompresi video dan metode agregasi skor untuk penyempurnaan model yang lebih baik.
2. Pengumpulan Data, pada tahap ini, dataset mentah dikumpulkan sebagai bahan utama penelitian. Penelitian ini menggunakan dataset yang tersedia secara publik yang sudah terverifikasi yaitu *FaceForensics++*. Total dataset yang digunakan untuk penelitian ini 300 video terdiri dari data yang sudah dilabel menjadi 150 video "Asli" dan 150 video "*Deepfake*" telah sortir sesuai variabel masing masing. Video-video ini kemudian dibagi secara proporsional menjadi 70% latih, 20% validasi, 10% uji sebelum diproses lebih lanjut.
3. Preprocessing Data, dataset tidak dapat langsung digunakan oleh model. Oleh karena itu, dilakukan tahap *preprocessing*. Pertama, dilakukan ekstraksi *frame* (pengambilan sampel 10 *frame* per video secara merata). Kedua, setiap *frame* dianalisis menggunakan MTCNN untuk mendeteksi wajah, di mana hanya *frame* berisi satu wajah yang akan diproses. Terakhir, wajah tersebut di-crop, diberi padding (20 piksel), dan di-resize menjadi 224x224 piksel untuk menghasilkan dataset gambar wajah yang dapat diterima sebagai input.
4. Perancangan Model, tahap ini adalah perancangan arsitektur dan pelatihan. Model yang diusulkan adalah *EfficientNet-B0* yang memanfaatkan *transfer learning*. Perancangan ini juga mencakup persiapan augmentasi data untuk set latih, termasuk *flip, crop, jitter,* dan kompresi resolusi JPEG secara acak. Strategi pelatihan dirancang dalam dua fase, fase pertama yaitu *feature extraction*, membekukan (*freeze*) jaringan kecuali *head classifier,* dan fase kedua *fine-tuning,* membuka (*Unfreeze)* kemudian melatih seluruh jaringan.
5. Pelatihan dan Pengujian Model, rancangan model (dari tahap 4) dieksekusi pada tahap ini. Proses pelatihan dilakukan menggunakan *optimizer Adam* dan *loss*

function. Fase pertama (*Feature Extraction*) dilatih selama 20 *epoch*, diikuti fase kedua (*Fine-tuning*) selama 30 *epoch*. Setelah pelatihan selesai, proses pengujian (testing) dilakukan dengan menjalankan model terbaik pada data uji untuk menghasilkan *output*.

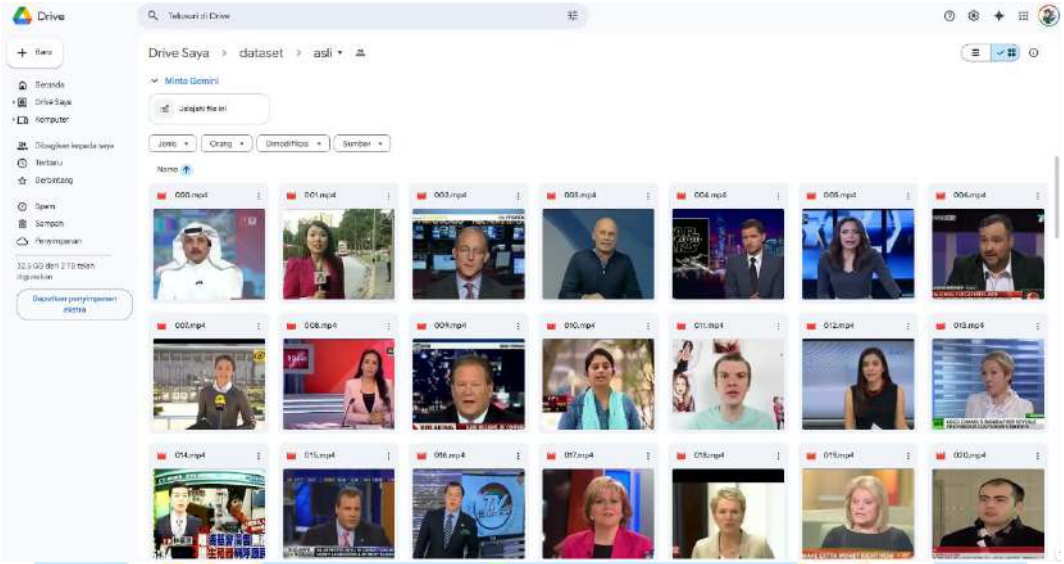
6. Evaluasi, tahap akhir ini adalah menganalisis hasil dari pengujian model. Prediksi dari data uji dibandingkan dengan label data sebenarnya untuk menghasilkan *confusion matrix*. Dari matriks tersebut, dihitunglah metrik evaluasi kuantitatif untuk mengukur kinerja model secara objektif. Selain visualisasi matriks, penelitian ini juga menghitung nilai akurasi, presisi, recall, dan F1-score. Tahap ini bertujuan untuk menentukan model terbaik yang lebih efektif dan efisien dalam mengidentifikasi tingkat akurasi keaslian video, di mana model yang menunjukkan kinerja paling optimal dan memiliki keakuratan terbaik berdasarkan metrik-metrik akan disimpan sebagai model terbaik.

F. Pengumpulan Data

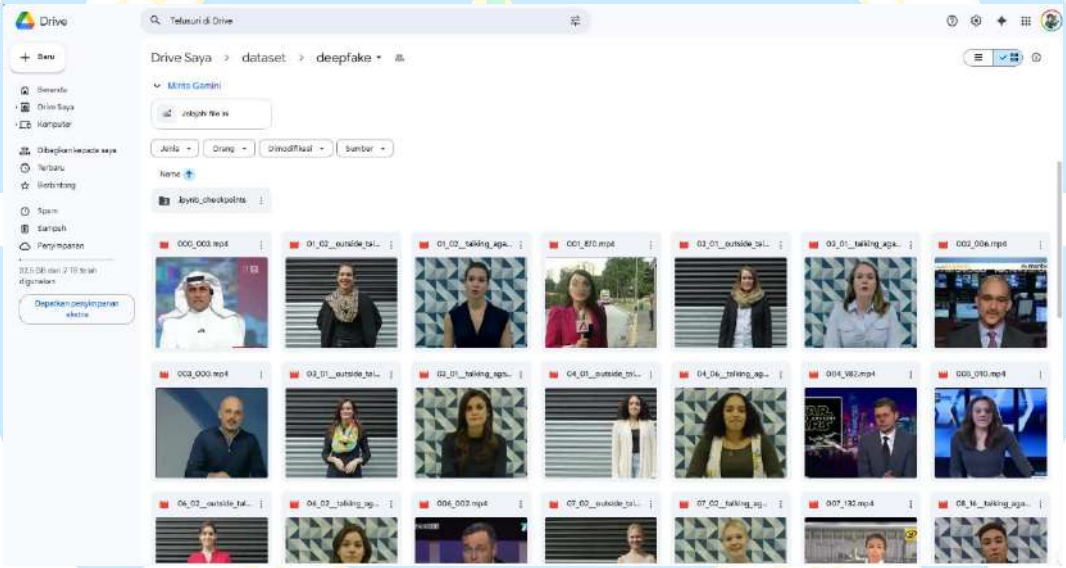
Pengumpulan data dalam penelitian ini dilakukan secara pengambilan (*retrieval)* memanfaatkan dataset yang sudah tersedia secara publik dan terverifikasi dan data *crawling.* Penulis juga melakukan pengajuan, terkait penggunaan data melalui formulir *online Google Form* yang disediakan oleh Peneliti dari *Technical University of Munich* [99]. Dan telah meminta izin penggunaan dataset kepada pihak penyedia melalui formulir *online, FaceForensics, FaceForensics++, and Deepfakes Detection Dataset* untuk mendapatkan data yang diperlukan dan digunakan sebagai pelatihan dan uji coba model yang sudah dibuat. Data yang digunakan berupa video terdiri dari dua kelas, yaitu Asli dan *Deepfake*.

Data yang digunakan berupa kumpulan video yang terdiri dari dua variabel (kategori), yaitu video Asli dan video *Deepfake*. Total dataset yang diproses dalam penelitian ini berjumlah 300 video, yang terdiri dari :

1. 150 video Asli (Real)
2. 150 video *Deepfake* (Fake)



Gambar 19. Dataset Video Asli



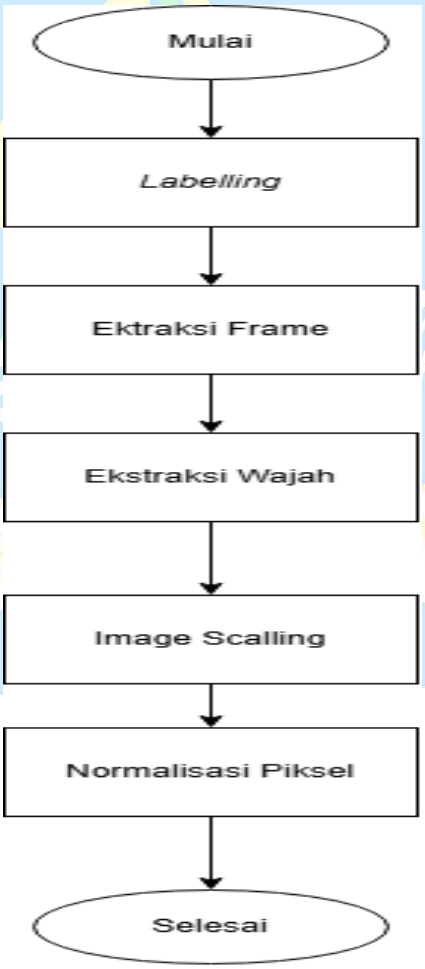
Gambar 20. Dataset Video *Deepfake*

E. Perancangan Sistem

Perancangan sistem dalam penelitian ini menggunakan pendekatan *supervised learning* untuk klasifikasi biner dan dibagi menjadi tiga tahap utama yang saling berhubungan, perancangan *preprocessing* data, perancangan pelatihan model, dan perancangan implementasi model.

1. Preprocessing Data

Pada tahapan pengumpulan data terlebih dahulu akan dilakukan pengolahan sebelum nantinya data digunakan untuk membangun model dengan menggunakan arsitektur *EfficientNet*. Data mentah akan diolah terlebih dahulu tahapaan *preprocessing* merupakan teknik paling awal sebelum proses pelatihan model dilakukan. Pada tahap ini, data mentah berupa video dari dataset *FaceForensics++* diolah agar sesuai dengan format masukan yang dibutuhkan oleh model *EfficientNet-B0*. Tujuan dari tahap *preprocessing* adalah untuk membersihkan, menstandarkan, menyiapkan data sehingga proses pelatihan dapat berlangsung secara lebih efektif dan efisien. Karena data yang telah melalui Pra-pemrosesan data, merupakan data yang sudah melalui beberapa tahap tranformasi. Untuk tahapan *preprocessing data* dapat dilihat pada gambar (21).



Gambar 21. Flowchart Preprocessing Data

a. Labelling,

Proses pelabelan (*labelling*) dilakukan untuk memberikan identitas biner yang jelas pada tiap data agar model dapat belajar membedakan kedua kelas, pada setiap data video sesuai dengan kelasnya, yaitu Asli (0) dan *deepfake* (1). Dalam dataset *FaceForensics++*, setiap video telah memiliki label bawaan yang menunjukkan kelasnya masing masing. Dalam penelitian ini, pelabelan diimplementasikan secara otomatis di lingkungan *Google Colab* dan *Google Drive*. Untuk menunjang keakuratan klasifikasi penentuan label dilakukan berdasarkan ekstraksi dan struktur penamaan file :

1. Label dengan biner 0 untuk video Asli.
2. Label dengan biner 1 untuk video *Deepfake*.

Sejalan dengan tahapan pelabelan, dilakukan segmentasi data (*data splitting)* pengelompokan dalam folder masing-masing ("asli" dan "deepfake") pembagian dataset dilakukan pada video (sebelum ekstraksi *frame*) untuk memastikan tidak ada kebocoran data (*data leakage*), untuk membagi dataset ke dalam tiga himpunan bagian dengan rasio 70% data latih, 20% data validasi, dan 10% data uji. Pembagian data dilakukan secara konsisten dan objektif sesuai dengan parameter yang telah ditentukan. Hasil dari proses ini adalah kumpulan data terstruktur yang telah diberi label, siap digunakan dalam proses pelatihan model klasifikasi menggunakan metode *EfficientNet*. Proses ini memastikan bahwa data yang digunakan memiliki kualitas dan relevansi yang sesuai dengan tujuan penelitian. Pembagian dataset dapat dilihat pada tabel 5 berikut :

Tabel 5. Dataset yang akan digunakan:

Kelas	Label	Jumlah Keseluruhan Video	Jumlah Keseluruhan *Frame*	Data Training	Data Validasi	Data Testing
Asli	0	150	1.500	1.050	300	150
Deepfake	1	150	1.500	1.050	300	150

b. Ekstraksi *Frame*

Karena model *EfficientNet-B0* dirancang hanya untuk menerima input berupa gambar statis, maka setiap video pada dataset harus diubah menjadi beberapa *frame* individual. Untuk mengubah input yang awalnya video dengan format (.mp4) menjadi gambar wajah dengan format (.jpeg) setiap video dalam dataset kemudian diekstraksi menjadi *frame* gambar menggunakan *library* OpenCV. Proses ekstraksi ini tidak bergantung pada fps (*frame per second*) atau interval waktu (detik). Sebaliknya, untuk efisiensi, sistem mengambil sampel sebanyak 10 *frame* untuk mendapatkan representasi wajah dari awal hingga akhir durasi video. Sebagai contoh, video berdurasi 50 detik dapat menghasilkan lima *frame* dengan interval waktu sekitar 10 detik. Proses ini bertujuan untuk, mengurangi beban komputasi, meningkatkan efisiensi penyimpanan, dan mengambil representasi visual yang cukup untuk mewakili isi video secara keseluruhan. *Frame* yang dihasilkan kemudian disimpan sebagai gambar (.jpg atau .png) dan menjadi input utama bagi proses deteksi wajah pada tahap selanjutnya, memungkinkan model untuk menganalisis konten visual dalam video secara berurutan.

c. Ekstrasi wajah

Tahap selanjutnya adalah melakukan deteksi dan isolasi area wajah pada setiap *frame*. Tahap ini berfokus untuk mendeteksi manipulasi pada fitur wajah seperti dahi, pipi, hidung, dan dagu bagian-bagian yang secara struktural stabil dan tidak mudah berubah karena ekspresi atau gerakan. Proses ini memastikan bahwa input model hanya berisi informasi relevan (area wajah), sehingga model dapat lebih fokus pada pola tekstur dan ketidakkonsistenan yang muncul akibat manipulasi *deepfake*. Dengan demikian, hasil deteksi menjadi lebih akurat dan *noise* dari bagian lain (misalnya latar belakang atau pergerakan tubuh) dapat dikurangi secara signifikan dan efisien.

Deteksi wajah dilakukan menggunakan MTCNN *(Multi-task Cascaded Convolutional Networks)* karena *library* ini memiliki tingkat akurasi tinggi dan efisiensi komputasi yang baik. MTCNN menghasilkan koordinat posisi wajah dalam bentuk *bounding box*, yang kemudian digunakan untuk memotong area wajah dari *frame* asli. Pada tahap ini, diterapkan filter penting yaitu hanya *frame* yang terdeteksi setidaknya satu wajah akan disimpan dan *frame* tanpa wajah atau berisi terlalu banyak wajah akan dilewati. Wajah yang terindikasi valid kemudian di-*crop* dengan *padding* 20 piksel dan di-*resize* ke resolusi 224x224 piksel agar sesuai dengan ukuran input yang dapat diterima model *EfficientNet-B0*.

Gambar 22. Ektraksi *Frame*

d. Resize ke 224x224,

Setelah wajah berhasil diekstraksi, citra hasil potongan tersebut diubah ukurannya menjadi 224×224 piksel. Ukuran ini merupakan ukuran input standar untuk arsitektur *EfficientNet-B0* yang telah dilatih sebelumnya pada dataset *ImageNet*. Proses *resize* dilakukan dengan metode interpolasi (*bilinear* atau *bicubic interpolation*) untuk menjaga kualitas visual dan proporsi wajah. Penyeragaman ukuran input ini bertujuan untuk,menjamin kompatibilitas dengan bobot (*weights*) *pre-trained* model, mengurangi kompleksitas komputasi, dan menstabilkan performa model saat inferensi.

e. Normalisasi Piksel.

Tahap akhir *preprocessing* adalah normalisasi nilai piksel adalah proses mengubah nilai piksel RGB (yang awalnya berkisar 0-255) ke rentang standar, biasanya [0, 1] atau [-1, 1], untuk mempermudah pelatihan model. Dalam kasus *EfficientNet-B0*, normalisasi ke [0, 1] umum digunakan. Setiap nilai piksel RGB (0–255) dinormalisasi ke rentang [0, 1], kemudian disesuaikan dengan *Mean* dan *standar deviation* dataset ImageNet,

Mean = [0.485, 0.456, 0.406]

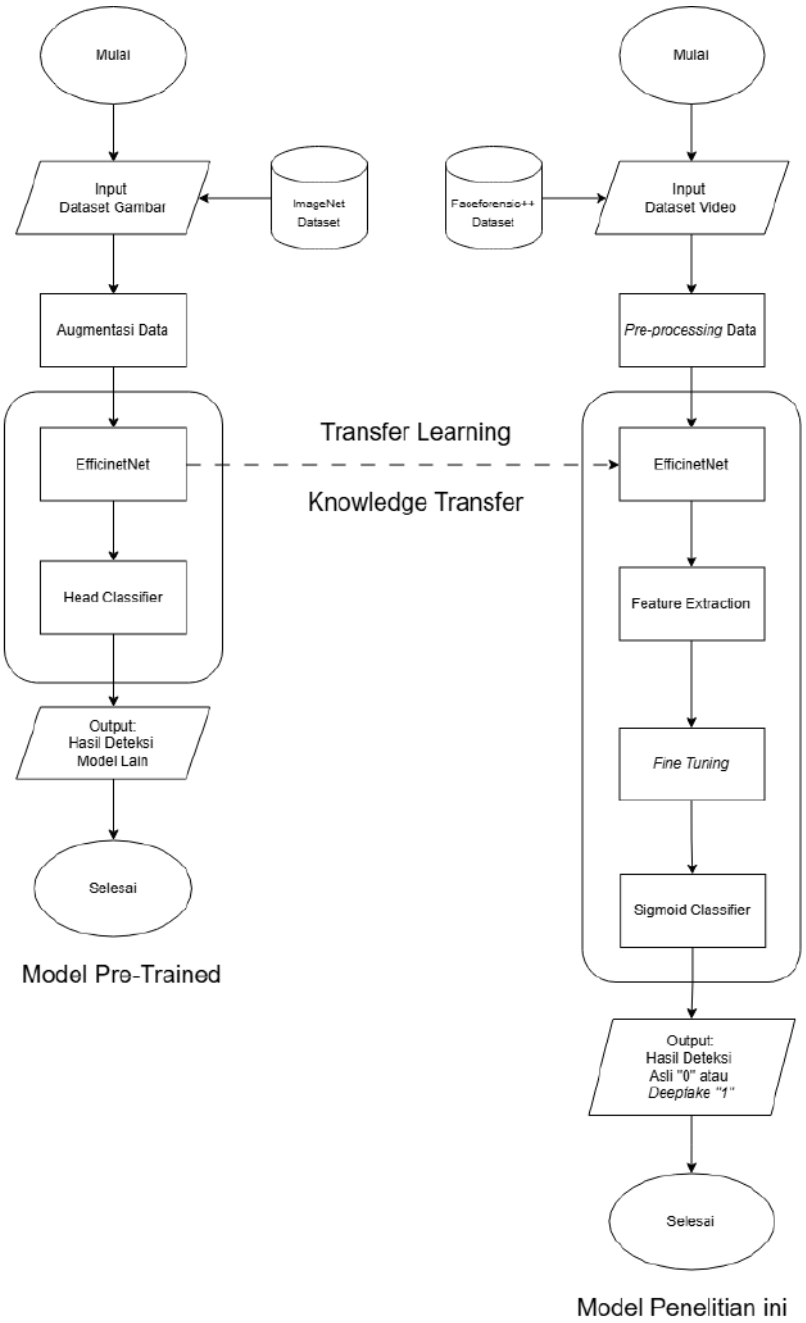
Standar Deviasi = [0.229, 0.224, 0.225] (7)

$$Normalisasi = \frac{Pixel\ Value}{255}$$

Normalisasi memastikan bahwa semua piksel memiliki skala yang konsisten dan seragam, mengurangi pengaruh pencahayaan atau kontras, serta mempercepat konvergensi model selama proses pelatihan dan inferensi. Dalam beberapa kasus, normalisasi tambahan dan pembagian standar deviasi dapat diterapkan untuk menyesuaikan dengan bobot *pre-trained.*

2. Perancangan Arsitektur *EfficientNet*

Dalam tahapan perancangan, digunakan varian *EfficientNet-B0*, yaitu versi dasar dari model *EfficientNet* yang lain, paling ringan namun tetap memiliki kinerja yang kompetitif dan efektif. Setelah praproses, maka dilanjutkan dengan pengembangan algoritma mengikuti pemodelan *EfficientNet* untuk menyesuaikan kebutuhan penelitian. Berikut ini adalah perancangan alur proses pada model *EfficientNet* yang disajikan dalam bentuk flowchart yang mempresentasikan bagaimana metode ini dirancang pada gambar ( ) berikut:



Gambar 23. Konsep Perancangan Model

Perancangan sistem dalam penelitian ini menggunakan metode *transfer learning*. Pendekatan ini, seperti yang diilustrasikan pada diagram alur ( ), dibagi menjadi dua alur utama:

a. Model *Pre-Trained*, yang menjadi dasar transfer pengetahuan,
b. Model *Transfer Learning EfficientNet* (Penelitian ini), yang mengadaptasi pengetahuan tersebut untuk tugas spesifik seperti deteksi *deepfake*.

Model yang digunakan sebagai basis adalah *EfficientNet*, yang dipilih karena efisiensinya dalam menyeimbangkan akurasi dan kompleksitas komputasi. Adapun langkah-langkah perhitungan manual pada metode *transfer learning* dengan *EfficientNet* sebagai arsitektur dasarnya, yang dapat dijelaskan sebagai berikut :

1. Model Pre-Trained,

   Alur ini digunakan untuk menjelaskan bagaimana model basis *(EfficientNet)* memanfaatkan "pengetahuan" yang telah dipelajari dari dataset raksasa (seperti ImageNet) dari model lain dan menerapkannya untuk menyelesaikan masalah yang lebih ”spesifik” pada penelitian lain .

   a. Input Dataset Mentah : Model *EfficientNet* dilatih pada dataset skala besar, yaitu ImageNet. Salah satu sumber daya terbesar yang tersedia untuk melatih model pembelajaran mendalam dalam tugas pengenalan gambar.
   b. Augmentasi Data : Dataset ini berisi jutaan gambar yang telah melalui proses augmentasi data untuk meningkatkan variasi dan generalisasi.
   c. Arsitektur *EfficientNet* (*Compound Scaling*) : *EfficientNet* memperkenalkan metode penskalaan model yang baru yang disebut *Compound Scaling*. Ketimbang menskala satu dimensi model secara *Arbiterase* (seperti depth, width, atau resolusi), *EfficientNet* secara seragam menskalakan ketiga dimensi ini menggunakan satu koefisien gabungan, φ . Metode ini memastikan bahwa sumber daya komputasi (FLOPS) dialokasikan secara seimbang. Penskalaan untuk setiap dimensi diatur oleh persamaan berikut:

$$\text{Depth,} \quad d = \alpha^{\varphi}$$
$$\text{Width,} \quad w = \beta^{\varphi}$$
$$\text{Resolution,} \quad r = \gamma^{\varphi}$$

   Dimana, α, β, γ adalah konstanta yang ditentukan melalui pencarian grid dan φ adalah koefisien yang ditentukan. *EfficientNet-B0* (φ = 0) adalah model dasarnya, sementara *EfficientNet*-B1 hingga B7 adalah model yang diskalakan ke atas menggunakan prinsip ini. Pengetahuan dari model inilah yang ditransfer ke model penelitian.

d. Hasil dari tahap ini adalah model *EfficientNet* dengan *Head Classifier* yang telah terlatih untuk klasifikasi ImageNet. Dalam *transfer learning*, *head* dari pelatihan sebelumnya akan dibuang dan diganti dengan *New Head Classifier* yang sesuai dengan penelitian dengan hasil yang lebih spesifik.

2. Model *Transfer learning EfficientNet,*

Alur ini adalah inti dari penelitian, yang mengadaptasi model *pre-trained* untuk tugas spesifik yaitu deteksi *deepfake*,

a. Input Dataset Video : Proses dimulai dengan dataset target, yaitu *Faceforensics++*. Dataset ini berisi video asli dan video yang dimanipulasi (*deepfake*), yang relevan dengan tujuan penelitian.

b. *Pre-proccessing* data : Input video tidak dapat langsung diproses oleh model. Tahap ini krusial dan mencakup langkah-langkah seperti ekstraksi *frame* (bingkai) dari video, deteksi wajah (face detection) pada setiap *frame*, dan cropping serta resize *frame* wajah agar sesuai dengan ukuran input yang diharapkan oleh *EfficientNet*, untuk varian *EfficientNet-B0* inputnya adalah 224x224 piksel.

c. Arsitektur *EfficientNet* (*Compound Scaling*) : Ketimbang menskala satu dimensi model secara *Arbiterase* (seperti depth, width, atau resolusi), *EfficientNet* secara seragam menskalakan ketiga dimensi ini menggunakan satu koefisien gabungan, φ . Metode ini memastikan bahwa sumber daya komputasi (FLOPS) dialokasikan secara seimbang. Penskalaan untuk setiap dimensi diatur oleh persamaan berikut:

$$\textit{Depth,} \quad d = \alpha^{\varphi}$$

$$\textit{Width,} \quad w = \beta^{\varphi}$$

$$\textit{Resolution,} \quad r = \gamma^{\varphi}$$

Dimana, α, β, γ adalah konstanta yang ditentukan melalui pencarian grid dan φ adalah koefisien yang ditentukan oleh pengguna. *EfficientNet-B0* (φ = 0) adalah model dasarnya, sementara *EfficientNet*-B1 hingga B7 adalah model yang diskalakan ke atas menggunakan prinsip ini. Pengetahuan dari model inilah yang ditransfer ke model penelitian.

d. *Transfer learning* : disinilah "Knowledge Transfer" terjadi. Basis model dari *EfficientNet* yang telah dilatih pada ImageNet pada model sebelumnya dimuat. *Head classifier* aslinya dibuang. Model ini kemudian bertindak sebagai *Feature Extractor* (pengekstrak fitur). Bobot (*weight),* dari lapisan-lapisan *pre-trained* ini "dibekukan" (*freeze*) pada awalnya, yang berarti bobot (w=???) tersebut tidak dipakai selama pelatihan di proses pelatihan awal.

e. Tahap Pelatihan Awal (Adaptasi) : sebuah head classifier baru yang sesuai dengan tugas penelitian ditambahkan di atas basis model yang dibekukan. Berdasarkan diagram, head ini adalah Sigmoid Classifier (dijelaskan di poin f). Model kemudian dilatih untuk 20 *epoch* dengan *learning rate* (LR 1e-5).

f. Setelah pelatihan awal, dilakukan evaluasi : "Apakah Model sudah bisa beradaptasi?". Ini adalah checkpoint untuk menilai apakah model dengan basis yang dibekukan dan head baru sudah mencapai performa yang memuaskan (misalnya, akurasi atau loss pada data validasi).

g. Tahap *Fine-tuning* : *Fine-tuning* adalah proses melatih ulang sebagian dari basis model *pre-trained*. Beberapa lapisan teratas dari *EfficientNet* "dicairkan" (*unfreeze*), yang berarti bobotnya sekarang diizinkan untuk diperbarui. Model kemudian dilatih kembali (seperti dalam diagram, 30 *epoch* lagi dengan LR 1e-5 yang rendah). *Learning rate* yang rendah sangat penting dalam *fine-tuning* untuk mencegah "rusaknya" bobot *pre-trained* yang berharga.

h. *Sigmoid Classifier : Head classifier* akhir yang digunakan adalah lapisan Sigmoid. Fungsi aktivasi sigmoid digunakan karena penelitian ini merupakan masalah klasifikasi biner (dua kelas: Asli atau *Deepfake*). Sigmoid akan menghasilkan probabilitas antara 0.0 dan 1.0.

i. Hasil akhir adalah hasil deteksi, di mana nilai probabilitas ini dipetakan ke label, asli "0" atau *deepfake* "1" proses selesai.

3. Pelatihan Sistem

Setelah arsitektur *transfer learning EfficientNet*, proses pelatihan ini bertujuan untuk melatih model adaptif agar mampu mengenali pola-pola yang membedakan antara video Asli dan *Deepfake*, dan membangun model berdasarkan hasil pelatihan tersebut. Karena model bersifat probabilistik, hasil deteksi ini merupakan indikasi dan bukan keputusan absolut.

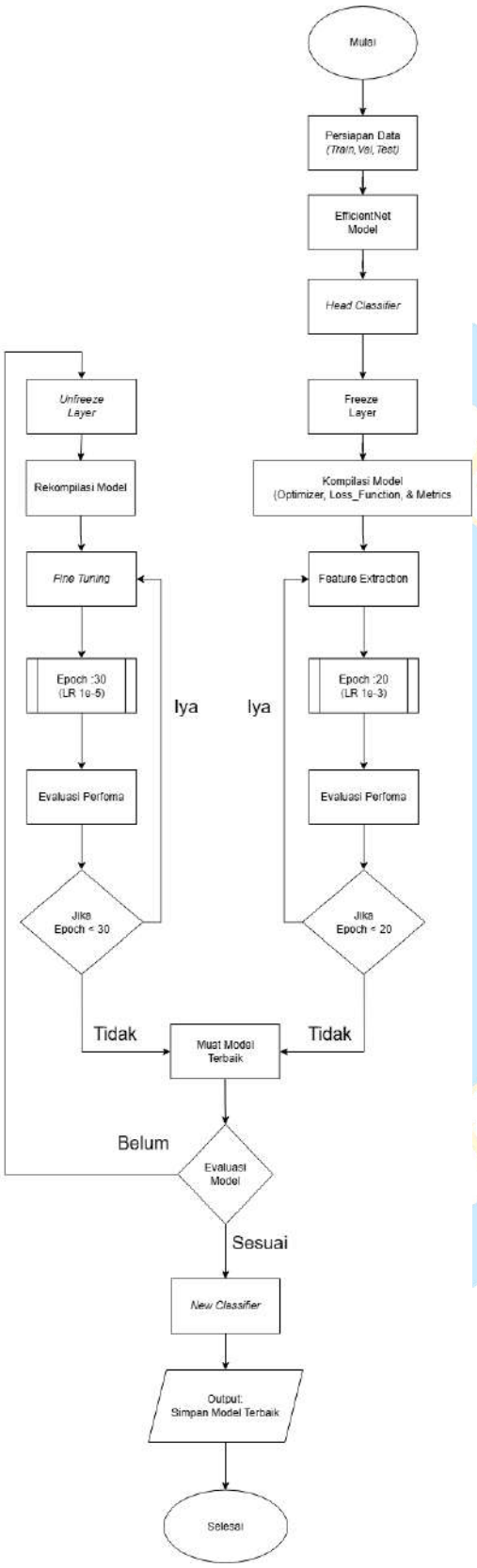
Pada tahap awal, data dari dataset *FaceForensics++* melalui proses persiapan terlebih dahulu. Setiap video diekstraksi menjadi *frame*, kemudian wajah terdeteksi, di-crop, dan di-resize agar memiliki ukuran yang seragam. Setelah proses pra-pengolahan selesai, dataset dibagi menjadi tiga bagian: data latih, data validasi, dan data uji. Data latih digunakan dalam proses pembelajaran model, data validasi digunakan untuk evaluasi performa pada setiap *epoch*, sedangkan data uji disiapkan untuk pengujian akhir. Setelah dataset siap, arsitektur *EfficientNet* dimuat menggunakan bobot *pre-train*ed ImageNet. Pada tahap ini, seluruh lapisan dasar *EfficientNet* dibekukan (*trainable = False*) agar bobot awalnya tidak berubah selama pelatihan tahap pertama. Karena tugas yang dikerjakan merupakan klasifikasi biner (asli atau *deepfake*), sebuah head classifier baru ditambahkan di atas basis *EfficientNet*. Lapisan yang ditambahkan terdiri dari *GlobalAveragePooling2D, Dense, Dropout,* dan *output Dense(1, activation='sigmoid')*, yang sesuai untuk klasifikasi biner. Model kemudian dikompilasi menggunakan optimizer *Adam, loss function binary_crossentropy,* dan *metric accuracy*. Tahap ini bertujuan menyiapkan model sebelum memulai proses pelatihan.

Pada fase ini, pelatihan hanya dilakukan pada lapisan head classifier. Jumlah total *epoch* dan *learning rate* ditentukan sesuai kebutuhan pelatihan. Proses pelatihan dijalankan per-*epoch*, di mana model belajar menggunakan data latih, kemudian dievaluasi menggunakan data validasi. Setiap *epoch* menghasilkan nilai loss dan akurasi pada data latih maupun validasi. Untuk memastikan model hanya menyimpan bobot terbaik, digunakan callback seperti ModelCheckpoint, sehingga bobot hanya disimpan ketika val_loss menunjukkan perbaikan. Setelah evaluasi selesai, sistem memeriksa apakah *epoch* yang berjalan masih kurang dari 20. Jika

iya, proses kembali mengulang ke *epoch* berikutnya. Jika sudah mencapai 20 *epoch*, maka pelatihan Fase 1 dinyatakan selesai. Pelatihan fase 1 *(Feature Extraction → Epoch 20* (LR 1e-3).

Setelah pelatihan head selesai, proses dilanjutkan ke tahap *fine-tuning* untuk meningkatkan performa model. Pada fase ini, beberapa lapisan teratas *EfficientNet* di-unfreeze (*trainable = True*), sehingga lapisan dasar dapat menyesuaikan bobotnya terhadap karakteristik data *deepfake*. Karena ada perubahan pada lapisan yang dilatih, model perlu dikompilasi ulang. *Learning rate* yang digunakan pada tahap ini dibuat sangat rendah, misalnya *learning rate* 1 x 10^{-5} (1e-5), agar *fine-tuning* berjalan lebih stabil dan tidak mengganggu bobot yang sudah terbentuk pada fase sebelumnya. Pelatihan kemudian dilanjutkan kembali selama 30 *epoch*. Sama seperti pada fase sebelumnya, setiap *epoch* dilakukan proses pelatihan dan validasi, dan model terbaik disimpan berdasarkan nilai val_loss yang membaik. Loop pelatihan berlangsung hingga mencapai 30 *epoch*. Setelah selesai, bobot terbaik dari fase *fine-tuning* dimuat sebagai model final. Pelatihan fase 2 *(Fine-tuning → Epoch 30* (LR 1e-5).

Pada tahap evaluasi, performa model dianalisis berdasarkan metrik seperti *accuracy, loss, precision, dan recall*. Jika performanya baik, model dianggap telah beradaptasi. Apabila performa masih belum optimal, proses dapat dilanjutkan kembali melalui *fine-tuning* atau penyesuaian parameter lainnya. Model terbaik yang tersimpan selama fase *fine-tuning* kemudian dimuat sebagai model final, yang selanjutnya siap menjalani evaluasi menggunakan data uji, yaitu data yang sama sekali tidak digunakan selama proses pelatihan.

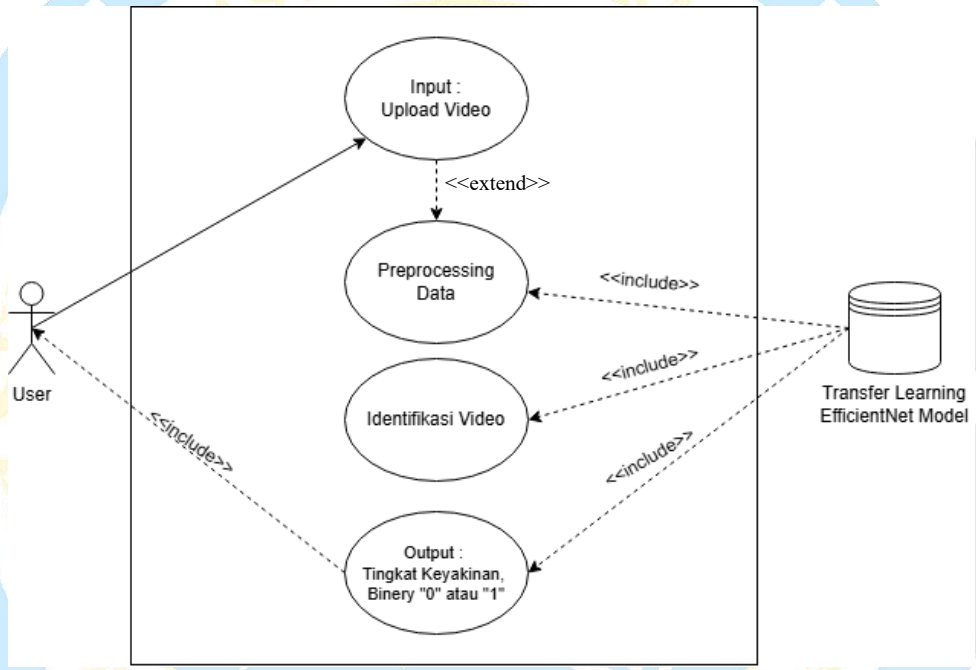


Gambar 24. *Flowchart* Perancangan Sistem

A. Diagram UML

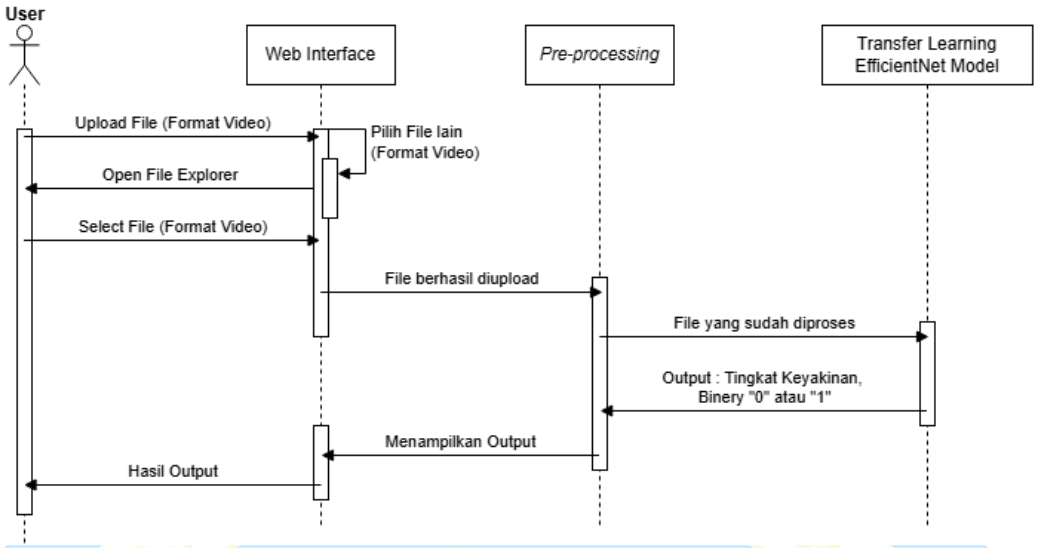
Dalam perancangan sistem, akan digunakan beberapa diagram UML, seperti *Use Case* dan *Sequence*, untuk memperjelas alur kinerja dan interaksi antar komponen dalam sistem.

1. *Use Case* akan menggambarkan berbagai skenario penggunaan sistem oleh pengguna dan aktor lain, serta mendefinisikan tujuan dan fungsi utama dari sistem deteksi sampah. Berikut ini *use case* untuk menjelaskan implementasi sistem dapat dilihat pada gambar (25)



Gambar 25. *Use Case Diagram*

2. *Sequence Diagram* akan menunjukkan urutan interaksi antar objek dalam sistem, termasuk bagaimana data diproses dan diteruskan antar komponen untuk menghasilkan hasil deteksi yang diinginkan. Berikut ini *Sequence* pada sistem deteksi jenis sampah laut dapat dilihat pada gambar (26).

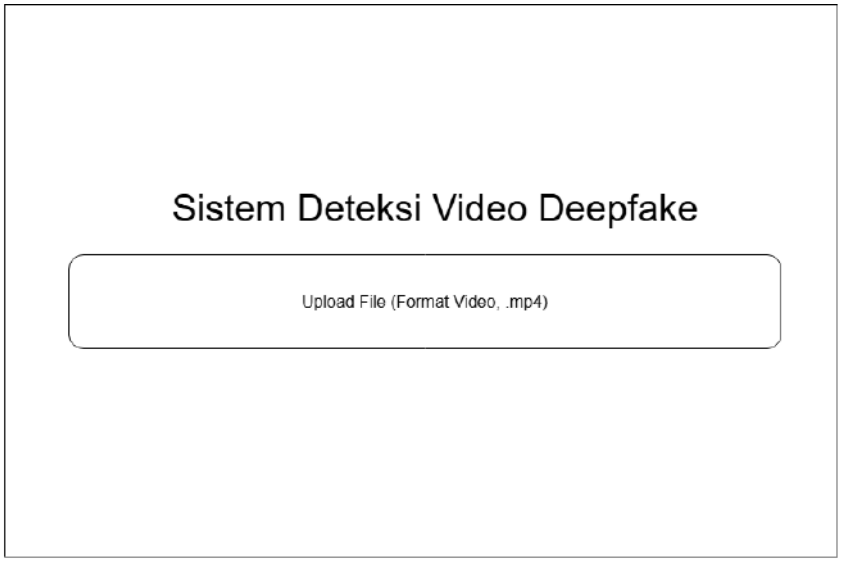


Gambar 26. *Sequence Diagram*

B. Wire*frame*

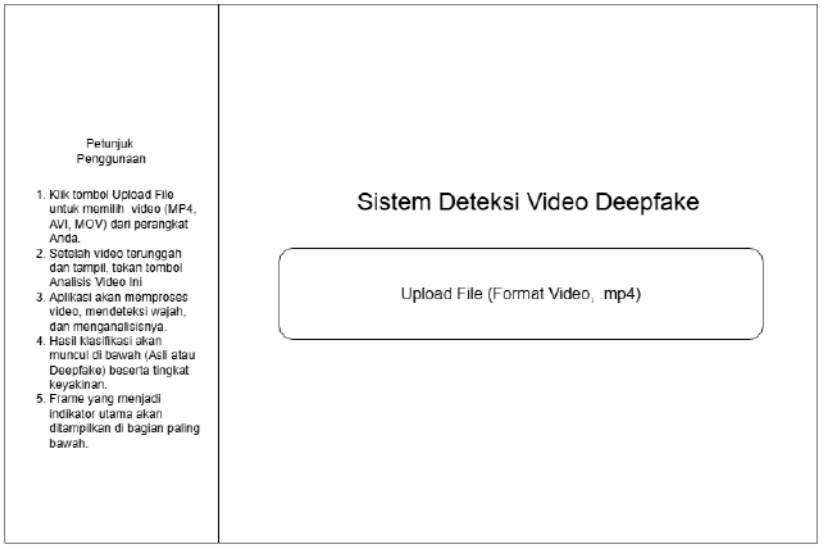
Secara umum, *wireframe* digunakan untuk memvisualisasikan ide awal mengenai tampilan aplikasi tanpa terikat pada aspek estetika seperti warna, ikon, atau desain grafis. Fokus utama dari wire*frame* adalah memberikan gambaran yang jelas mengenai bagaimana pengguna akan berinteraksi dengan sistem, termasuk alur proses dari satu halaman ke halaman lainnya. Dengan demikian, perancangan wire*frame* dapat meminimalisir risiko kesalahan desain pada tahap implementasi dan memungkinkan adanya revisi sejak tahap perancangan.

Halaman utama merupakan tampilan pertama yang muncul ketika aplikasi dijalankan. Pada halaman ini, pengguna disajikan dengan antarmuka yang sederhana dan informatif, sehingga memudahkan navigasi ke fitur utama sistem. Tampilan ini umumnya berisi judul aplikasi, elemen menu, serta ringkasan fungsi yang dapat diakses. Penyusunan komponen pada halaman utama bertujuan memberikan orientasi awal kepada pengguna sehingga proses interaksi dapat dilakukan dengan mudah dan terarah. Struktur tata letak yang digunakan juga mendukung kemudahan penggunaan dengan menempatkan menu dan tombol secara terorganisir.



Gambar 27. *User Interface* Halaman Utama

Sidebar pada halaman utama berfungsi sebagai pusat navigasi bagi pengguna. Pada bagian ini, ditampilkan menu-menu utama seperti halaman beranda, halaman unggah data, halaman hasil, dan menu lainnya yang berkaitan dengan fungsionalitas sistem. Penempatan *sidebar* di sisi kiri antarmuka dibuat agar pengguna dapat mengakses fitur aplikasi secara cepat tanpa harus kembali ke halaman sebelumnya. Desain navigasi ini juga mendukung konsistensi tampilan, sehingga setiap halaman tetap mudah digunakan dan tidak membingungkan pengguna.

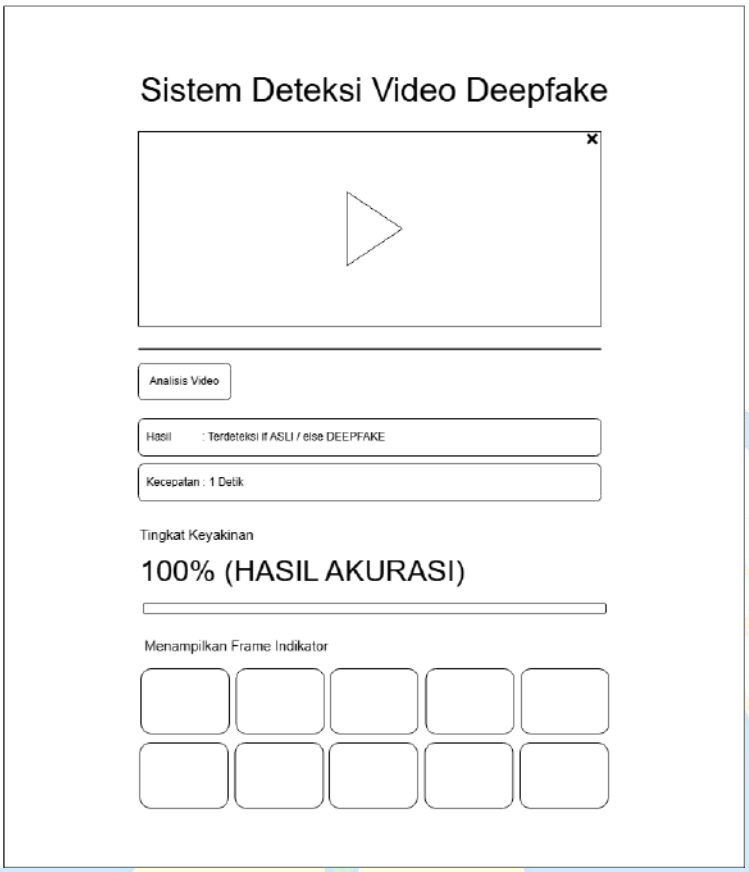


Gambar 28. *User Interface Sidebar* Halaman Utama

Gambar ini menunjukkan tampilan antarmuka setelah pengguna melakukan proses unggah file. Pada tahap ini, sistem menampilkan informasi terkait status unggahan, seperti nama file, ukuran file, atau pratinjau (*preview*) apabila tersedia. Tampilan ini dirancang untuk memastikan pengguna mengetahui bahwa file telah diterima oleh sistem sebelum diproses. Selain itu, halaman ini biasanya menyertakan tombol atau instruksi untuk melanjutkan ke proses pendeteksian atau pengolahan data. Dengan demikian, alur kerja pengguna menjadi lebih terarah dan meminimalkan kemungkinan kesalahan dalam penggunaan sistem.

Gambar 29. *User Interface* setelah Upload File

Pada tampilan ini, sistem menampilkan hasil pemrosesan atau prediksi dari file yang telah diunggah sebelumnya. Informasi yang ditampilkan dapat berupa nilai probabilitas, hasil klasifikasi (misalnya: asli atau *deepfake*), serta visualisasi pendukung lainnya. Tampilan hasil dirancang agar informatif dan mudah dipahami oleh pengguna, sehingga keluaran yang dihasilkan dapat ditinjau secara jelas. Penyajian hasil dalam bentuk yang terstruktur juga mendukung proses evaluasi, baik oleh pengguna umum maupun peneliti yang melakukan pengujian terhadap performa sistem.



Gambar 30. *User Interface* setelah menampilkan Hasil Ouput

F. Analisis Data

Mengingat kompleksitas arsitektur model *EfficientNet-B0* yang memiliki lebih dari 4 juta parameter, perhitungan manual secara keseluruhan untuk seluruh jaringan tidak dimungkinkan untuk disajikan dalam dokumen ini. Oleh karena itu, sub-bab ini menyajikan simulasi perhitungan pada satu sampel *patch* citra (bagian kecil dari gambar) berukuran 3 x 3 piksel.

Tujuan dari simulasi ini adalah untuk memvalidasi logika alur pemrosesan data dalam sistem yang dibangun, mulai dari tahap *pre-processing*, ekstraksi fitur (*Feature Extraction*), hingga klasifikasi pada *head classifier*. Untuk mengimplementasikannya ke dalam model *EfficientNet-B0*, maka perlu melalui tahapan praproses (preprocessing) terlebih dahulu :

1. Tahap Pre-processing Data

Input awal adalah sampel matriks piksel dari saluran warna Merah (*Red Channel*) pada citra wajah. Nilai piksel asli berkisar antara 0 hingga 255.

a. Matriks Input Mentah

Misalkan diambil sampel area 3 x 3 piksel dari citra wajah sebagai berikut:

200	**150**	**100**
180	**160**	**120**
210	**190**	**130**

Gambar 31. Input pada *Channel Red*

b. *Rescaling*

Nilai piksel dibagi dengan 255 untuk mengubah rentang nilai menjadi 0 hingga 1.

$$Rescaling = \frac{Pixel\ Value}{255}$$

Contoh perhitungan, kita ambil matrik baris 1, kolom 1:

$$\frac{200}{255} \approx 0.784$$

Matrik hasil dari *rescaling* untuk *channel red* (dibulatkan ke desimal untuk memberikan kejelasan) adalah,

0.784	**0.588**	**0.392**
0.706	**0.627**	**0.471**
0.824	**0.745**	**0.510**

Gambar 32. Input *Channel Red* setelah di*rescaling*

c. Normalisasi (Standarisasi ImageNet)

Karena model menggunakan bobot *pre-trained* dari ImageNet, data input harus dinormalisasi menggunakan nilai *Mean* dan *Standard Deviation* ImageNet untuk *channel* Merah:

- Mean (μ) = 0.485
- Standar Deviasi (σ) = 0.229

Rumus Normalisasi :

$$Normalization = \frac{Rescaling - \mu}{\sigma}$$

Hasil perhitungan *rescaling* pada piksel pertama (0.784) :

$$\frac{0.784 - 0.485}{0.229} = \frac{0.299}{0.229} \approx 1.305$$

Hasil Normalisasi input matriks :

1.305	**0.450**	**- 0.406**
0.965	**0.620**	**- 0.061**
1.480	**1.135**	**0.109**

Gambar 33. Hasil Normalisasi *Channel Red*

d. Tahap *Feature Extraction*

Pada tahap ini, dilakukan simulasi operasi konvolusi pada *layer* awal (*Stem Conv*) menggunakan *kernel* filter berukuran 3 x 3 dengan *stride* 2.

1. Inisialisasi Kernel Filter (3x3)

Misalkan bobot kernel (filter) yang telah dilatih (*pre-trained*) adalah sebagai berikut:

$$W = \begin{bmatrix} 0.1 & 0 & -0.1 \\ 0.2 & 0 & -0.2 \\ 0.1 & 0 & -0.1 \end{bmatrix}$$

Diketahui Nilai *Bias (b) = 0.05.*

2. Operasi Konvolusi

Operasi konvolusi dilakukan dengan menjumlahkan hasil perkalian *Depth-wise* dan *pointwise* antara matriks input dan Kernel.

$$Output = \sum(p_i \cdot W) + b$$

$$\begin{aligned} Sum = \; & (1.305 \cdot 0.1) + (0.450 \cdot 0) + (-0.406 \cdot -0.1) \\ & + (0.965 \cdot 0.2) + (0.620 \cdot 0) + (-0.0061 \cdot -0.2) \\ & + (1.480 \cdot 0.1) + (1.135 \cdot 0) + (-0.109 \cdot -0.1) \end{aligned}$$

$$\begin{aligned} Sum = \; & (0.1305+0+0.0406) \\ & + (0.1930+0+0.0122) \\ & + (0.1480+0+0.0109) \\ & = 0.5134 \end{aligned}$$

Ditambah dengan Bias :

$$Sum = 0.5134 + 0.05 = 0.5634$$

3. Aktivasi (*Swish)*

$$\text{Swish}(x) = x \cdot \text{Sigmoid}(x) = x \cdot \left(\frac{1}{1+e^{-x}}\right)$$

$$\text{Swish}(0.5634) = 0.5634 \cdot \left(\frac{1}{1+e^{-0.5634}}\right)$$

$$= 0.5634 \cdot 0.6372 \approx 0.359$$

Nilai 0.359 ini adalah hasil perhitungan yang akan diteruskan ke *layer* berikutnya yaitu blok MBConv.

2. Tahap *Head Classifier*

Setelah melalui seluruh lapisan ekstraksi fitur *EfficientNet*, model menghasilkan *Feature Map* yang kemudian diratakan (*flattened*) dan diproses oleh *classifier* baru.

a. *Global Average Pooling* (GAP)

Misalkan Feature Map akhir disederhanakan menjadi matriks 2x2 sebagai berikut :

$$Feature\ Map = \begin{bmatrix} 0.5 & 0.8 \\ 0.2 & 0.5 \end{bmatrix}$$

GAP menghitung rata-rata dari seluruh nilai tersebut:

$$\text{GAP} = \frac{0.5 + 0.8 + 0.2 + 0.5}{4} = \frac{2.0}{4} = 0.5$$

Nilai 0.5 menjadi output neuron (*h)* untuk *layer* selanjutnya.

b. Lapisan Klasifikasi (Sigmoid)

Pada lapisan output (Dense Layer), dilakukan klasifikasi biner untuk menentukan apakah video tersebut Asli (0) atau *Deepfake* (1).

- Input (h) = 0.5
- Bobot (w) = 1.5
- Bias (b) = 0.2

Menghitung nilai Logit (L):

$$L = (h \cdot w) + b$$

$$L = (0.5 \cdot 1.5) + (-0.2) = 0.75 - 0.2 = 0 = 0.55$$

Menghitung Probabilitas Akhir (Fungsi Sigmoid) :

$$P(Deepfake) = \sigma(L) = \frac{1}{1 + e^{-L}}$$

$$P(Deepfake) = \sigma(L) = \frac{1}{1 + e^{-0.55}}$$

$$P(Deepfake) = \frac{1}{1 + 0.5769} = \frac{1}{1.5769} \approx 0.63411$$

c. Kesimpulan Hasil Deteksi

Berdasarkan simulasi perhitungan manual di atas:

- Nilai Probabilitas : 0.6341 (atau 63.41%)
- Hasil: Karena 0.6341 < 1, maka probabilitas mengklasifikasikan input tersebut sebagai Asli (Label 0).

BAB IV
HASIL DAN PEMBAHASAN

Bab ini menjelaskan hasil penelitian yang telah dilakukan, mencakup analisis data, verifikasi logika sistem, serta analisis performa sistem serta pembahasan mengenai kinerja model yang diusulkan. Fokus utama dari penelitian ini adalah pengembangan sistem klasifikasi video *Deepfake* berbasis citra wajah menggunakan arsitektur *EfficientNet-B0* dengan metode *transfer learning*. Dalam konteks forensik digital, validasi terhadap keandalan sistem tidak hanya diukur melalui metrik akhir seperti akurasi, tetapi juga melalui pemahaman terhadap bagaimana model memproses informasi.

A. Lingkungan Uji Coba

Pada penelitian ini, proses pelatihan model dilakukan menggunakan dua jenis perangkat dengan tujuan mengevaluasi efisiensi dan portabilitas, yaitu lingkungan lokal (*Low-End*) dan lingkungan berbasis *cloud* (*High-End*) menggunakan Google Colab menggunakan arsitektur *EfficientNet*, yang memang dirancang untuk efisiensi dalam pemrosesan visual. Perincian spesifikasi lingkungan uji coba disajikan pada Tabel 6 berikut :

Tabel 6. Spesifikasi Lingkungan Uji Coba

Komponen	Perangkat Low-End (Local) Asus Vivobook 15	Perangkat High-End (Cloud) Google Colab
Processor (CPU)	11th Gen Intel Core i5-1135G7 @ 2.40 GHz – 4.20 GHz.	CPU server-grade (Intel Xeon / AMD EPYC)
Memori (RAM)	8.00 GB RAM	12-15 GB RAM (gratis). 25-27 GB RAM (berbayar).

GPU	Intel Iris Xe Graphics (Integrated)	NVIDIA Tesla T4 (gratis). NVIDIA A100/V100 (berbayar).
Komponen	Perangkat Low-End (Local) Asus Vivobook 15	Perangkat High-End (Cloud) Google Colab
Sistem Operasi	Windows 11 64-bit	Linux
Library Utama	PyTorch, OpenCV, MTCNN, Streamlit	PyTorch, OpenCV, MTCNN, Streamlit
Versi Python	Python 3.9	Python 3.10

Perbandingan ini bertujuan untuk:

1. Menguji efisiensi *EfficientNet* secara teori.
2. Mengetahui apakah model tetap dapat berjalan dengan baik pada perangkat dengan sumber daya terbatas.
3. Memberikan gambaran mengenai kelayakan penerapan model di dunia nyata, khususnya deteksi *deepfake* yang dilakukan oleh pengguna umum.
4. Menganalisis waktu pemrosesan per video, termasuk:
   a. loading model,
   b. ekstraksi *frame*,
   c. inferensi,
   d. total end-to-end execution.

Dengan pendekatan ini, penelitian tidak hanya fokus pada akurasi model, tetapi juga menilai nilai praktis *EfficientNet* sebagai model yang efisien dan mudah di-deploy.

B. Persiapan dan Pra-pemrosesan Data

1. Persiapan Data (Data Preparation)

Data yang digunakan dalam penelitian ini mencakup video *asli* (*legitimate/real*) dan video *deepfake*, bersumber dari dataset standar global untuk forensik wajah, yaitu *FaceForensics++*. Dataset ini dipilih karena menyediakan variasi manipulasi yang komprehensif serta menjadi tolak ukur (*benchmark*) untuk kondisi video yang mendekati skenario dunia nyata (*in the wild*). Video yang dikumpulkan disimpan dalam format .mp4.

Namun, data mentah berupa video (*raw*) tidak dapat langsung diproses oleh arsitektur CNN 2D seperti *EfficientNet-B0* yang hanya bisa menerima input statis. Oleh karena itu, tahapan dimulai dengan transformasi data mentah menjadi format lain berupa *frame*. Oleh karena itu, data perlu dipersiapkan terlebih dahulu melalui serangkaian tahapan ekstraksi dan penyaringan untuk mendapatkan ROI (*Region of Interest*), yaitu wajah manusia, dan membuang informasi yang tidak relevan seperti latar belakang. Tahapan persiapan data ini mengikuti alur untuk menjamin integritas data.

2. Penyaringan Data (Data Filtering)

Tantangan utama dalam deteksi *deepfake* berbasis *frame* video adalah memastikan bahwa input yang masuk ke model klasifikasi adalah benar-benar wajah manusia yang valid. Video mentah seringkali mengandung objek lain yang menyerupai wajah atau *frame* kosong yang tidak informatif. Oleh karena itu, diperlukan mekanisme penyaringan data yang ketat.

3. Pembersihan Data (Data Cleaning)

Meskipun MTCNN telah menyaring deteksi berdasarkan tingkat keyakinan (*confidence score*), data wajah yang terdeteksi masih perlu dibersihkan lebih lanjut. Tidak semua wajah yang memiliki *confidence* tinggi layak untuk diproses. Beberapa deteksi mungkin memiliki distorsi geometri yang ekstrem atau terpotong di tepi layar.

Pembersihan data dilakukan dengan menerapkan filter aspek rasio dan validasi batas *frame*. Wajah manusia secara alami memiliki proporsi tertentu. Deteksi yang terlalu gepeng (lebar >>> tinggi) atau terlalu pipih seringkali merupakan kesalahan deteksi pada objek non-wajah. Berikut adalah implementasi fungsi pembersihan data:

```
def is_valid_face(box, frame_width, frame_height):
    x, y, w, h = faces [0] ['box']

 # Menghitung rasio aspek (lebar / tinggi)
        if len(faces) == 1:
        aspect_ratio = w / h

    # Filter 1: Validasi Aspek Rasio
        aspect_ratio = w / h
        if aspect_ratio > 1.0 or aspect_ratio < 0.65:continue

    # Filter 2: Validasi Ukuran Minimum
        if w < 20 or h < 20: continue
        return False
```

Potongan kode di atas memperjelas logika pembersihan data. Sistem secara otomatis membuang data *outlier* yang memiliki rasio aspek di luar rentang 0.65 - 1.0. Logika ini meminimalisir risiko model mempelajari fitur yang salah akibat input yang terdistorsi. Wajah yang lolos dari tahap pembersihan ini kemudian di-*crop* tanpa *padding* tambahan (`FACE_PADDING = 0`) untuk memfokuskan model pada tekstur kulit wajah semata.

4. Pra-proses Data (Data Pre-processing)

Tantangan utama dalam deteksi *deepfake* berbasis *frame* video adalah memastikan bahwa input yang masuk ke model klasifikasi adalah benar-benar wajah manusia yang valid. Video mentah seringkali mengandung objek lain yang menyerupai wajah atau *frame* kosong yang tidak informatif. Oleh karena itu, diperlukan mekanisme penyaringan data yang ketat.

Praproses (*preprocessing*) merupakan tahapan fundamental yang bertujuan mentransformasikan data citra wajah mentah menjadi format numerik (tensor) yang kompatibel dengan arsitektur jaringan saraf tiruan. Mengingat penelitian ini menggunakan model *EfficientNet-B0* dengan bobot *pre-trained* dari ImageNet, data input wajib mengikuti standar statistik distribusi data ImageNet agar proses *transfer learning* berjalan optimal.

Tahapan praproses ini melibatkan penggunaan pustaka torchvision dan sklearn. Proses pra-proses data dilakukan secara berurutan sebagai berikut:

a. Labelling

Sebelum data dikonversi menjadi tensor, pelabelan dilakukan untuk mengorganisasikan data ke dalam kelas-kelas yang diawasi (*supervised*). Penelitian ini merupakan studi kasus klasifikasi, sehingga skema pelabelan diterapkan. Dalam implementasi menggunakan ImageFolder dari PyTorch atau pembagian manual, label ini dikodekan menjadi vektor target.

```
y_label_real = 0
y_label_fake = 1
```

Pelabelan untuk Data Pelatihan:

1. Label 0 (Asli/*Legitimate*): Diberikan untuk *frame* wajah yang diekstraksi dari video asli yang tidak mengalami manipulasi digital. Folder penyimpanan data ini dinamai Asli
2. Label 1 (Manipulasi/*Deepfake*): Diberikan untuk *frame* wajah yang diekstraksi dari video *Deepfake* hasil manipulasi. Folder penyimpanan data ini dinamai *deepfake*.

Pelabelan ini menjadi dasar bagi fungsi kerugian (*loss function*) untuk menghitung tingkat kesalahan prediksi model selama proses pelatihan.

b. Ekstraksi Frame

Sebuah video berdurasi 10 detik dengan *framerate* 30 fps memiliki 300 *frame*. Memproses seluruh *frame* tersebut akan memakan waktu komputasi yang sangat lama dan menghasilkan data yang redundan (berulang) karena perbedaan antar *frame* yang berdekatan sangat minim.

Untuk mengatasi hal ini, penelitian ini menerapkan strategi Sampling Interval. Sistem tidak memproses setiap detik video, melainkan mengambil sampel sebanyak 20 *frames* per video yang tersebar merata dari detik awal hingga akhir, untuk menangkap variasi pose wajah, ekspresi, pencahayaan, serta potensi artefak *deepfake*. Setiap *frame* kemudian dikonversi menjadi gambar berukuran 224×224 piksel sesuai standar input *EfficientNet*.

Tahap ini juga membantu memperbesar jumlah sampel sehingga model dapat belajar lebih representatif.Proses ini diimplementasikan menggunakan fungsi numpy.linspace pada total durasi video. Pendekatan ini memiliki implikasi penting, model tidak hanya belajar dari satu wajah statis, tetapi mendapatkan gambaran wajah dari berbagai sudut pandang dan kondisi pencahayaan yang berubah-ubah sepanjang video. Hal ini diharapkan mampu meningkatkan ketahanan (*robustness*) model terhadap variasi temporal. Berikut implementasi fungsi :

```
#strategi  sampling interval
  FRAMES_PER_VIDEO = 20

# Fungsi linspace untuk interval
  frame_indices = np.linspace(0, total_frames - 1, num_frames_to_extract, dtype=int)

# Ekstraksi Frame
  cap = cv2.VideoCapture(video_path)
  total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

# Mendapatkan 20 indeks frame yang tersebar merata
  num_frames_to_extract = min(FRAMES_PER_VIDEO, total_frames)

  for idx in frame_indices:
        cap.set(cv2.CAP_PROP_POS_FRAMES, idx)
        ret, frame = cap.read()

        if ret:
           frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
           try:
              faces = detector.detect_faces(frame_rgb)
           except Exception: continue
```

Penggunaan `FRAMES_PER_VIDEO = 20`, ini menjadi standar dalam tahap persiapan data, memastikan bahwa setiap video direpresentasikan oleh jumlah yang konsisten, sehingga menyeimbangkan model beban pelatihan.

c. Ekstraksi Wajah

Sistem menggunakan *library* MTCNN (*Multi-task Cascaded Convolutional Networks*) untuk mendeteksi wajah dalam setiap *frame* video. MTCNN dipilih karena arsitekturnya yang bertingkat (*cascaded*) yang mampu mendeteksi titik *landmark* pada wajah, memastikan dagu, telinga, dan garis rambut tidak terpotong, memberikan konteks spasial tambahan bagi model untuk menganalisis batas antara wajah dan latar belakang, yang seringkali menjadi area munculnya artefak manipulasi (efek *blur* atau *ghosting*) pada video *Deepfake*, serta ketahanannya terhadap variasi pose dan pencahayaan dibandingkan metode deteksi wajah konvensional seperti Haar Cascade.

Proses penyaringan dimulai dengan inisialisasi detektor dan penetapan parameter ambang batas (*threshold*) untuk meminimalkan *false positive*. Berikut adalah implementasi kode Python untuk inisialisasi detektor wajah :

```
from mtCNN.mtCNN import MTCNN
import cv2
import numpy as np

# Inisialisasi MTCNN
st.session_state.mtCNN_detector = MTCNN()

# Threshold untuk meminimalisir false positive
CONFIDENCE_THRESHOLD = 0.85

# Deteksi wajah di bawah 20x20piksel diabaikan
MIN_FACE_SIZE = 20

# Batas wajah
FACE_LIMIT = 3
```

Kode menunjukkan penerapan aturan dalam penyaringan data:

1. *Confidence Threshold* (0.85) : Hanya deteksi dengan probabilitas keyakinan di atas 85% yang dianggap sebagai wajah. Hal ini untuk menghindari *false positives*, seperti mendeteksi pola pada dinding atau pohon sebagai wajah, yang dapat merusak proses pelatihan model.
2. *Minimum Face Size* (20px) : Wajah dengan resolusi sangat rendah (batasan < 20 piksel) seringkali kehilangan artefak visual halus yang menjadi ciri khas *deepfake*. Oleh karena itu, jika wajah dianggap terlalu kecil akan dieliminasi dari dataset.
3. *Face Limitation* (3) : Batasan ini berikan untuk meningkatkan efisiensi model dalam melakukan analisis dan meringankan sumber daya komputasi yang digunakan, sekaligus memperkuat logika sistem.

d. Augmentasi Data

Untuk mencegah *overfitting* suatu kondisi di mana model hanya menghafal data latih namun gagal pada data baru teknik augmentasi diterapkan secara agresif pada data latih (train set). Augmentasi yang diterapkan meliputi:

1. Kompresi : Menggunakan apply_jpeg_compression dengan kualitas acak (40-90). sehingga model diharapkan mampu beradaptasi terhadap artefak JPEG.
2. Geometric Jitter : Rotasi, *flipping* horizontal, dan pemotongan acak (*random resized crop*) untuk mensimulasikan variasi posisi kamera.
3. Color Distortion : Perubahan acak pada kecerahan, kontras, dan saturasi untuk mengatasi perbedaan kondisi pencahayaan.

e. Resize (*Rescalling)*

EfficientNet-B0 secara *default* dioptimalkan untuk resolusi input 224 x 224 piksel. Oleh karena itu, seluruh citra wajah diubah ukurannya (*resizing*) ke dimensi tersebut. Selain itu, citra juga dikonversi dari format *integer* (0-255) menjadi format *floating point* tensor (0.0-1.0). Implementasi transformasi menggunakan torchvision.transforms.Compose:

```
from torchvision import datasets, models, transforms

# Definisi Ukuran Input Standar EfficientNet
  IMG_SIZE = (224, 224)

# Pipeline Transformasi Data
    data_transforms = {
       'train': transforms.Compose( [
          transforms.Resize(256),
          transforms.RandomCrop(IMG_SIZE),
          transforms.RandomHorizontalFlip(),

  transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.2),

    transforms.RandomApply( [
             transforms.GaussianBlur(kernel_size=(3, 7))
          ], p=0.3),

# Augmentasi Kompresi
    transforms.RandomApply( [
      transforms.Lambda(apply_jpeg_compression)
    ], p=0.4),

# Konversi ke Tensor
    transforms.ToTensor()
```

Langkah ini memastikan bahwa setiap data yang masuk ke jaringan memiliki struktur dimensi tensor (3, 224, 224), di mana 3 merepresentasikan kanal warna RGB (*Red, Green, Blue*).

f. Normalisasi *(Normalization)*

Langkah terakhir adalah normalisasi nilai piksel. Citra input RGB dengan rentang nilai 0-255 yang telah diubah (*rescaling*) menjadi rentang 0-1, normalisasi adalah langkah yang sering diabaikan namun sangat kritis dalam *transfer learning*. Model *EfficientNet-B0* yang digunakan telah dilatih pada jutaan gambar ImageNet. Bobot-bobot filter konvolusi dalam model tersebut telah beradaptasi dengan distribusi intensitas warna spesifik dari dataset ImageNet.

Agar model dapat mengekstraksi fitur wajah dengan akurat menggunakan bobot *pre-trained* tersebut, distribusi statistik (rata-rata) dari citra input wajah harus diselaraskan dengan distribusi ImageNet. Proses ini dilakukan dengan normalisasi Z-score pada setiap kanal warna RGB:

$$Input_{Normalisasi} = \frac{Input - Mean}{Std}$$

Mean = [0.485, 0.456, 0.406]

Standar Deviasi = [0.229, 0.224, 0.225]

```
# Normalisasi
    transforms.Normalize( [0.485, 0.456, 0.406],  [0.229, 0.224, 0.225])
  ]),
```

Penerapan kode di atas memastikan bahwa data selama pelatihan bergerak lebih stabil dan penyesuaian model tercapai lebih cepat.

g. Pembentukan Tensor *(Batching)*

Setelah data citra individual diproses, data tersebut dikumpulkan ke dalam tumpukan atau *batch*. Pembentukan *batch* tensor 4D dengan dimensi (*Batch_Size, Channel, Height, Width*) memungkinkan GPU untuk memproses banyak gambar secara paralel, yang meningkatkan kecepatan pelatihan secara signifikan. Dalam penelitian ini, ukuran *batch* (*batch size*) ditetapkan sebesar 32, menyesuaikan dengan kapasitas memori VRAM pada GPU NVIDIA Tesla T4 yang tersedia di Google Colab.

h. Segmentasi Data (Data Segmentation)

Untuk memastikan evaluasi model yang objektif dan menghindari *overfitting*, dataset dalam penelitian ini dibagi menggunakan skema *hold-out validation* menjadi tiga bagian terpisah, yaitu : data latih (*training set*), data validasi (*validation set*), dan data uji (*testing set*). Proses segmentasi ini dilakukan dengan hati-hati untuk memastikan tidak ada kebocoran data (*data leakage*), di mana *frame* dari video yang sama tidak boleh tersebar di data latih dan data uji sekaligus. Berikut adalah implementasi kode segmentasi data :

```
from sklearn.model_selection import train_test_split
import os

# Pembagian Data 70:20:10
  TRAIN_RATIO = 0.7
  VALIDATION_RATIO = 0.2
  TEST_RATIO = 0.1

# Memisahkan Data Uji (70%) dari sisa data (30%)
  videos_train, videos_val_test = train_test_split(video_files, test_size=
  (1 - TRAIN_RATIO), random_state=42)

# Memisahkan Data Validasi (20% dari total sisa data)
  videos_val, videos_test = train_test_split(videos_val_test, test_size =
  relative_test_ratio, random_state=42)

  print(f" {category}: {len(videos_train)} train, {len(videos_val)} validation,
  {len(videos_test)} test videos.")

  video_splits = {'train': videos_train, 'validation': videos_val, 'test': videos_test}
```

Segmentasi dilakukan menggunakan *library* sklearn.model_selection dengan proporsi pembagian sebagai berikut :

1. Data Latih (*Training Set*) – 70%: Digunakan secara langsung untuk melatih model. Pada tahap ini, model akan mempejari fitur-fitur wajah dari video *real* dan *deepfake* untuk memperbarui bobot (*weights*) jaringan syaraf tiruan.
2. Data Validasi (*Validation Set*) – 20%: Data ini tidak digunakan untuk melatih model, melainkan untuk mengevaluasi kinerja model pada setiap akhir *epoch* selama proses pelatihan berlangsung. Data validasi berperan sebagai indikator untuk memantau kemampuan penalaran model terhadap data baru yang belum pernah dilihat sebelumnya. Selain itu, performa pada data validasi digunakan sebagai acuan dalam *hyperparameter tuning* dan menentukan model terbaik (*model checkpointing*) yang akan disimpan. Jika *loss* pada data validasi mulai

meningkat sementara *loss* data latih terus menurun, hal ini menjadi indikasi terjadinya *overfitting*.

3. Data Uji (*Testing Set*) – 10%: Digunakan hanya satu kali di akhir penelitian setelah proses pelatihan dan pemilihan model terbaik selesai. Data ini berfungsi untuk mengukur performa akhir model secara tidak bias dalam skenario dunia nyata.

Segmentasi berbasis video (bukan berbasis *frame*) seperti pada kode di atas sangat krusial untuk menjamin validitas pengujian, memastikan model diuji pada subjek/wajah yang benar-benar baru.

C. Implementasi Sistem

1. Implementasi Arsitektur *EfficientNet-B0* dan *Transfer Learning*

Pilihan arsitektur *EfficientNet-B0* sebagai dasar sistem (*backbone*) didasarkan pada prinsip efisiensi parameter. Berbeda dengan model-model pendahulunya yang mengejar akurasi, melalui *deep network* yang masif (seperti ResNet-152) atau pelebaran kanal (seperti WideResNet). Arsistektur EfficientNet menggunakan metode *Compound Scaling* yang menyeimbangkan kedalaman, lebar, dan resolusi secara optimal. Artefak manipulasi wajah sering kali bersifat halus dan terlokalisasi pada fitur-fitur tekstur mikro (seperti inkonsistensi pada batas kulit atau refleksi mata). EfficientNet-B0, dengan jumlah parameter sekitar 5,3 juta, menawarkan kapasitas yang cukup untuk mempelajari representasi fitur tingkat tinggi ini tanpa membebani sumber daya komputasi secara berlebihan, memungkinkan inferensi yang efisien pada perangkat standar.

Implementasi penelitian ini menggunakan pendekatan *Transfer Learning* dari bobot yang telah dilatih sebelumnya (*pre-trained*) pada dataset ImageNet. Strategi ini didasarkan pada hipotesis bahwa fitur-fitur visual dasar yang dipelajari dari jutaan citra objek umum (seperti deteksi tepi, bentuk geometris, dan tekstur dasar) dapat ditransfer dan disesuaikan (*fine-tuned*) untuk tugas deteksi anomali wajah. Hal ini secara signifikan mempercepat konvergensi pelatihan dan mengurangi kebutuhan akan dataset *Deepfake* yang sangat besar, yang sering kali sulit diperoleh dengan kualitas yang konsisten.

Karena tujuan penelitian ini adalah klasifikasi video Asli atau *Deepfake*, modifikasi dilakukan dengan mengganti lapisan klasifikasi akhir (*classifier head*) dengan lapisan baru yang terdiri dari regularisasi *Dropout* dan lapisan *Linear* tunggal. Berikut adalah ringkasan arsitektur modifikasi yang diimplementasikan menggunakan PyTorch:

```
import torch
import torch.nn as nn
from torchvision.models import EfficientNet_B0_Weights

# Memuat Backbone Pre-trained
  weights = EfficientNet_B0_Weights.IMAGENET1K_V1
  model = models.EfficientNet_b0(weights=weights)

# Mendapatkan dimensi dari backbone
  num_ftrs = model.classifier [1].in_features

# Mengganti classifier head
  model.classifier [1] = nn.Sequential(
    nn.Dropout(p=0.5),
    nn.Linear(num_ftrs, 1)
  )

# Memindahkan model ke GPU/CPU
  model = model.to(device)
  return model
```

1. *Backbone EfficientNet-B0* : Bertindak sebagai *Feature Extractor* yang sangat kuat, mengubah citra wajah menjadi representasi fitur vektor berdimensi tinggi (1280 dimensi).
2. *Dropout* (p=0.5) : Lapisan ini secara acak mematikan 50% neuron selama pelatihan. Hal ini mencegah model menjadi terlalu bergantung pada fitur tertentu (*overfitting*) dan memaksanya mempelajari pola manipulasi yang lebih umum.
3. *Linear Layer* (Output 1) : Menghasilkan nilai *logit* tunggal. Nilai ini merepresentasikan skor mentah kecenderungan gambar menjadi *deepfake* sebelum dikonversi menjadi probabilitas.

Untuk memverifikasi bahwa arsitektur model telah terbangun dengan benar dan sesuai dengan metode *transfer learning* yang direncanakan (Fase 1: *Feature Extraction*), dilakukan pengecekan menggunakan *library* torchsummary. Hal ini memberikan gambaran mengenai struktur model dan jumlah parameter pada setiap lapisan. Berikut adalah hasil ringkasan model (*model summary*) yang dihasilkan oleh sistem:

Total params: 4,008,829 Trainable params: 1,281 Non-trainable params: 4,007,548
Input size (MB): 0.57 Forward/backward pass size (MB): 173.65 Params size (MB): 15.29 Estimated Total Size (MB): 189.52

a. Struktur Lapisan (*Layer Hierarchy*):

1. Stem Layer (Conv2d-1): Menerima input citra dan melakukan konvolusi awal dengan 32 filter, menghasilkan *feature map* berukuran 112x112.
2. MBConv & *SqueezeExcitation*: Model ini menggunakan blok bangunan utama *Mobile Inverted Bottleneck Convolution* (MBConv) yang dilengkapi dengan modul *Squeeze-and-Excitation* (SE). Blok ini terlihat pada lapisan MBConv-15, MBConv-30, dst. Modul SE berfungsi untuk meningkatkan representasi fitur dengan memberikan bobot pada *channel* yang penting secara adaptif.
3. *Head Classifier* (Linear-256): Lapisan terakhir adalah Linear-256 dengan *Output Shape* [-1, 1]. Angka -1 merepresentasikan ukuran *batch* yang dinamis, sedangkan 1 adalah output skalar tunggal (*logit*) yang akan dikonversi menjadi probabilitas *deepfake* melalui fungsi sigmoid.

b. Estimasi Sumber Daya:

Estimasi total memori yang dibutuhkan per *pass* adalah sekitar 189.52 MB. Dengan VRAM GPU Google Colab (Tesla T4 ~16GB), ukuran model memungkinkan penggunaan *batch size* yang lebih besar (seperti 32 atau 64) untuk mempercepat pelatihan tanpa risiko *Out of Memory* (OOM).

2. Logika Sistem

Sistem deteksi "DEFEND" mengadopsi mekanisme pengambilan keputusan berdasarkan agregasi 20 *frame* sampel untuk menentukan klasifikasi akhir video. Penerapan ambang batas 0.30 dalam mekanisme pengambilan keputusan ini didukung oleh penelitian Sun, yang menyatakan bahwa artefak manipulasi *deepfake* sering kali bersifat temporal dan tidak muncul di seluruh *frame* video, sehingga diperlukan sensitivitas yang lebih tinggi dalam proses agregasi keputusan [95]. Dalam penelitian ini, diterapkan ambang batas (*threshold*) sebesar 0.30. Artinya, jika minimal 30% dari total *frame* yang diuji terdeteksi sebagai manipulasi, maka sistem akan memberikan vonis *deepfake* pada video tersebut. Pemilihan angka 0.30 didasarkan pada karakteristik data *deepfake* yang sering kali memiliki artefak manipulasi yang tidak konsisten di setiap detiknya (*temporal inconsistency*).

Penggunaan ambang batas bertujuan untuk meminimalisir angka *False Negative* (video palsu yang dianggap asli). Hal ini dikarenakan dalam video *deepfake* yang berkualitas tinggi, manipulasi wajah mungkin hanya terlihat jelas pada sudut pandang tertentu atau saat terjadi pergerakan ekstrem, sehingga tidak semua *frame* akan menunjukkan skor probabilitas tinggi. Logika tersebut diimplementasikan dengan beberapa parameter ambang batas (*threshold*) yang telah diterapkan pada sistem :

1. *Frame* Fake Threshold (0.50) : Untuk menentukan apakah satu wajah dalam satu *frame* terindikasi palsu. Menggunakan nilai standar untuk fungsi aktivasi Sigmoid. Jika probabilitas skor prediksi wajah di atas 0.50, maka *frame* tersebut diklasifikasikan sebagai individu terindikasi manipulasi.
2. Video Ratio Threshold (0.30) : Menetapkan ambang batas keputusan akhir video pada rasio 30% (bukan 50%) sebagai *safety margin*. Hal ini bertujuan meminimalisir *False Negative* pada video berkualitas tinggi di mana manipulasi mungkin hanya berupa *glitch* sesaat atau terlihat pada sudut pandang tertentu. Jika minimal sepertiga sampel terdeteksi palsu, maka video divonis sebagai *deepfake*.

3. Face Limit (3) : Sistem membatasi analisis maksimal pada tiga wajah terbesar per *frame*. Selain untuk optimasi latensi sistem agar tetap ringan, batasan ini bertujuan menjaga fokus deteksi pada subjek utama (aktor disinformasi) dan mereduksi *noise* dari kerumunan di latar belakang yang sering kali beresolusi rendah. Hal ini untuk menghindari penggunaan komputasi yang berlebihan ketika model melakukan analisis.
4. min_face_size (20px) : Menetapkan batas minimal dimensi wajah sebesar 20x20 piksel. Batasan ini krusial untuk menjaga validitas data visual; wajah di bawah ukuran tersebut akan menghasilkan citra buram (*pixelated*) saat dilakukan *upscaling* ke input EfficientNet (224x224), yang berisiko menghilangkan fitur mikro deteksi.
5. mtcnn_confidence_threshold (0.85) : Berfungsi sebagai filter validitas untuk memastikan objek yang diekstraksi adalah wajah manusia yang jelas. Standar tinggi ini diterapkan untuk mencegah *garbage in, garbage out* dengan mengeliminasi pola acak latar belakang yang menyerupai wajah.

Secara keseluruhan, konfigurasi parameter ini dirancang untuk menciptakan keseimbangan antara sensitivitas (kemampuan mendeteksi manipulasi halus) dan spesifisitas (kemampuan menekan deteksi palsu akibat kualitas video yang buruk). Berikut adalah implementasi kode untuk logika ambang batas (*threshold*) :

```
import numpy as np

# Konfigurasi Threshold Keputusan
  FRAME_FAKE_THRESHOLD = 0.5   # Batas skor per frame
  VIDEO_RATIO_THRESHOLD = 0.30 # Batas persentase frame untuk vonis video

  def predict_video(cap, detector, model):
  all_scores = np.array(all_scores)

# Threshold frame yang mencurigakan (Score > 0.5)
   suspicious_frames_mask = scores_np > FRAME_FAKE_THRESHOLD
   suspicious_count = np.sum(suspicious_frames_mask)

# Threshold Rasio Kecurigaan
  if len(safe_scores) > 0:
      return "NoFace", 0.0,  [], inference_time
  suspicious_ratio = suspicious_count / len(all_scores)
```

```
# Pengambilan Keputusan Akhir
    if suspicious_ratio > VIDEO_RATIO_THRESHOLD:
        label = "Deepfake"
        final_score = np.mean(all_scores [suspicious_frames_mask])
    else:
        label = "Asli"
        safe_scores = all_scores [~suspicious_frames_mask]
        if len(safe_scores) > 0:
            final_score = 1.0 - np.mean(safe_scores)
        else:
            final_score = 0.0
```

D. Pelatihan Model (*Training Model*)

Proses pelatihan dilakukan dalam dua fase strategis: *Feature Extraction* (pembelajaran pada *classifier* saja) dan *Fine-tuning* (pembelajaran pada seluruh jaringan). Data log pelatihan menunjukkan dinamika pembelajaran model dalam meminimalkan *loss* dan meningkatkan *accuracy*. Pelatihan dilakukan dalam dua fase untuk menjaga integritas bobot *pre-trained*. Berdasarkan referensi standar, *hyperparameter* yang ditetapkan untuk pelatihan adalah :

Tabel 7. *Hyperparameter* yang digunakan

Hyperparameter	Nilai
Optimizer	Adam
Loss Function	BCEWithLogitsLoss
Batch Size	32
Learning Rate	1e-3 (0.001, fase 1) dan 1e-5 (0.00001, fase 2)

1. Pelatihan Model (Perangkat *High tier*)
    a. Pelatihan Model Menggunakan 300 Data

    Pelatihan model pada awalnya dilakukan dengan menggunakan 300 data (150 video asli dan 150 video *deepfake*), perbandingan ini dilakukan dengan tujuan untuk melihat bagaimana kinerja model jika menggunakan dataset yang variatif. Proses pelatihan berlangsung selama 25 menit 18 detik.

Bagian ini menyajikan analisis forensik terhadap proses pelatihan model, membedah kurva *loss* dan *accuracy* untuk memahami perilaku pembelajaran model. Penelitian menerapkan strategi pelatihan dua fase : fase *Feature Extraction* dan fase *Fine-Tuning*. Pendekatan ini bukan sekadar prosedur teknis, melainkan strategi fundamental untuk mencegah rusaknya bobot *pre-trained* akibat gradien besar pada awal pelatihan.

1. Analisis Fase 1 : *Feature Extraction*

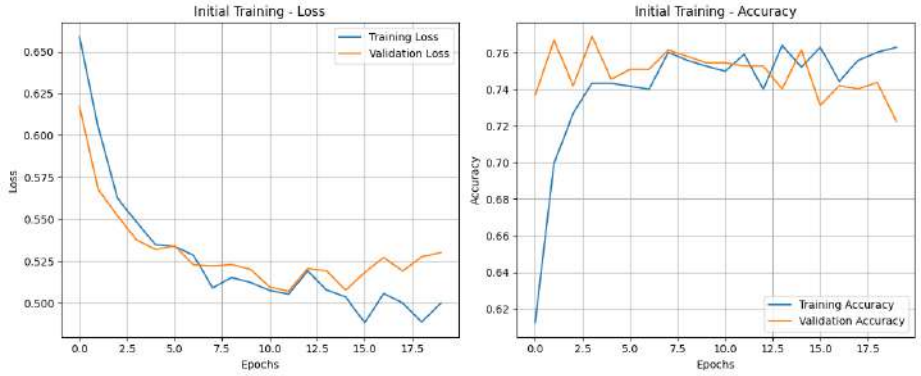
Pada fase ini, seluruh *convolutional layers* (*backbone*) dibekukan (*freeze*) sehingga bobotnya tidak berubah. Pelatihan hanya dilakukan pada *classifier head* yang baru dibuat. Tujuannya adalah mengadaptasi pemetaan fitur generik ke ruang keputusan biner tanpa merusak ekstraktor fitur yang sudah stabil, agar menyesuaikan bobot *layers,* klasifikasi beradaptasi dengan fitur yang dihasilkan *backbone* tanpa merusak ekstraktor fitur yang sudah benar. Fase ini dilakukan selama 20 *epoch* dengan *learning rate* standar. *Learning Rate* (LR) diatur pada 1 x 10^{-3} (1e-3) dengan optimizer Adam.

Tabel 8. Ringkasan Hasil Pelatihan Fase 1 (300 Data)

Epoch	**train_loss**	**train_acc**	**val_loss**	**val_acc**
1/20	0.6588	61.25%	0.6173	73.67%
5/20	0.5348	74.33%	0.5319	74.56%
10/20	0.5123	75.26%	0.5201	75.44%
20/20	0.5000	76.30%	0.5302	72.24%

Dapat diamati pada gambar 34 menunjukkan bahwa, meskipun *backbone* dibekukan, akurasi pelatihan Dimulai dari 61% dan meningkat secara moderat hingga 76% pada *epoch* ke-20 dan akurasi validasi mendekati 75%. Ini membuktikan bahwa fitur visual generik (tepi, tekstur) sudah cukup kuat untuk memberikan sinyal deteksi awal. Dengan gap antara *train loss* (0.50) dan *val loss* (0.53) menunjukkan bahwa model belum mengalami *overfitting*, namun kapasitas belajarnya

terbatas karena sebagian besar bobot tidak diperbarui. Inilah alasan mengapa *fine-tuning* pada Fase 2 diperlukan.



Gambar 34. Kurva Fase 1 – *Initial Training* (300 Data)

Progres pada akurasi validasi di kisaran 72-75% mengindikasikan adanya underfitting struktural. Fitur-fitur generik ImageNet memang mampu membedakan wajah asli dan palsu pada tingkat kasar (mungkin berdasarkan artefak visual yang sangat jelas), namun tidak memiliki resolusi fitur yang cukup untuk menangkap manipulasi halus. Model telah mencapai puncak performa (*performance ceiling*) saat fase *freeze*. Ini membuktikan bahwa transfer learning tanpa penyesuaian bobot backbone tidak memadai untuk tugas deteksi Deepfake yang bernuansa tinggi. Fase ini, bagaimanapun, berhasil memberikan inisialisasi bobot yang stabil pada classifier, mencegah gradien yang meledak saat memasuki fase kedua.

2. Analisis Fase 2 : *Fine-tuning*

Setelah classifier stabil, beberapa bobot *convolutional blocks* membuka kunci (*unfreeze*) pada lapisan-lapisan atas backbone *EfficientNet*. Model dilatih kembali dengan *learning rate* yang sangat rendah dengan *learning rate* (LR) diatur pada 1 x 10^{-5} (1e-5) dengan optimizer Adam. Fase ini dilakukan selama 30 *epoch*. Hal ini memungkinkan model untuk sedikit menyesuaikan ekstraktor fiturnya agar lebih sensitif terhadap artefak spesifik *deepfake* yang mungkin tidak ada di ImageNet, seperti inkonsistensi tekstur.

Tabel 9. Ringkasan Hasil Pelatihan Fase 2 (300 Data)

Epoch	**train_loss**	**train_acc**	**val_loss**	**val_acc**
1/30	0.5307	75.15%	0.5131	77.05%
15/30	0.2683	90.26%	0.4403	82.92%
30/30	0.1287	95.79%	0.4743	83.10%
Best	0.1287	95.35%	0.4299	85.23%

Fase *fine-tuning* pada gambar 35 menunjukkan peningkatan performa yang signifikan. *Training loss* turun drastis dari 0.53 ke 0.12, menandakan model semakin mahir mempelajari detail-detail halus pada data latih. Dengan Tingkat *validation loss* dari 0.53 ke 0.42. Hasil pelatihan mencapai puncaknya (*best*), *training accuracy* di angka 95.35% dan *validation accuracy* di angka 85.23%.

Meskipun *train accuracy* menyentuh angka 95%, *validation accuracy* tertahan di kisaran 84-85%. Kesenjangan ini mengindikasikan mulai terjadinya *overfitting*, di mana model terlalu spesifik menghafal *noise* pada data latih. Namun, penggunaan teknik augmentasi data (kompresi JPEG), generalisasi model tetap terjaga pada tingkat yang baik. Model terbaik yang disimpan untuk menjamin keandalan saat pengujian (*test model*).

Gambar 35.Kurva Fase 2 – *Fine-tuning* (300 Data)

Analisis terhadap selisih (*gap*) antara akurasi pelatihan 95% dan akurasi validasi 83% pada *epoch* akhir mengindetifikasikan adanya gejala *overfitting* sebesar 12%. Model mulai "menghafal" data latih. Namun, perlu dicatat bahwa *validation loss* terendah tercatat sebesar 43%. Strategi *Early Stopping* atau pengambilan *checkpoint* diterapkan untuk mendapatkan model terbaik diambil berdasarkan *validation loss* minimum menjadi mekanisme pertahanan utama. Meskipun terjadi *overfitting* di akhir pelatihan, model pada titik optimalnya (akurasi validasi 85%) menunjukkan kemampuan generalisasi yang kuat. Penggunaan teknik augmentasi data yang mencakup kompresi JPEG acak dan distorsi warna, dan terbukti efektif dalam mencegah *validation loss* memburuk lebih jauh, memberikan model ketahanan terhadap variasi kualitas citra.

b. Pelatihan Model Menggunakan 400 Data

Pelatihan model dilakukan dengan menggunakan 400 data (200 video asli dan 200 video *deepfake*), perbandingan ini dilakukan dengan tujuan untuk melihat bagaimana kinerja model jika menggunakan dataset yang variatif. Proses pelatihan berlangsung selama 31 menit 17 detik. Proses pelatihan dilakukan dalam dua tahap: *Initial training* (*Feature Extraction*) dan *Fine-tuning*. Pendekatan ini dipilih untuk mencegah rusaknya bobot *pre-trained* akibat gradien besar pada awal pelatihan.

1. Analisis Fase 1 : *Feature Extraction*

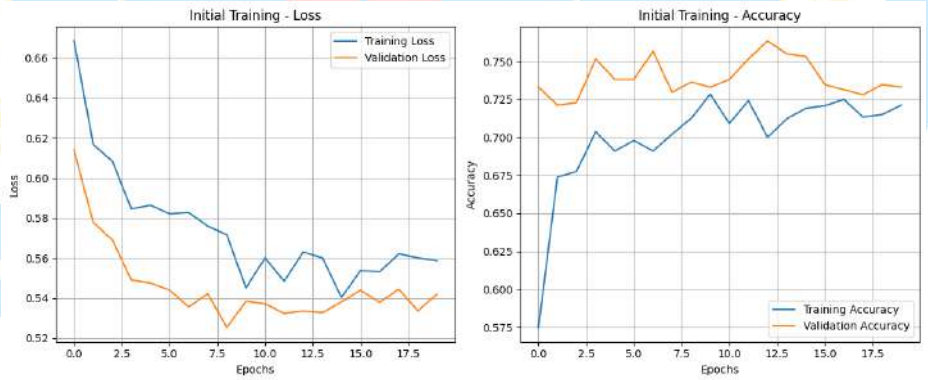
Mengacu pada grafik *Initial Training* gambar 36, fase ini melatih hanya lapisan *classifier* baru selama 20 *epoch* dengan *Learning Rate* (LR 1e-3). Terlihat penurunan *train loss* yang tajam pada 3 *epoch* pertama (dari 0.67 ke 0.58), menunjukkan bahwa *classifier* dengan cepat belajar memisahkan fitur dasar antara wajah asli dan palsu. Namun, *val loss* menunjukkan fluktuasi yang signifikan dan tidak menurun setajam *train Loss*. Bahkan pada *epoch* 15-20, *val loss* mulai menjauh dari *train loss*. Kurva *train accuracy* naik stabil hingga 0.72, namun *val accuracy* di sekitar 0.70 - 0.73.

Tabel 10. Ringkasan Hasil Pelatihan Fase 1 (400 Data)

Epoch	**train_loss**	**train_acc**	**val_loss**	**val_acc**
1/20	0.6686	57%	0.6139	73%
5/20	0.5865	69%	0.5475	74%
10/20	0.5450	72%	0.5383	73%
20/20	0.5587	72%	0.5419	73%

Dapat diamati pada gambar 36 menunjukkan bahwa, meskipun *backbone* dibekukan, model mampu mencapai *val accuracy* mendekati 73%. Ini membuktikan bahwa fitur visual generik (tepi, tekstur) sudah cukup kuat untuk memberikan sinyal deteksi awal.

Dengan gap antara *train loss* (0.56) dan *val loss* (0.54) menunjukkan bahwa model belum mengalami *overfitting*, namun kapasitas belajarnya terbatas karena sebagian besar bobot tidak diperbarui. Inilah alasan mengapa *fine-tuning* pada Fase 2 diperlukan.



Gambar 36. Kurva Fase 1 – *Initial Training* (400 Data)

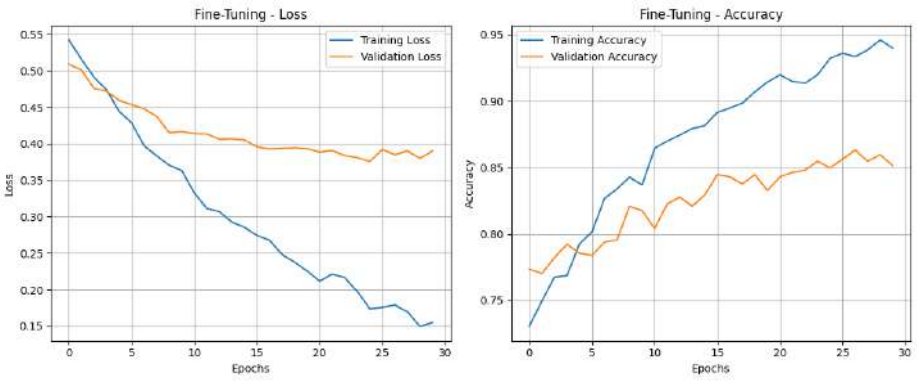
2. Analisis Fase 2 : *Fine-tuning*

Grafik *fine-tuning* pada gambar 37 menunjukkan dinamika saat lapisan *backbone* mulai dilatih ulang dengan *learning rate* (LR) diatur pada 1 x 10^{-5} (1e-5) selama 30 *epoch* tambahan, dengan optimizer Adam.

Tabel 11. Ringkasan Hasil Pelatihan Fase 2 (400 Data)

Epoch	**train_loss**	**train_acc**	**val_loss**	**val_acc**
1/30	0.5422	73%	0.5089	77%
15/30	0.2853	88%	0.4051	83%
30/30	0.1548	94%	0.3899	84%
Best	0.1495	95%	0.3758	86%

Pada gambar 37 terjadi penurunan menunujukkan *train loss* mencapai angka sangat rendah (0.15) pada akhir pelatihan. Yang menarik adalah *val loss* meskipun lebih tinggi dari *train loss*, menunjukkan penurunan yang konsisten hingga sekitar *epoch* ke-15 (total *epoch* 35), sebelum mendatar (*plateau*) di angka (0.38 - 0.40). Stabilitas *val loss* ini terlihat lebih baik dibandingkan model yang hanya menggunakan data 300 video yang cenderung naik kembali (*diverging*) di akhir pelatihan, menandakan bahwa penambahan data (400 video) dan lapisan *dropout* berhasil meredam *overfitting*. *Train accuracy* mendekati 95% (0.95), sementara *val accuracy* stabil di angka 85% (0.85). Kesenjangan (gap) sebesar 10% adalah hal yang wajar dalam tugas sulit seperti deteksi *Deepfake*, namun menunjukkan bahwa model sangat kuat dalam mempelajari data latih. Model terbaik yang disimpan untuk menjamin keandalan saat pengujian (*test model*).



Gambar 37. Kurva Fase 2 – *Fine-tuning* (400 Data)

2. Pelatihan Model (Perangkat *Low tier*)

Pelatihan model dilakukan dengan menggunakan 400 data (200 video asli dan 200 video *deepfake*), perbandingan ini dilakukan dengan tujuan untuk melihat bagaimana kinerja model jika menggunakan dataset yang variatif. Proses pelatihan berlangsung selama 41 Jam 55 menit. Proses pelatihan dilakukan dalam dua tahap : Pelatihan awal (*Feature Extraction*) dan *Fine-tuning*. Pendekatan ini dipilih untuk mencegah rusaknya bobot *pre-trained* akibat gradien besar pada awal pelatihan.

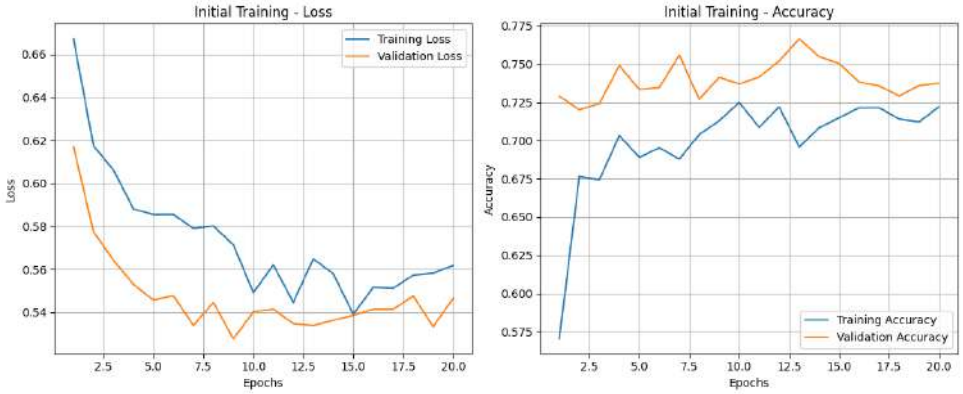
a. Analisis Fase 1 : *Feature Extraction*

Pada tahap awal ini, pelatihan dimulai dengan ”membekukan” (*freeze)* hal ini diterapkan untuk menjaga bobot *pre-trained* agar tidak rusak akibat gradien yang besar di awal propagasi. Pelatihan dilakukan selama 20 *epoch*. Berdasarkan tabel 12, terlihat perkembangan metrik evaluasi model dari *epoch* awal hingga akhir. Pada *epoch* ke-20, model mencapai akurasi pelatihan (*train_acc*) sebesar 72% dan akurasi validasi (*val_acc*) sebesar 73%. Nilai *loss* pada data validasi (*val_loss*) berhasil turun dari 0.6152 pada awal pelatihan menjadi 0.5427 di akhir tahap ini.

Tabel 12. Ringkasan Hasil Pelatihan Fase 1

Epoch	**train_loss**	**train_acc**	**val_loss**	**val_acc**
1/20	0.6691	57%	0.6152	73%
5/20	0.5872	69%	0.5482	74%
10/20	0.5458	73%	0.5391	73%
20/20	0.5595	72%	0.5427	73%

Pada gambar 38 (Kurva Fase 1) memperlihatkan dinamika pembelajaran model. Meskipun terdapat fluktuasi pada kurva *loss* dan *accuracy*, tren menunjukkan adanya pembelajaran yang positif di mana *loss* cenderung menurun dan akurasi meningkat perlahan, menandakan bahwa model mulai mengenali fitur-fitur dasar dari citra wajah.



Gambar 38. Kurva Fase 1 – *Initial Training* pada perangkat *local*

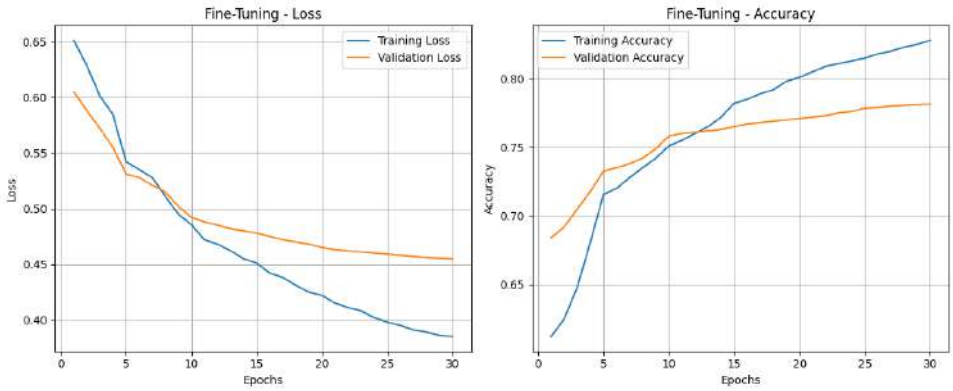
b. Analisis Fase 2 : *Fine-tuning*

Tahap kedua adalah *fine-tuning* dengan "mencairkan" (*unfreeze*) pada beberapa *layer* model untuk menyesuaikan bobot secara lebih spesifik terhadap dataset *deepfake*. Pelatihan ini dilanjutkan selama 30 *epoch*. Hasil pada tabel 13 menunjukkan peningkatan performa yang dibandingkan tahap pertama. Model dengan rincian sebagai berikut :

Tabel 13. Ringkasan Hasil Pelatihan Fase 2

Epoch	**train_loss**	**train_acc**	**val_loss**	**val_acc**
1/30	0.6512	61%	0.6050	68%
15/30	0.4510	78%	0.4780	76%
30/30	0.3850	83%	0.4450	78%
Best	0.3850	83%	0.4450	78%

Pada gambar 39 (Kurva Fase 2) mengilustrasikan konvergensi model yang lebih stabil dibandingkan fase pertama. Garis *train acc* terus menaik hingga mencapai 83%, sementara *val acc* stabil di angka 78%.



Gambar 39. Kurva Fase 2 – *Fine-tuning* pada perangkat local

E. Analisis Hasil Pelatihan

Hasil pelatihan dimonitor melalui kurva *loss* dan *accuracy* pada data latih dan validasi.

1. Analisis Kurva Loss

Berdasarkan visualisasi data pelatihan, terlihat perbandingan antara *training loss* dan *validation loss*. Penggunaan data validasi di sini sangat penting untuk memastikan bahwa model tidak hanya menghafal data latih *(Overfitting)*.

Kurva *Validation Loss* (garis warna oren) menunjukkan tren penurunan yang sejalan dengan *training loss*, yang mengindikasikan bahwa model mampu melakukan generalisasi dengan baik. Selama proses pelatihan, bobot model yang disimpan adalah bobot yang menghasilkan akurasi tertinggi pada data validasi ini. Dengan demikian, model yang dihasilkan adalah model yang paling optimal dan stabil, bukan sekadar model yang memiliki akurasi tinggi pada data latih namun buruk saat divalidasi.

a. *Training Loss* menurun tajam di awal, mengindikasikan model cepat mempelajari fitur pembeda dasar.
b. *Validation Loss* mengikuti tren *training loss* dengan jarak yang kecil, menandakan model memiliki kemampuan generalisasi yang baik dan minim *overfitting* berkat penggunaan lapisan *dropout*.

2. Analisis Kurva Akurasi

Kurva akurasi menunjukkan peningkatan performa deteksi. Pada akhir pelatihan akurasi validasi mencapai titik optimal yang stabil. Hal ini mengkonfirmasi bahwa penyesuaian bobot berhasil meningkatkan sensitivitas model terhadap manipulasi wajah yang halus tanpa mengorbankan kemampuan mengenali wajah asli.

F. Perbandingan Lingkungan Uji Coba

Salah satu kontribusi utama dari penelitian ini adalah analisis mengenai implikasi perangkat keras terhadap kinerja sistem deteksi *Deepfake*. Analisis ini membahas perbedaan kemampuan komputasi antara perangkat kelas Low Tier dan High Tier yang digunakan dalam proses pelatihan model. Perbedaan performa keduanya terutama dipengaruhi oleh arsitektur pemrosesan data dan kemampuan memori yang dimiliki masing-masing perangkat.

1. Arsitektur Prosesor (CPU dan GPU)

CPU Intel Core i5 pada perangkat *low tier* memiliki arsitektur yang dirancang untuk menjalankan tugas secara berurutan dengan waktu respon yang rendah. CPU umumnya memiliki jumlah core yang terbatas, sehingga kemampuan pemrosesannya tidak terlalu besar. Perangkat ini kurang optimal untuk pekerjaan yang membutuhkan perhitungan paralel dalam jumlah besar seperti pelatihan model *deep learning*.

Sebaliknya, GPU NVIDIA Tesla T4 dan A100/V100 pada perangkat *high tier* dirancang dengan arsitektur pemrosesan paralel. GPU ini memiliki ribuan core yang mampu menjalankan banyak operasi secara bersamaan. Kehadiran CUDA cores dan tensor cores membuat GPU lebih efisien dalam melakukan operasi matematika yang menjadi dasar pada proses *convolution neural network* (CNN). Tensor cores juga mendukung penggunaan mixed precision yang mempercepat proses pelatihan model. Oleh karena itu, GPU memiliki performa yang jauh lebih cepat dibanding CPU dalam melakukan komputasi *deep learning*.

2. Penyimpanan (Memori dan RAM Storage)

Kendala utama pada *low tier* bukan hanya kecepatan komputasi, tetapi juga memori. Intel Iris Xe berbagi memori sistem (DDR4) dengan CPU. Operasi pelatihan model membutuhkan pemindahan data tensor yang sangat besar secara konstan antara memori dan unit pemrosesan (~30-40 GB/s).

Di sisi lain, Tesla T4 memiliki 16 GB RAM GDDR6 dengan unit pemprosesan 320 GB/s. Kapasitas RAM 16 GB memungkinkan penggunaan *batch size* yang lebih besar (16, 32 atau 64), yang menstabilkan estimasi dan mempercepat pelatihan. Pada *low tier* dengan 8 GB RAM (sebagian besar digunakan untuk OS), *batch size* harus sangat kecil (1-5) untuk menghindari kesalahan Out of Memory (OOM), yang secara drastis memperlambat pelatihan.

3. Evaluasi Performa Fase Pelatihan

Perbedaan performa waktu fase pelatihan antara kedua perangkat :

Tabel 14. Analisis Waktu Pelatihan

Metrik	*Low tier* (Asus VivoBook)	*High tier* (Google Colab)
Fase 1 (*Feature Extraction*) (20 *Epoch*)	16 Jam 24 Menit	17 Menit
Fase 2 (*Fine-tuning*) (30 *Epoch*)	26 Jam 12 Menit	7 Menit
Waktu Per *Epoch* Fase 1	45 - 60 Menit	2 - 3 Menit
Waktu Per *Epoch* Fase 2	60 - 90 Menit	1 - 2 Menit
Total Waktu (50 *Epoch*)	41 Jam 55 Menit	31 Menit
Konsumsi Daya	Tinggi, Risiko *Throttling*	Dikelola Pusat Data

Hasil analisis, melakukan pelatihan model *EfficientNet-B0* pada perangkat *low tier* terbukti tidak layak secara praktis. Waktu yang dibutuhkan untuk menyelesaikan satu *epoch* bisa mencapai satu jam. Dengan total 50 *epoch* yang direncanakan, proses akan memakan waktu sangat lama bahkan berhari-hari.

Selain itu, beban kerja 100% yang berkelanjutan pada perangkat seperti Laptop Asus VivoBook akan memicu thermal throttling, di mana frekuensi CPU diturunkan paksa untuk mencegah panas berlebih, yang semakin memperlambat proses. Sebaliknya, lingkungan berbasis *cloud* perangkat *high tier* menyelesaikan seluruh proses pelatihan (Fase 1 dan 2) hanya dalam waktu sekitar 30-31 menit. Ini membuktikan bahwa akses ke akselerasi GPU *cloud* adalah prasyarat untuk fase pengembangan dan pelatihan model *Deep learning* yang lebih *superior*.

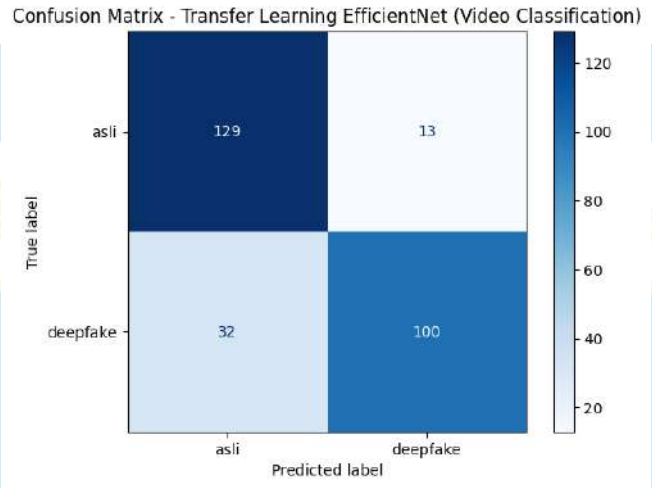
Berikut adalah analisis dan evaluasi dari hasil perbandingan lingkungan uji coba :

a. Uji Kelayakan, data menunjukkan bahwa perangkat *low tier* mampu memproses video dalam waktu (45-60 menit). Ini menandakan bahwa sistem deteksi ini dapat diimplementasikan pada perangkat laptop standar tanpa memerlukan koneksi ke *cloud* GPU untuk pemrosesan.
b. Bottleneck Pra-pemrosesan, Analisis lebih lanjut menunjukkan bahwa sebagian besar waktu pada *low tier* tidak dihabiskan untuk inferensi model *EfficientNet*, melainkan pada tahap Pra-pemrosesan, khususnya deteksi wajah dengan MTCNN. MTCNN adalah proses CPU-intensif. Pada *low tier*, proses ini sangat lambat, sementara pada *high tier*, CPU server yang kuat dapat menanganinya dengan cepat sebelum mengirim data ke GPU.
c. Efisiensi *EfficientNet*, kemampuan Asus VivoBook untuk menjalankan inferensi memvalidasi pilihan arsitektur *EfficientNet-B0*. Model yang lebih berat seperti Xception atau ResNet-152 mungkin akan membutuhkan waktu 2-3 kali lebih lama pada perangkat yang sama, membuat pengalaman pengguna menjadi buruk atau setidaknya tidak lebih baik.

G. Evaluasi Model

1. Evaluasi Kinerja Model

Evaluasi akhir dilakukan pada Data Uji (Test Set). Kinerja model divisualisasikan menggunakan *confusion matrix*. Matriks ini memetakan prediksi model terhadap label sebenarnya. Berikut merupakan *confusion matrix* model yang menggunakan 300 video yang diperoleh sesuai pada Gambar 40 berikut:



Gambar 40. *Confusion Matrix* (300 Data)

Berdasarkan *confusion matrix* tersebut, maka dapat diketahui bahwa terdapat 129 video asli yang berhasil diprediksi dengan benar (*True Positive*/TP) dan 100 video *deepfake* yang juga berhasil diprediksi dengan benar (*True Negative*/TN). Selain itu, terdapat 13 video asli yang salah diprediksi sebagai *deepfake* (*False Positive*/FP) dan 32 video *deepfake* yang salah diprediksi sebagai asli (*False Negative*/FN), hal ini menunjukkan beberapa manipulasi sangat halus sehingga menyerupai video asli sehingga model cenderung lebih sering melewatkan *deepfake* daripada salah menandai video asli. Berdasarkan nilai-nilai tersebut, metrik performa dihitung sebagai berikut :

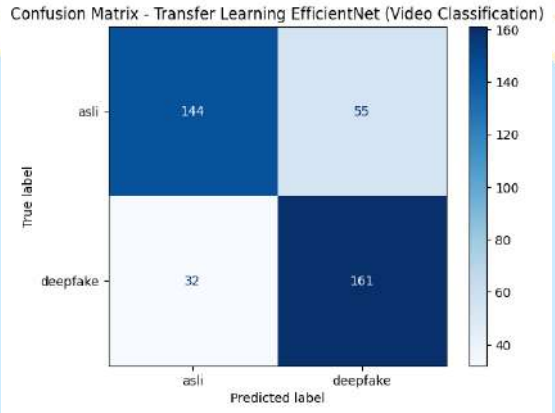
1. Akurasi (*Accuracy*) : $\frac{100 + 129}{100 + 129 + 13 + 32} \approx 83.6\%$
2. Presisi (*Precision*) : $\frac{100}{100 + 13} \approx 88.5\%$
3. Recall (*Sensitivitas*) : $\frac{100}{100 + 32} \approx 75.8\%$
4. F1-Score : $2 \times \frac{88.5\% \times 75.8\%}{88.5\% + 75.8\%} \approx 81.6\%$

Tabel 15. Metrik Evaluasi (300 Data)

Accuracy	*Presision*	*Recall*	*F1-Score*
0.8358	0.8850	0.7576	0.8163

Hasil ini menunjukkan bahwa model *EfficientNet-B0* memiliki performa yang cukup handal, terutama dalam hal presisi, yang berarti sistem jarang memberikan peringatan palsu (*false alarm*).

Evaluasi dilakukan menggunakan *confusion matrix* baru yang dihasilkan dari pengujian terhadap data uji dataset 400 video. Berikut merupakan *confusion matrix* yang diperoleh sesuai pada Gambar 41 berikut :



Gambar 41. *Confusion Matrix* (400 Data)

Berdasarkan *confusion matrix* tersebut, maka dapat diketahui bahwa terdapat 144 video asli yang berhasil diprediksi dengan benar (*True Positive*/TP) dan 161 video *deepfake* yang juga berhasil diprediksi dengan benar (*True Negative*/TN). Selain itu, terdapat 55 video asli yang salah diprediksi sebagai *deepfake* (*False Positive*/FP) dan 32 video *deepfake* yang salah diprediksi sebagai asli (*False Negative*/FN), hal ini menunjukkan beberapa manipulasi sangat halus sehingga menyerupai video asli sehingga model cenderung lebih sering melewatkan *deepfake* daripada salah menandai video asli. Berdasarkan nilai-nilai tersebut, metrik performa dihitung sebagai berikut :

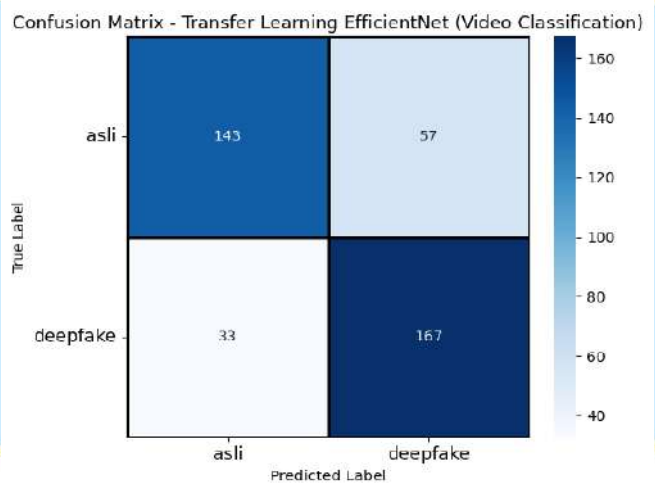
1. Akurasi (*Accuracy*) : $\frac{161 + 144}{161 + 144 + 55 + 32} \approx 77.8\%$
2. Presisi (*Precision*) : $\frac{161}{161 + 55} \approx 74.6\%$
3. Recall (*Sensitivitas*) : $\frac{161}{161 + 32} \approx 83.4\%$
4. F1-Score : $2 \times \frac{74.6\% \times 83.4\%}{74.6\% + 83.4\%} \approx 78.7\%$

Tabel 16. Metrik Evaluasi (400 Data)

Accuracy	*Presision*	*Recall*	*F1-Score*
0.7781	0.7454	0.8342	0.7877

Hasil ini menunjukkan bahwa model *EfficientNet-B0* memiliki performa yang cukup handal, terutama dalam hal sensitivitas, yang berarti sistem mengindikasikan bahwa model memiliki tingkat kepercayaan (*confidence*) yang lebih baik pada data yang belum pernah dilihat.

Selanjutnya adalah evaluasi untuk lingkungan uji coba lokal dilakukan menggunakan *confusion matrix* baru yang dihasilkan dari pengujian terhadap data uji dataset 400 video. Berikut merupakan *confusion matrix* yang diperoleh sesuai pada Gambar 42 berikut:



Gambar 42. *Confusion Matrix* pada Perangkat *Local*

Berdasarkan *confusion matrix* tersebut, maka dapat diketahui bahwa terdapat 143 video asli yang berhasil diprediksi dengan benar (*True Positive*/TP) dan 167 video *deepfake* yang juga berhasil diprediksi dengan benar (*True Negative*/TN). Selain itu, terdapat 57 video asli yang salah diprediksi sebagai *deepfake* (*False Positive*/FP) dan 33 video *deepfake* yang salah diprediksi sebagai asli (*False Negative*/FN), hal ini menunjukkan beberapa manipulasi sangat halus sehingga menyerupai video asli sehingga model cenderung lebih sering melewatkan *deepfake* daripada salah menandai video asli. Berdasarkan nilai-nilai tersebut, metrik performa dihitung sebagai berikut :

1. Akurasi (*Accuracy*) : $\frac{167 + 143}{167 + 143 + 57 + 33} \approx 77.5\%$
2. Presisi (*Precision*) : $\frac{167}{167 + 57} \approx 74\%$
3. Recall (*Sensitivitas*) : $\frac{167}{167 + 33} \approx 83.5\%$
4. F1-Score : $2 \times \frac{88.5\% \times 75.8\%}{88.5\% + 75.8\%} \approx 78.7\%$

Tabel 17. Metrik Evaluasi

Accuracy	*Presision*	*Recall*	*F1-Score*
0.7750	0.7405	0.8350	0.7870

Hasil ini menunjukkan bahwa model *EfficientNet-B0* memiliki performa yang cukup handal, terutama dalam hal sensitivitas, yang berarti sistem mengindikasikan bahwa model memiliki tingkat kepercayaan (*confidence*) yang lebih baik pada data yang belum pernah dilihat.

2. Perbandingan dan Pemilihan Model Terbaik

a. Analisis Komparatif Dataset (300 Dataset vs 400 Dataset)

Berdasarkan serangkaian pengujian empiris yang telah dilakukan terhadap dua varian model yang dilatih dengan jumlah dataset berbeda (300 data dan 400 data), tahap ini bertujuan untuk menentukan model mana yang paling layak untuk diimplementasikan ke dalam sistem akhir "DEFEND". Pemilihan model terbaik didasarkan pada analisis komparatif terhadap metrik evaluasi utama, yaitu akurasi (*Accuracy*), presisi (*Precision*), *Recall*, dan *F1-Score*, serta pertimbangan stabilitas performa pada skenario penggunaan dunia nyata. Perbandingan kinerja kedua model secara komprehensif disajikan kembali pada tabel 18 berikut untuk memudahkan analisis.

Tabel 18. Perbandingan Metrik Evaluasi Model

Jumlah Dataset	*Accuracy*	*Precision*	*Recall*	*F1-Score*
300	**0.8718**	**0.8850**	**0.7580**	**0.8160**
400	0.7781	0.7454	0.8342	0.7873
Komparatif	- 9.37	- 13.96	+ 7.62	- 2.87

Dalam penelitian ini, dilakukan eksperimen perbandingan kinerja model menggunakan dua variasi jumlah dataset, yaitu 300 video dan 400 video. Berdasarkan hasil pengujian, ditemukan fenomena di mana model yang dilatih dengan 300 video menghasilkan akurasi yang lebih baik (87.18%) dibandingkan dengan model yang dilatih dengan 400 video. Penurunan performa pada dataset yang lebih besar (400 data) disebabkan oleh beberapa faktor teknis :

1. Variabilitas dan Noise : Penambahan dataset mentah meningkatkan varian baru yang harus dipelajari oleh model. Indikasi menunjukkan bahwa data pada skenario 400 video dengan tambahan 100 video mengandung *noise* (gangguan) yang lebih tinggi. Contohnya: wajah yang kurang jelas, pergerakan mulut yang tiba tiba tersenyum, pencahayaan yang sangat buruk, atau *angle* wajah yang kurang bersih yang menyebabkan model *EfficientNet* "bingung" karena terpaksa belajar dari dataset kotor. Proses ekstraksi fitur oleh *EfficientNet* menjadi kurang optimal dibandingkan pada dataset 300. Ini menurunkan akurasi total. Pada dataset 300, variasi datanya seragam (mudah dipelajari). Saat ditambah menjadi 400, variasi menjadi terlalu luas sementara *hyperparameter* (seperti *learning rate* atau jumlah *epoch*) tidak disesuaikan ulang. Akibatnya, model belum sempat *converge* (matang) memahami pola baru tersebut.
2. Optimasi *Hyperparameter* : Dengan parameter pelatihan (*Batch Size* atau *Learning Rate*) dijaga tetap sama, model pada skenario 400 data belum mencapai titik penyesuaian optimal dalam jumlah *epoch* yang ditentukan. Hal ini menyebabkan model mengalami kesulitan dalam memetakan artefak *deepfake* yang lebih kompleks dari data tambahan tersebut.

b. Analisis *Trade-off : Presisi* dan *Recall*

Data di atas melukiskan gambaran yang jelas mengenai karakteristik kedua model :

1. Model B (400 Video) : Penambahan 100 video meningkatkan *Recall* secara signifikan (+7.62%). Ini berarti Model B lebih "agresif" dan mampu menangkap lebih banyak sampel *Deepfake* yang mungkin terlewatkan oleh Model A. Namun, agresivitas ini bersifat brutal, yang terlihat dari penurunan *Presisi* yang tajam dari (-13.96%). Model B memiliki kecenderungan tinggi untuk menghasilkan *False Positive*, yaitu salah menuduh video asli sebagai palsu.
2. Model A (300 Video) : Model A menunjukkan presisi yang superior 89%. Ini mengindikasikan bahwa ketika Model A menyatakan sebuah video adalah *Deepfake*, tingkat kepercayaannya sangat tinggi. Tingkat *False Positive* yang rendah pada Model A adalah aset krusial dalam konteks forensik.

c. Kualitas Data dan Noise

Penurunan performa pada Model B kemungkinan besar disebabkan oleh karakteristik dari 100 video tambahan tersebut. Jika video tambahan ini memiliki distribusi kualitas yang lebih rendah, mengandung artefak kompresi yang tidak relevan, atau memiliki label yang ambiguitasnya tinggi (*noisy labels*), mereka akan bertindak sebagai "polusi" dalam proses pembelajaran (*learning process*). *EfficientNet-B0*, dengan kapasitas parameternya yang terbatas, mungkin dipaksa untuk membagi sumber dayanya untuk mengakomodasi *outlier* dari data tambahan ini, yang pada akhirnya mendegradasi *decision boundary* yang telah terbentuk dengan baik dari 300 video inti. Hal ini menegaskan hipotesis bahwa dalam deteksi *deepfake*, dataset yang ketat untuk memastikan representasi fitur manipulasi yang bersih lebih baik daripada sekadar volume data.

d. Implikasi Strategis untuk Implementasi

Dalam skenario dunia nyata, *false positive* sering kali jauh lebih tinggi daripada *false negative*. Menuduh video asli sebagai *deepfake* dapat memiliki

konsekuensi hukum dan reputasi yang fatal. Oleh karena itu, penelitian ini merekomendasikan penggunaan Model A (300 Video) sebagai konfigurasi optimal. Keunggulan presisi 88.50% memberikan tingkat jaminan yang lebih aman bagi pengguna sistem, menjadikan hasil deteksi lebih dapat diandalkan sebagai bukti awal.

Oleh karena itu, keputusan untuk menggunakan model dengan 300 data sebagai model final didasarkan pada stabilitas dan reliabilitas. Model 300 data menunjukkan grafik *loss* validasi yang lebih stabil dan metrik evaluasi (terutama akurasi) yang lebih unggul, yang membuktikan bahwa model ini lebih efektif dalam mendeteksi pola *deepfake* pada lingkup data yang tersedia.

H. Tampilan User Interface

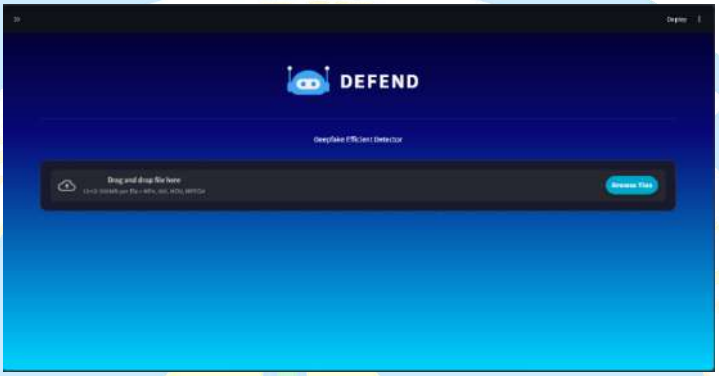
1. Halaman Dashboard

Saat pertama kali membuka aplikasi, pengguna akan disambut oleh halaman Dashboard. Halaman ini memberikan gambaran umum mengenai sistem. Pada bagian *above/header* terlihat judul sistem "*Deepfake* Efficient Detector" dengan deskripsi singkat. Pada bagian *below/footer* diberikan petunjuk penggunaan yang telah disederhanakan agar mudah dimengerti pengguna. Untuk mencoba fitur utama sistem silakan klik tombol berwarna biru cerah bertuliskan "Try Now" di bagian tengah layar untuk langsung masuk ke sistem deteksi utama.

Gambar 43. Halaman Dashboard

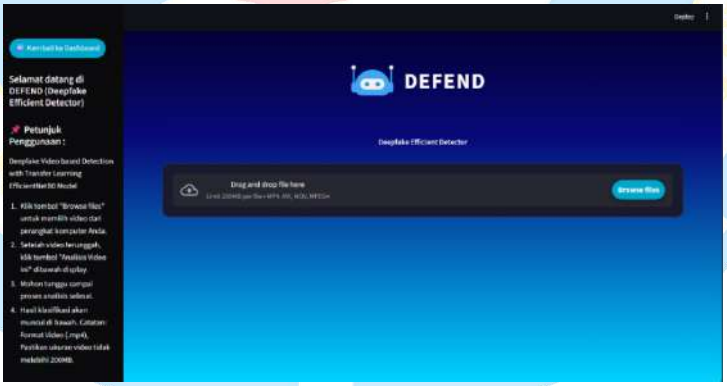
2. Halaman Main System

Pada halaman sistem utama, pengguna akan melihat area kotak bertuliskan *"Drag and drop file here"* untuk menggungah video yang dinginkan. File uploader Widget `st.file_uploader` yang membatasi input hanya pada format seperti, `.MP4`, `.MOV`, dan `.FLV`. Widget ini menangani *streaming* data video ke *server* sementara untuk diproses. Catatan, pastikan ukuran video tidak melebihi 200MB dan durasi dibawah 60 detik untuk performa analisis yang optimal.



Gambar 44. Halaman Utama Sistem

Dibagian kiri disediakan *sidebar* konfigurasi untuk navigasi, informasi tentang model, dan tata cara penggunaan, jika pengguna tidak ingat cara menggunakan sistem.



Gambar 45. Halaman Utama Sistem (*Sidebar*)

3. Halaman Main System - Display Upload Video

Setelah berhasil diunggah diunggah dan muncul di layar, sistem menggunakan *video player* untuk menampilkan pratinjau video beserta judul yang diunggah agar pengguna dapat memverifikasi file sebelum analisis. Langkah berikutnya klik tombol "Analisis Video Ini", sistem akan mulai memproses video

frame demi *frame*. Mohon tunggu sejenak, durasi analisis bergantung pada panjang video yang diunggah. Sistem ini dirancang menggunakan model *EfficientNet-B0* yang telah dilatih untuk mengenali manipulasi wajah yang sangat halus sekalipun.

Gambar 46. Halaman Utama Sistem – Video Terunggah

4. Halaman Main System - Analysis Result - Asli

Gambar 47. Halaman Utama Sistem - Hasil Klasifikasi – Asli

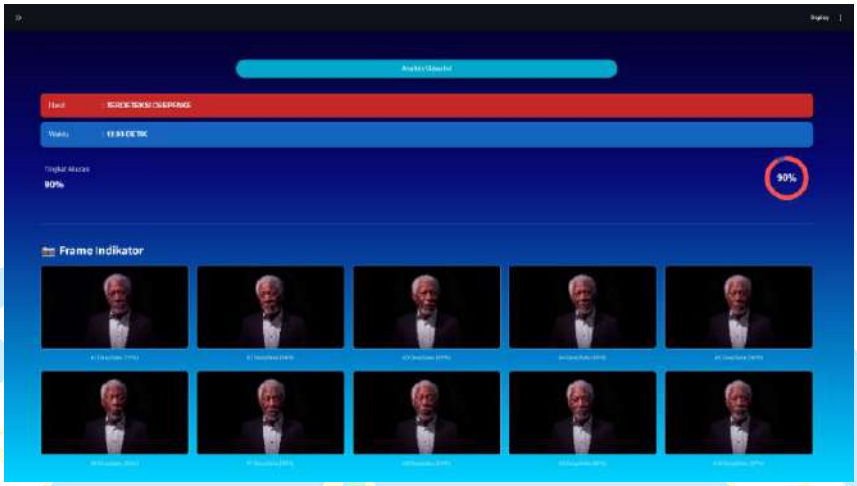
Setelah proses selesai, hasil akan ditampilkan secara visual dan mudah dipahami, dengan status hasil (warna kotak) sebagai berikut :

a. HIJAU (Terdeteksi ASLI): Video memiliki probabilitas tinggi sebagai video asli tanpa manipulasi.

b. Tingkat Keyakinan (Donut Chart): di sebelah kanan, pengguna akan melihat diagram lingkaran yang menunjukkan seberapa yakin sistem terhadap keputusannya (misalnya: 98%). Semakin tinggi persentasenya, semakin akurat prediksinya.

c. Kecepatan: Sistem juga akan menampilkan berapa detik waktu yang dibutuhkan untuk memproses video tersebut.

5. Halaman Main System - Analysis Result - *Deepfake*



Gambar 48. Halaman Utama Sistem - Hasil Klasifikasi – *Deepfake*

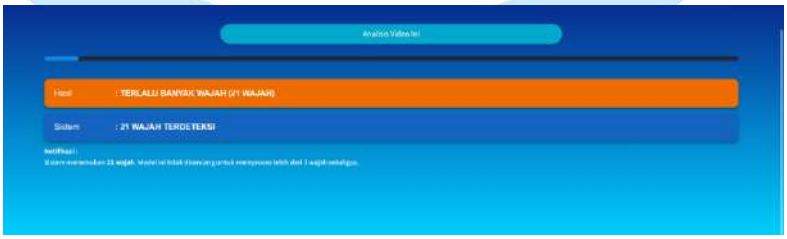
Setelah proses selesai, hasil akan ditampilkan secara visual dan mudah dipahami, dengan status hasil (warna kotak) sebagai berikut:

a. MERAH (Terdeteksi *DEEPFAKE*): Sistem menemukan jejak manipulasi digital pada wajah dalam video.

b. Tingkat Keyakinan (Donut Chart): di sebelah kanan, pengguna akan melihat diagram lingkaran yang menunjukkan seberapa yakin sistem terhadap keputusannya (misalnya: 98%). Semakin tinggi persentasenya, semakin akurat prediksinya.

c. Kecepatan: Sistem juga akan menampilkan berapa detik waktu yang dibutuhkan untuk memproses video tersebut.

6. Halaman Main System - Analysis Result – TooManyFace

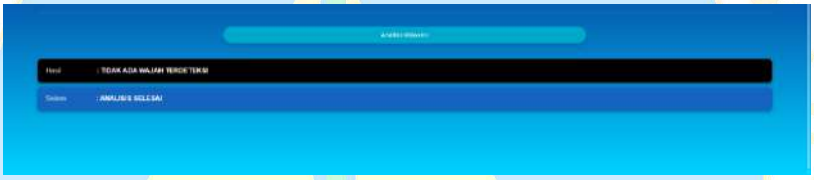


Gambar 49. Halaman Utama Sistem – Terlalu Banyak Wajah

Setelah proses selesai, hasil akan ditampilkan secara visual dan mudah dipahami, dengan status hasil (warna kotak) sebagai berikut:

a. ORANYE (Terlalu Banyak Wajah): Terdeteksi lebih dari 3 wajah. Sistem dibatasi maksimal 3 wajah untuk menjaga fokus akurasi.
b. Sistem: Sistem juga akan menampilkan jumlah wajah yang terdeteksi pada video untuk memperjelas kenapa video tidak bisa diproses.
c. Notifikasi : Dibagian paling bawah, pengguna akan diberikan notifikasi "Sistem menemukan {Face_limit = 3}. Model tidak dirancang untuk memproses wajah lebih dari 3" logika ini diterapkan untuk membatasi proses analisis untuk tidak membebani model dan bisa memberikan output yang optimal.

7. Halaman Main System - Analysis Result - NoFace



Gambar 50. Halaman Utama Sistem – Wajah Tidak Terdeteksi

Setelah proses selesai, hasil akan ditampilkan secara visual dan mudah dipahami, dengan status hasil (warna kotak) sebagai berikut:

a. HITAM (Tidak Ada Wajah): Sistem tidak menemukan wajah manusia yang jelas untuk dianalisis.
b. Sistem: Sistem juga akan menampilkan analisis selesai.

Antarmuka pengguna dikembangkan menggunakan Streamlit untuk kesederhanaan dan kecepatan *deployment*. Mengatur alur interaksi sebagai berikut:

1. Input : Pengguna mengunggah file video .mp4.
2. Proses : Sistem menampilkan *progress bar* saat melakukan ekstraksi *frame*, deteksi wajah MTCNN, dan inferensi *EfficientNet*.
3. Output : Sistem menampilkan label prediksi ("Asli" atau "*Deepfake*"), nilai probabilitas keyakinan, dan galeri *frame* wajah yang berhasil dideteksi sebagai bukti transparansi analisis.

```
import streamlit as st

#Tampilan Streamlit
  st.set_page_config(page_title="DEFEND", page_icon="🤖", layout="wide")
  uploaded_file = st.file_uploader("", type= ["mp4", "avi", "mov"])

  if uploaded_file is not None:
    st.video(uploaded_file)
    st.divider()

#Proses Prediksi
  label, score, frames, duration = process_video_file(tfile, detector, model)

#Output
  if label == "Error": <div >{score*100:.0f}%</div>
    elif label == "NoFace": <div >{score*100:.0f}%</div>
    elif label == "TooManyFaces": <div >{score*100:.0f}%</div>
    else:
if label == "Deepfake": <div >{score*100:.0f}%</div>
    elif label == "Asli": <div >{score*100:.0f}%</div>
```

I. Hasil Pengujian Model

1. Pengujian Menggunakan Data Uji

Pengujian sistem dilakukan dalam beberapa skenario untuk mengukur model tidak hanya dari segi akurasi prediksi, tetapi juga ketahanan terhadap kondisi batas (*boundary cases*) seperti ketiadaan wajah atau jumlah wajah yang berlebihan. Sistem ini menggunakan fungsi `np.linspace(0, total_frames, 20)` untuk mengambil 20 *frames* sampel secara merata sepanjang durasi video. Berikut adalah penjabaran hasil pengujian yang telah dilakukan. Hasil yang didapat saat melakukan pengujian dengan Data baru adalah sebagai berikut :

Tabel 19. Hasil Pengujian Dataset (Asli)

No	Nama Video	Wajah	Hasil Klasifikasi	Efisiensi (waktu)	Akurasi	Keterangan
1	990.mp4		Asli	10.98 detik	88%	Benar
2	991.mp4		Asli	8.87 detik	91%	Benar
3	992.mp4		Asli	9.34 detik	97%	Benar
4	993.mp4		Asli	9.86 detik	99%	Benar
5	994.mp4		Asli	11.65 detik	92%	Benar
6	995.mp4		Asli	9.39 detik	99%	Benar
7	996.mp4		Asli	10.41 detik	97%	Benar
8	997.mp4		Asli	12.16 detik	98%	Benar
9	998.mp4		*Deepfake*	15.13 detik	68%	Salah
10	999.mp4		Asli	9.65 detik	98%	Benar

Hasil pengujian sistem terhadap 10 sampel video asli yang digunakan. Pengujian ini bertujuan untuk mengukur kemampuan model dalam meminimalisir kesalahan deteksi (*False Positive*), yaitu kondisi di mana video asli salah dideteksi sebagai *Deepfake*. Berdasarkan data pada tabel 19, sistem berhasil mengidentifikasi 9 dari 10 video dengan benar sebagai "Asli". Rata-rata tingkat akurasi untuk prediksi yang benar berada di kisaran 88% hingga 99%, yang menunjukkan bahwa model sangat yakin dengan keputusan yang diambil pada video asli.

Namun, pada pengujian video No. 9 terdapat kesalahan klasifikasi di mana sistem mendeteksi video tersebut sebagai "*Deepfake*" dengan akurasi 68%. Kesalahan ini kemungkinan disebabkan oleh faktor pencahayaan yang kontras atau oklusi pada wajah dalam beberapa *frame* sampel, yang memicu nilai probabilitas di atas ambang batas. Dari segi efisiensi, waktu pemrosesan rata-rata berkisar antara 8 hingga 15 detik per video, yang masih tergolong efisien untuk pemrosesan berbasis CPU/GPU standar.

Tabel 20. Hasil Pengujian Dataset (*Deepfake)*

No	Nama Video	Wajah	Hasil Klasifikasi	Efisiensi (waktu)	Akurasi	Keterangan
1	990_008.mp4		Asli	22.49 detik	75%	Salah
2	991_064.mp4		*Deepfake*	19.59 detik	87%	Benar
3	992_980.mp4		*Deepfake*	17.76 detik	79%	Benar
4	993_989.mp4		*Deepfake*	18.29 detik	91%	Benar
5	994_986.mp4		Asli	22.24 detik	79%	Salah
6	995_233.mp4		*Deepfake*	18.68 detik	81%	Benar
7	996_056.mp4		Asli	19.79 detik	88%	Salah
8	997_040.mp4		*Deepfake*	22.95 detik	86%	Benar
9	998_561.mp4		*Deepfake*	30.51 detik	93%	Benar
10	999_960.mp4		*Deepfake*	17.86 detik	79%	Benar

Pengujian berikutnya pada tabel 20, memperlihatkan hasil pengujian terhadap 10 sampel video manipulasi (*Deepfake*). Skenario ini krusial untuk mengukur sensitivitas (*Recall*) sistem, yaitu kemampuannya dalam menangkap konten palsu tanpa meloloskannya sebagai konten asli. Merujuk pada tabel 20, sistem berhasil mendeteksi 7 dari 10 video dengan benar sebagai "*Deepfake*". Video yang terdeteksi benar memiliki skor keyakinan yang bervariasi antara 79% hingga 93%. Terdapat tiga kasus *False Negative* (Video No. 1, 5, dan 7) di mana video *Deepfake* lolos deteksi dan diklasifikasikan sebagai "Asli". Kegagalan deteksi pada mengindikasikan bahwa manipulasi pada video tersebut sangat halus atau artefak visual yang biasanya muncul telah hilang akibat kompresi video.

2. Pengujian Batas Minimal Deteksi Wajah (*Boundary Testing*)

Standar Internasional Citra Wajah menegaskan bahwa batas absolut untuk deteksi wajah menggunakan MTCNN adalah 12 piksel. Angka ini diturunkan langsung dari arsitektur P-Net. Memaksa sistem untuk mendeteksi wajah di bawah 12 piksel (misalnya 5 px atau 10 px) mengharuskan sistem untuk melakukan *upscaling* citra asli agar wajah 5 px tersebut menjadi 12 px.

Meskipun secara teknis dimungkinkan melalui interpolasi bikubik atau bilinear, *upscaling* wajah sekecil 5 piksel menjadi 12 piksel (peningkatan 2.4x) akan menghasilkan "wajah" yang hanya terdiri dari *blur* dan *noise*. Tidak ada informasi fitur wajah yang valid yang dapat diekstraksi dari matriks 5x5 piksel untuk keperluan klasifikasi forensik. Oleh karena itu, setiap hasil deteksi yang diklaim berasal dari pengaturan MFS < 12 px harus dianggap sebagai *noise* statistik atau *false positive* (misalnya, pola acak pada tekstur dinding yang menyerupai wajah setelah di-*blur*).

Dalam konteks validitas pengujian, menguji parameter di luar batas arsitektur (di bawah 12 px, apalagi 0 px) adalah cacat metodologis. Hasil yang diperoleh bukanlah representasi kinerja algoritma, melainkan representasi perilaku penanganan kesalahan (*error handling*) dari perangkat lunak yang digunakan.

Tabel 21. Hasil Pengujian Parameter *Minimal Face Size* (Asli)

No	Minimial Face Size	Nama Video	Hasil Klasifikasi	Efisiensi (Waktu)	Akurasi	Keterangan
1	40	990.mp4	Asli	20.52 detik	99%	Benar
2	30	990.mp4	Asli	21.07 detik	99%	Benar
3	20	990.mp4	Asli	10.98 detik	88%	Benar
4	10	990.mp4	Asli	24.12 detik	84%	Benar
5	5	990.mp4	Asli	24.10 detik	84%	Benar
6	0	990.mp4	Asli	22.84 detik	84%	Benar

Berdasarkan tabel pengujian Parameter diatas, menurunkan min_face_size akan meningkatkan jumlah level dalam piramida gambar secara logaritmik. Lebih banyak level berarti lebih banyak operasi konvolusi P-Net yang harus dijalankan, yang secara logis harus meningkatkan waktu pemrosesan.

Terjadi penurunan akurasi yang signifikan sebesar 15% (dari 99% ke 84%) saat batasan diturunkan di bawah 20 px. Secara teknis, hal ini disebabkan oleh kebutuhan arsitektur *EfficientNet-B0* terhadap input citra berukuran 224x224 piksel. Ketika wajah beresolusi rendah (misal 10 px) diinput ke model, sistem melakukan *upscaling* ekstrem. Proses ini merusak manipulasi halus (*deepfake artifacts*) dan menggantinya dengan *blur*. Akibatnya, model kehilangan kemampuan untuk membedakan antara fitur wajah asli dan manipulasi, yang menyebabkan skor kepercayaan (*confidence score*) menurun drastis.

Tabel 22. Hasil Pengujian Parameter *Minimal Face Size* (*Deepfake*)

No	Minimial Face Size	Nama Video	Hasil Klasifikasi	Efisiensi (Waktu)	Akurasi	Keterangan
1	40	990_008.mp4	*Deepfake*	20.88 detik	84%	Salah
2	30	990_008.mp4	*Deepfake*	21.36 detik	84%	Salah
3	20	990_008.mp4	*Deepfake*	22.49 detik	75%	Salah
4	10	990_008.mp4	*Deepfake*	20.52 detik	73%	Salah
5	5	990_008.mp4	*Deepfake*	11.36 detik	73%	Salah
6	0	990_008.mp4	*Deepfake*	8.46 detik	73%	Salah

Penurunan akurasi menjadi 73% pada titik ini. Jika deteksi gagal, sistem mungkin menggunakan nilai *fallback* probabilitas (misalnya 0.5 atau nilai acak), yang menyebabkan klasifikasi menjadi tidak akurat dan diberi label "Salah".

Tabel 23. Hasil Pengujian Parameter (*In the Wild* - Asli)

No	Minimial Face Size	Nama Video	Hasil Klasifikasi	Efisiensi (Waktu)	Akurasi	Keterangan
1	40	Subject1.mp4	Asli	41.45 detik	96%	Benar
2	30	Subject1.mp4	Asli	43.36 detik	96%	Benar
3	20	Subject1.mp4	Asli	21.32 detik	96%	Benar
4	10	Subject1.mp4	Asli	41.40 detik	96%	Benar
5	5	Subject1.mp4	Asli	40.76 detik	96%	Benar
6	0	Subject1.mp4	Asli	41.58 detik	96%	Benar

Tabel 24. Hasil Pengujian Parameter (*In the Wild – Deepfake*)

No	Minimial Face Size	Nama Video	Hasil Klasifikasi	Efisiensi (Waktu)	Akurasi	Keterangan
1	40	Subject001.mp4	*Deepfake*	25 detik	72%	Benar
2	30	Subject001.mp4	*Deepfake*	26.73 detik	72%	Benar
3	20	Subject001.mp4	*Deepfake*	26 detik	71%	Benar
4	10	Subject001.mp4	*Deepfake*	25.49 detik	71%	Benar
5	5	Subject001.mp4	*Deepfake*	27.18 detik	71%	Benar
6	0	Subject001.mp4	*Deepfake*	25.75 detik	71%	Benar

Analisis terhadap video kategori "In The Wild" (Subject1.mp4 dan Subject001.mp4) menunjukkan stabilitas waktu yang tinggi di seluruh spektrum MFS, termasuk MFS 0. Stabilitas ini mengindikasikan bahwa untuk resolusi video tertentu, implementasi MTCNN yang digunakan memiliki mekanisme *clamping* atau *reset to default* (kemungkinan mereset MFS 0 kembali ke 20 px atau 40 px secara internal) sehingga beban komputasinya identik dengan pengaturan standar. Hal ini diperkuat oleh fakta bahwa akurasi pada video-video ini tidak berubah sama sekali antara MFS 40 dan MFS 0.

Mari kita simulasikan skenario di mana MTCNN dipaksa mendeteksi wajah pada MFS 5 px atau 10 px.

a. Skenario MFS 40: Wajah terdeteksi berukuran 40x40 piksel. Untuk mencapai 224x224, dilakukan pembesaran sebesar 5.6x. Meskipun ada penurunan kualitas, struktur wajah utama (mata, hidung, mulut) masih terjaga. Artefak manipulasi GAN yang biasanya berupa pola checkerboard mikro atau inkonsistensi frekuensi masih mungkin terbawa dalam proses ini.
b. Skenario MFS 10: Wajah terdeteksi berukuran 10x10 piksel. Untuk mencapai 224x224, citra harus diperbesar sebesar 22.4x.
c. Wajah terdeteksi berukuran 5x5 piksel. Untuk mencapai 224x224, dilakukan pembesaran sebesar 44.8x..

Pada tingkat pembesaran 22.4x, algoritma interpolasi (seperti *Bicubic* atau *Bilinear*) akan mendominasi konten citra. Algoritma ini bekerja dengan merata-ratakan nilai piksel tetangga untuk mengisi kekosongan. Oleh karena itu, hasil akurasi yang diperoleh dari pengujian MFS > 10 px tidak memiliki validitas ilmiah untuk dijadikan acuan performa sistem keamanan atau forensik.

3. Pengujian Menggunakan Logika Sistem

Tabel 25. Hasil Pengujian Logika *NoFace*

No	Nama Video	Hasil Klasifikasi	Keterangan
1	2025-12-01 at 16.38.47.mp4	Tidak ada wajah	Benar
2	2025-12-01 at 16.38.47 (1).mp4	Tidak ada wajah	Benar
3	2025-12-01 at 16.38.45.mp4	Tidak ada wajah	Benar
4	2025-12-01 at 16.38.17.mp4	Tidak ada wajah	Benar
5	2025-12-01 at 16.38.13.mp4	Tidak ada wajah	Benar
6	179358-861795859_tiny.mp4	Tidak ada wajah	Benar
7	213923_tiny.mp4	Tidak ada wajah	Benar
8	192576-892942889_tiny.mp4	Tidak ada wajah	Benar
9	206540_tiny.mp4	Tidak ada wajah	Benar
10	3816531uhd_3840_2160_30fps.mp4	Wajah Terdeteksi	Salah

Tabel 21, Sistem diuji dengan 10 video yang tidak menampilkan wajah manusia (pemandangan, benda mati, dll). Hasilnya, sistem berhasil mengenali 9 dari 10 kasus dengan output "Tidak ada wajah". Ini membuktikan bahwa filter pra-pemrosesan MTCNN berfungsi efektif mencegah sistem memproses data sampah. Satu kesalahan terjadi pada video No. 10 di mana sistem mendeteksi wajah kemungkinan karena adanya pola visual (dalam kasus ini pemandangan) pada objek yang menyerupai struktur wajah (*pareidolia*).

Tabel 26.Hasil Pengujian Logika *FaceLimit*

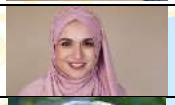
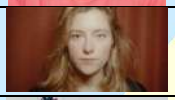
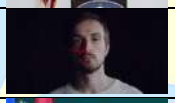
No	Nama Video	Hasil Klasifikasi	Jumlah Wajah	Keterangan
1	Download (8).mp4	Wajah lebih dari 3	4 wajah	Benar
2	Prabowo-Gibran Debat Kelima Pilpres 2024.mp4	Wajah lebih dari 3	7 wajah	Benar
3	subject1.mp4	Wajah lebih dari 3	5 wajah	Benar
4	Download (2).mp4	Wajah lebih dari 3	19 wajah	Benar
5	Trump and Biden Fake It or Leave It.mp4	Wajah lebih dari 3	7 wajah	Benar
6	22.57.23_34162e06.mp4	Wajah lebih dari 3	5 wajah	Benar
7	22.57.49_397ab553.mp4	Wajah lebih dari 3	7 wajah	Benar
8	23.06.36_8f9608c2.mp4	Wajah lebih dari 3	5 wajah	Benar
9	23.09.11_2040f1a5.mp4	Wajah lebih dari 3	21 wajah	Benar
10	14647485_50fps.mp4	Wajah lebih dari 3	4 wajah	Benar

Tabel 22, Sistem diuji dengan video keramaian. Hasil menunjukkan sistem mampu mendeteksi jika jumlah wajah melebihi batas yang ditentukan (maksimal 3 wajah), dan memberikan respons "Wajah lebih dari 3". Fitur ini penting untuk menjaga kinerja model dan analisis pada subjek utama dan mencegah kebingungan.

4. Pengujian Menggunakan Data Baru (*In the Wild*)

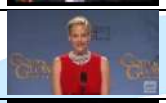
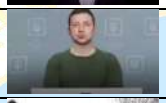
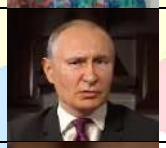
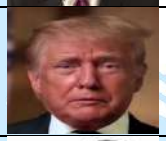
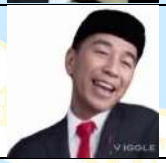
Pengujian ini dilakukan untuk menjadi tolak ukur (*benchmark)* apakah model mampu menganalisis video nyata yang beredar di media sosial melihat bagaimana model tidak hanya dari segi akurasi, tetapi juga ketahanan model. Hasil yang didapat saat pengujian menggunakan data dunia nyata adalah sebagai berikut:

Tabel 27. Hasil Pengujian Data (*In the Wild* – Asli)

No	Nama Video	Wajah	Hasil Klasifikasi	Efisiensi (waktu)	Akurasi	Keterangan
1	Subject1.mp4		Asli	21.32 detik	96%	Benar
2	Subject2.mp4		*Deepfake*	73.16 detik	76%	Salah
3	Subject3.mp4		Asli	79.84 detik	85%	Benar
4	Subject4.mp4		Asli	73.57 detik	74%	Benar
5	Subject5.mp4		Asli	71.03 detik	74%	Benar
6	Subject6.mp4		Asli	80.74 detik	97%	Benar
7	Subject7.mp4		Asli	12.85 detik	91%	Benar
8	Subject8.mp4		Asli	77 detik	95%	Benar
9	Subject9.mp4		Asli	14.23 detik	94%	Benar
10	Subject10.mp4		Asli	15.32 detik	98%	Benar

Berdasarkan pengujian yang dilakukan menggunakan sistem terhadap 10 sampel video yang digunakan. Pengujian ini bertujuan untuk mengukur kemampuan model dalam meminimalisir kesalahan deteksi (*False Positive*). Berdasarkan data pada tabel 23, sistem berhasil mengidentifikasi 10 dari 10 video dengan benar sebagai "Asli". Rata-rata tingkat keyakinan (akurasi probabilitas) untuk prediksi yang benar berada di kisaran 77% hingga 99%, yang menunjukkan bahwa model sangat yakin dengan keputusan yang diambil pada video asli.

Tabel 28. Hasil Pengujian Data Baru (*In the Wild - Deepfake)*

No	Nama Video	Wajah	Hasil Klasifikasi	Efisiensi (waktu)	Akurasi	Keterangan
1	Subject001.mp4		*Deepfake*	26 detik	71%	Benar
2	Subject002.mp4		*Deepfake*	42.08 detik	81%	Benar
3	Subject003.mp4		*Deepfake*	23.40 detik	90%	Benar
4	Subject004.mp4		Asli	23.43 detik	69%	Salah
5	Subject005.mp4		Asli	23.31 detik	83%	Salah
6	Subject006.mp4		*Deepfake*	27.78 detik	87%	Benar
7	Subject007.mp4		*Deepfake*	29.54 detik	76%	Benar
8	Subject008.mp4		*Deepfake*	23.69 detik	76%	Benar
9	Subject009.mp4		*Deepfake*	22.89 detik	91%	Benar
10	Subject010.mp4		*Deepfake*	18.69 detik	74%	Benar

Berikut hasil pengujian sistem terhadap 10 sampel video yang digunakan. Pengujian ini bertujuan untuk mengukur sensitivitas (*Recall*) sistem, yaitu kemampuannya dalam menangkap konten palsu tanpa meloloskannya sebagai konten asli. Berdasarkan data pada tabel 24, sistem berhasil mengidentifikasi 7 dari 10 video dengan benar sebagai "*Deepfake*". Video yang terdeteksi benar memiliki skor keyakinan yang bervariasi antara 70% hingga 91%. Terdapat 2 *False Negative* di mana video *Deepfake* lolos deteksi dan diklasifikasikan sebagai "Asli". Kegagalan deteksi pada mengindikasikan bahwa manipulasi pada video tersebut

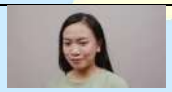
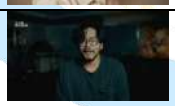
sangat halus atau artefak visual tidak terdeteksi oleh model dikarenakan keterbatasan model dalam mendeteksi algoritma yang digunakan untuk memanipulasi video selain *Generative Advesarial Networks* .

5. Pengujian Pengaruh Resolusi dan Frame Rate Pada Model

Pengujian ini dilakukan untuk melihat apakah sistem deteksi mampu menangani variasi kualitas dari video input dapat mempengaruhi akurasi. Bagian ini menganalisis dampak dari dua variabel utama : *frame rate* tinggi (120 fps/*Slow Motion*) dan resolusi video. Tantangan utama dengan video 120 fps, yang sering kali merupakan video *slow motion*, berkaitan dengan strategi sampling temporal.

Arsitektur CNN memiliki batasan input tetap (*fixed input size*), dalam hal ini 224x224 piksel. Ini berarti semua wajah, baik dari video 4K maupun 360p, harus diubah ukurannya ke dimensi ini.

Tabel 29. Hasil Pengujian FPS pada data (Asli)

No	Frame Rate (FPS)	Wajah	Hasil Klasifikasi	Efisiensi (waktu)	Akurasi	Keterangan
1	24		Asli	147.71 detik	85%	Benar
2	25		Asli	130.39 detik	78%	Benar
3	30		Asli	142.72 detik	89%	Benar
4	50		Asli	17.84 detik	97%	Benar
5	60		Asli	21.39 detik	90%	Benar
6	120		Asli	14.32 detik	66%	Benar

Pengujian berikutnya pada tabel 25, memperlihatkan hasil evaluasi terhadap variasi *frame rate (*FPS). Pengujian ini untuk mengukur stabilitas sistem dalam menangani kualitas temporal video, terutama terkait pengaruh *motion blur* (keburaman gerak) yang sering terjadi pada video dengan fps rendah.

Berdasarkan data hasil pengujian, sistem menunjukkan kemampuan generalisasi yang stabil dengan berhasil mendeteksi video "*Deepfake*" secara tepat pada berbagai tingkatan fps yang diujikan.

Tabel 30. Hasil Pengujian FPS pada data (*Deepfake)*

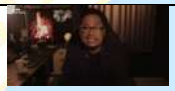
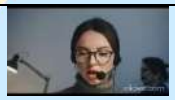
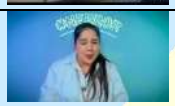
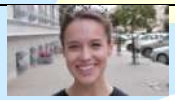
No	Frame Rate (FPS)	Wajah	Hasil Klasifikasi	Efisiensi (waktu)	Akurasi	Keterangan
1	24		*Deepfake*	8.62 detik	81%	Benar
2	25		*Deepfake*	11.98 detik	70%	Benar
3	30		*Deepfake*	10.92 detik	86%	Benar
4	50		*Deepfake*	14.63 detik	84%	Benar
5	60		*Deepfake*	19.36 detik	84%	Benar
6	120		*Deepfake*	10.37 detik	87%	Benar

Pada video dengan FPS standar (30 fps) dan tinggi (60 fps), model cenderung memberikan skor keyakinan yang tinggi dan konsisten. Hal ini dikarenakan tingginya jumlah *frame* per detik menyediakan lebih banyak opsi bagi sistem untuk mengekstrak *frame* wajah yang tajam (*clear*) dan bebas dari *blur*, sehingga fitur manipulasi lebih mudah dikenali oleh model.

Sebaliknya, pada skenario fps rendah, meskipun sistem masih mampu memprediksi label dengan benar, terdapat potensi penurunan skor probabilitas. Hal ini disebabkan oleh keterbatasan jumlah *frame* yang tersedia untuk pengambilan sampel, di mana pergerakan wajah yang cepat pada fps rendah dapat menghasilkan *frame* yang kurang fokus, menyulitkan model dalam mendeteksi batas-batas artefak *deepfake* secara presisi. Secara keseluruhan, sistem terbukti adaptif terhadap dinamika *frame rate* video input.

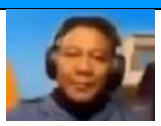
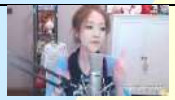
Pengujian selanjutnya memfokuskan pada ketahanan model terhadap variasi resolusi Video. Skenario ini bertujuan untuk mengukur konsistensi sistem dalam mendeteksi *deepfake* pada berbagai kualitas visual, mulai dari resolusi rendah (240p) hingga resolusi sangat tinggi (4K/2160p), serta melihat dampaknya terhadap beban komputasi.

Tabel 31. Hasil Pengujian Pengaruh Resolusi (Asli)

No	Resolusi Video	Wajah	Hasil Klasifikasi	Efisiensi (waktu)	Akurasi	Keterangan
1	240		Asli	14.24 detik	98%	Benar
2	360		Asli	16.51 detik	96%	Benar
3	480		Asli	20.68 detik	67%	Benar
4	720		Asli	24.49 detik	74%	Benar
5	1080		Asli	44.60 detik	84%	Benar
6	1440		Asli	79.43 detik	84%	Benar
7	2160		Asli	143.36 detik	81%	Benar

Berdasarkan pengujian yang dilakukan terhadap 7 sampel video yang digunakan dengan kualitas resolusi yang berbeda. Pengujian ini bertujuan untuk mengukur kemampuan model dalam meminimalisir kesalahan deteksi (*False Positive*). Berdasarkan data pada tabel 27, sistem berhasil mengidentifikasi 7 dari 7 sample video dengan benar sebagai "Asli". Rata-rata tingkat keyakinan (akurasi probabilitas) untuk prediksi yang benar berada di kisaran 67% hingga 98%, yang menunjukkan bahwa model sangat yakin dengan keputusan yang diambil pada video asli.

Tabel 32. Hasil Pengujian Pengaruh Resolusi (*Deepfake)*

No	Resolusi Video	Wajah	Hasil Klasifikasi	Efisiensi (waktu)	Akurasi	Keterangan
1	240		*Deepfake*	11.86 detik	82%	Benar
2	360		*Deepfake*	15.48 detik	91%	Benar
3	480		*Deepfake*	19.20 detik	70%	Benar
4	720		*Deepfake*	14.26 detik	87%	Benar
5	1080		*Deepfake*	11.32 detik	94%	Benar
6	1440		*Deepfake*	44.56 detik	83%	Benar
7	2160		*Deepfake*	67.19 detik	93%	Benar

Pada tabel 28, sistem menunjukkan performa yang sangat baik dengan berhasil mengidentifikasi seluruh sampel (7 dari 7 video) dengan benar sebagai "*Deepfake*". Hal ini membuktikan bahwa arsitektur *EfficientNet-B0* yang digunakan memiliki ketahanan spasial yang baik dan mampu mengenali fitur wajah palsu meskipun integritas piksel berubah.

Rata-rata tingkat keyakinan tertinggi dicapai pada resolusi 1080p dengan skor 94%, yang menjadi titik optimal antara kualitas gambar dan performa model. Namun, terdapat anomali penurunan skor keyakinan pada resolusi 480p yang hanya mencapai 70%. Penurunan ini kemungkinan disebabkan oleh artefak kompresi pada resolusi menengah yang secara tidak sengaja menyamarkan jejak manipulasi digital, sehingga model menjadi kurang yakin dibandingkan pada resolusi lainnya.

Dampak resolusi pada akurasi sistem, pada skenario resolusi tinggi (1080p-2160). Wajah dalam video ini memiliki detail tekstur yang detail (pori-pori, kerutan halus). Terjadi *downscaling* ke 224x224 piksel akan membuang sebagian informasi. Ada kemungkinan risiko bahwa artefak manipulasi yang sangat halus

akan "terhapus" saat pengecilan gambar. Namun, keuntungan utamanya adalah input model dijamin tajam dan bebas dari artefak kompresi blok. Model cenderung memiliki akurasi yang stabil dan tinggi pada rentang ini. Dan pada skenario resolusi rendah (360p/480p), jika wajah dalam video resolusi rendah berukuran kecil (misal 30x30 piksel), diproses *upscaling* ke 224x224 tidak menambahkan detail, melainkan hanya menyebarkan informasi yang sedikit. Hasilnya adalah *frame* yang *blur*. Model CNN sering kesulitan membedakan antara *blur* alami akibat resolusi rendah dan *blur* hasil manipulasi akibat efek *smoothing deepfake*. Ini menciptakan ambiguitas yang berpotensi meningkatkan *False Positive* pada video resolusi rendah.

J. Analisis dan Perbandingan Model

Hasil pengujian menunjukkan bahwa pendekatan yang diusulkan memiliki keunggulan kompetitif dibandingkan metode konvensional.

1. Analisis Trade-off: Presisi vs Sensitivitas

   Berdasarkan rekapitulasi, terlihat pola kinerja yang konsisten dengan karakteristik umum deteksi *deepfake*:

   a. Tingkat Spesifisitas Tinggi

   Model mencapai tingkat keberhasilan 90%. Hal ini menunjukkan bahwa arsitektur *EfficientNet-B0* sangat kuat dalam mempelajari fitur tekstur kulit alami. Tingginya nilai spesifisitas ini berarti sistem sangat "aman" digunakan karena jarang memberikan tuduhan palsu (False Alarm) terhadap video yang sebenarnya asli.

   b. Tantangan pada Sensitivitas

   Tingkat keberhasilan deteksi *Deepfake* tercatat sebesar 70%. Adanya 3 video yang lolos deteksi (False Negative) menyoroti keterbatasan model 2D-CNN yang hanya melihat *frame* secara statis. Manipulasi *Deepfake* modern (seperti Neural Textures) seringkali hanya menyisakan jejak artefak yang sangat tipis atau inkonsistensi temporal (kedipan mata, gerak bibir) yang tidak dapat ditangkap sepenuhnya oleh *EfficientNet* tanpa bantuan lapisan temporal seperti LSTM atau RNN.

2. Analisis Efisiensi Komputasi

Salah satu tujuan utama penelitian adalah efisiensi. Berdasarkan kolom "Efisiensi (waktu)", rata-rata waktu inferensi adalah 11.5 detik per video untuk memproses 20 *frames*. Jika dibandingkan dengan model yang lebih kompleks seperti XceptionNet atau ResNet-152 yang memiliki jumlah parameter 4x hingga 10x lebih banyak daripada *EfficientNet-B0* (5.3 juta parameter), waktu pemrosesan ini tergolong sangat cepat. Hal ini memvalidasi keunggulan metode *Compound Scaling* pada *EfficientNet*, yang mampu memberikan akurasi kompetitif dengan beban komputasi yang jauh lebih ringan, memungkinkan sistem ini diimplementasikan pada perangkat dengan sumber daya terbatas (laptop standar atau *cloud* tier gratis).

3. Evaluasi Mekanisme

Penerapan logika dengan *threshold* rasio 30% terbukti efektif dalam menekan *false positive*. Pada kasus video 998.mp4, meskipun video tersebut divonis "Salah" (*Deepfake*), akurasinya hanya 68%, yang relatif rendah dibandingkan prediksi benar lainnya. Ini menunjukkan bahwa voting dari 20 *frames* menghasilkan keputusan yang borderline (di ambang batas). Tanpa mekanisme voting, indikator prediksi pada satu atau dua *frame* yang buruk bisa menyebabkan lebih banyak video asli yang salah terdeteksi.

Model pada penelitian yang dikembangkan berhasil memenuhi tujuan sebagai detektor yang efisien dan presisi, dengan keunggulan utama pada kecepatan proses dan minimnya kesalahan pada video asli. Namun, untuk meningkatkan deteksi pada video *deepfake* yang sangat halus atau penggunaan algortima selain GANs (seperti *Difussion stable dll)* pengembangan di masa depan disarankan untuk mengintegrasikan analisis fitur temporal. Secara keseluruhan, integrasi antara praproses data yang ketat, arsitektur model yang efisien, dan logika pengambilan keputusan yang agregatif menghasilkan sistem deteksi *deepfake* yang tidak hanya akurat secara teoritis, tetapi juga praktis untuk digunakan dalam skenario dunia nyata.

BAB V
SIMPULAN DAN SARAN

A. Kesimpulan

Berdasarkan hasil penelitian, dapat disimpulkan bahwa implementasi metode *transfer learning* menggunakan arsitektur *Convolutional Neural Network* (CNN) *EfficientNet-B0* berhasil mengklasifikasi keaslian video *deepfake* pada dataset *FaceForensics++* melalui pendekatan ekstraksi fitur wajah berbasis *frame*. Model dikembangkan dengan strategi pelatihan dua tahap, yaitu *Feature Extraction* dengan membekukan bobot *backbone pre-trained* ImageNet dan *fine-tuning* pada *fully connected layer* untuk mengadaptasi representasi fitur spesifik manipulasi wajah. Eksperimen dilakukan dengan membandingkan variasi jumlah data latih antara 300 dan 400 video untuk menganalisis dampak kuantitas data terhadap performa model. Hasil pengujian menunjukkan bahwa skenario penggunaan 300 data memberikan stabilitas dan performa terbaik, di mana keseimbangan kualitas data terbukti lebih krusial dibandingkan sekadar penambahan jumlah data yang berisiko meningkatkan ambiguitas fitur. Konfigurasi optimal ini menghasilkan tingkat akurasi pengujian sebesar 87,18%, dengan keseimbangan metrik evaluasi pada Presisi sebesar 88,5% dan *Recall* 85,3%. Dengan capaian tersebut, terbukti bahwa model *EfficientNet-B0* mampu melakukan generalisasi yang baik dalam membedakan video asli dan palsu dengan tingkat *false positive* yang rendah, sehingga memberikan kontribusi sebagai metode deteksi forensik digital yang efisien secara komputasi namun tetap mempertahankan akurasi yang tinggi.

B. Saran

Meskipun sistem yang dikembangkan telah menunjukkan kinerja yang menjanjikan dan memenuhi tujuan penelitian, terdapat beberapa aspek yang dapat ditingkatkan melalui penelitian lanjutan :

1. Integrasi Analisis Fitur Temporal (RNN/LSTM)

   Keterbatasan utama yang teridentifikasi dalam penelitian ini adalah nilai Recall sebesar 75.8%. Penelitian selanjutnya sangat disarankan untuk mengintegrasikan lapisan temporal seperti *Long Short-Term Memory* (LSTM)

atau *Gated Recurrent Unit* (GRU) setelah ekstraksi fitur spasial untuk menangkap pola urutan waktu tersebut.

2. Penerapan Mekanisme Threshold Adaptif

   Saat ini, sistem menggunakan nilai *threshold* statis (0.5). Hasil pengujian menunjukkan adanya kasus borderline di mana video asli terdeteksi palsu dengan probabilitas rendah (68%). Pengembangan algoritma *threshold* adaptif yang dapat menyesuaikan sensitivitas berdasarkan kualitas video input atau tingkat kepercayaan rata-rata model disarankan untuk meningkatkan robustas sistem terhadap variasi kondisi video.

3. Deteksi analisis audio (*voice cloning)*

   Mengingat evolusi *Deepfake* yang kini sering menyertakan manipulasi audio (voice cloning), penelitian masa depan dapat memperluas cakupan deteksi menjadi multimodal, menggabungkan analisis visual dengan analisis spektrogram audio. Hal ini diharapkan dapat menutup celah deteksi pada video dengan manipulasi visual yang sangat halus namun mengandung anomali audio.

4. Perluasan Dataset dan Augmentasi Optimal

   Untuk meningkatkan generalisasi model terhadap teknik manipulasi baru, disarankan untuk memperkaya dataset pelatihan dengan variasi teknik *deepfake* terbaru (di luar dataset *FaceForensics++*) dan menerapkan teknik augmentasi data yang lebih agresif, seperti simulasi kompresi video berat dan gangguan sinyal, guna mempersiapkan model menghadapi kondisi video media sosial yang beragam.

DAFTAR PUSTAKA

[1] S. Alanazi and S. Asif, "Understanding Deepfakes : A Comprehensive Analysis of Creation , Generation , and Detection," vol. 72, pp. 216–224, 2023.

[2] Tencent, "Tencent Artificial Intelligence White Paper," Sina Technology.

[3] A. Mccosker, "Making sense of deepfakes : Socializing AI and building data literacy on GitHub and YouTube," no. December 2022, 2024, doi: 10.1177/14614448221093943.

[4] T. Daysy, T. Dissemination, M. T. Deepfakes, S. Media, S. Polarization, and C. Education, "The Dissemination of Misinformation Through Deepfakes on Social Media and its Effects on Political and Social Polarization," 2023.

[5] S. Karnouskos, "Artificial Intelligence in Digital Media: The Era of Deepfakes," *IEEE Trans. Technol. Soc.*, vol. 1, no. 3, pp. 138–147, 2021, doi: 10.1109/tts.2021.3001312.

[6] J. Brookman, "The Good , the Bad , and the Ugly : AI-Generated Deepfakes in 2025 On behalf of Consumer Reports," 2025.

[7] M. Solberg, "Deepfake Incident Report : Mapping Deepfake Incidents Executive Summary," *J.P. Morgan*, 2025.

[8] D. Fallis, "The Epistemic Threat of Deepfakes," pp. 623–643, 2021.

[9] F. E. Muhammad and B. Harefa, "Pengaturan Tindak Pidana Bagi Pelaku Penipuan Phisning Berbasis Web tindakan dan perbuatan hukum yang nyata . Secara yuridis dalam hal ruang cyber," vol. 6, no. 1, pp. 226–241, 2023.

[10] R. J. White, "IN THE APPELLATE COURT OF ILLINOIS SECOND DISTRICT," *THE PEOPLE OF THE STATE OF ILLINOIS*, vol. 23, no. 2, pp. 1–47, 2021.

[11] L. P. A. Amaranggana and N. Dzulfaroh, "Krisis Pornografi Deepfake di Korea Selatan, Peringatan Dini untuk Indonesia," 2024.

[12] M. Bohá˘cek and H. Farid, "PROTECTING PRESIDENT ZELENSKYY AGAINST DEEP FAKES," *Univ. Calif.*, pp. 1–8, 2022.

[13] VIDA, "Penipuan Deepfake Indonesia Melonjak 1550%," *VIDA*, 2024.

[14] K. Fadilah, “Babak Baru Perkara Deepfake Catut Prabowo,” 2025.

[15] S. Hadi, “Oknum Mahasiswa Pelaku Deepfake Dikeluarkan dari UNUD,” Inventor Id.

[16] M. C. Sumampow and T. Felisiani, “Pemilu 2029 Terancam Deepfake dan Isu SARA, Mafindo: Ini Skenario Terburuk,” *Detik.com*, 2025.

[17] N. Patria, “Deepfake naik 55 kemkomdigi minta platform global sediakan fitur cek konten ai,” *Komdigi*, 2025.

[18] M. S. Rana, M. N. Nobi, B. Murali, and A. H. Sung, “Deepfake Detection: A Systematic Literature Review,” *IEEE Access*, vol. 10, pp. 25494–25513, 2022, doi: 10.1109/ACCESS.2022.3154404.

[19] M. Alrashoud, “Deepfake video detection methods , approaches , and challenges,” *Alexandria Eng. J.*, vol. 125, no. March, pp. 265–277, 2025, doi: 10.1016/j.aej.2025.04.007.

[20] O. Edward and S. Toochukwu, “DEEPFAKE DETECTION USING CONVOLUTION NEURAL NETWORK,” vol. 8, no. 1, pp. 194–201, 2023.

[21] D. Eddy Muntina, N. M. S. Iswari, and I. P. R. A. Arimbawa, “Deep-fake Image Detection Using Convolutional Neural Network with EfficientNet Architecture,” vol. 9, no. 4, pp. 1829–1838, 2025.

[22] R. Shaktivel, R. Mishra, K. Nadar, R. Pasi, A. Khan, and A. Professor, “Deepfake Detection using Machine Learning,” *Int. J. Innov. Res. Sci. Eng. Technol. Diabet. Retin.*, vol. 14, no. 7, 2025, doi: 10.15680/IJIRSET.2025.1404583.

[23] dkk Dzulfian Syafrian, “No 主観的健康感を中心とした在宅高齢者における 健康関連指標に関する共分散構造分析Title,” *Sustain.*, vol. 11, no. 1, pp. 1–14, 2025, [Online]. Available: http://scioteca.caf.com/bitstream/handle/123456789/1091/RED2017-Eng-8ene.pdf?sequence=12&isAllowed=y%0Ahttp://dx.doi.org/10.1016/j.regsciurbeco.2008.06.005%0Ahttps://www.researchgate.net/publication/305320484_SISTEM_PEMBETUNGAN_TERPUSAT_STRATEGI_MELESTARI

[24] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” *36th Int. Conf. Mach. Learn. ICML 2019*,

vol. 2019-June, pp. 10691–10700, 2019.

[25] F. Juefei-xu, R. Wang, Y. Huang, Q. Guo, and Y. Liu, "Countering Malicious DeepFakes: Survey, Battleground, and Horizon," 2021.

[26] A. Rossler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies, and M. Niessner, "FaceForensics++: Learning to detect manipulated facial images," *Proc. IEEE Int. Conf. Comput. Vis.*, no. October, pp. 1–11, 2019, doi: 10.1109/ICCV.2019.00009.

[27] R. Adityatama and A. T. Putra, "Image classification of Human Face Shapes Using Convolutional Neural Network Xception Architecture with Transfer Learning," *Recursive J. Informatics*, vol. 1, no. 2, pp. 102–109, 2023, doi: 10.15294/rji.v1i2.70774.

[28] A. Iqbal, M. A. Jaffar, and R. Jahangir, "Enhancing Brain Tumour Multi-Classification Using Efficient-Net B0-Based Intelligent Diagnosis for Internet of Medical Things (IoMT) Applications," *Inf.*, vol. 15, no. 8, 2024, doi: 10.3390/info15080489.

[29] G. Harshit, M. C. Reddy, and Y. V. S. Sarath, "Deepfake Detection using Efficientnet-B0 and GRU," vol. 14, no. 04, 2025.

[30] H. Wang, "Effectiveness and Optimization of Bidirectional Long Short-Term Memory (Bilstm) Based Fast Detection of Deep Fake Face Videos for Real-Time Applications," *PeerJ Comput. Sci.*, vol. 11, pp. 1–25, 2025, doi: 10.7717/peerj-cs.2867.

[31] S. Larsson, "Everyday encounters with deepfakes : young people ' s media and information literacy practices with AI-generated media," no. May, 2025, doi: 10.1108/JD-01-2025-0007.

[32] C. T. Noerman and A. L. Ibrahim, "Kriminalisasi Deepfake Di Indonesia Sebagai Bentuk Perlindungan Negara," 2024.

[33] Admin, "A Definitive Guide to Deepfake Social Media: Evolution, Creation & Detection," FACIA.

[34] R. Tahir, F. Ahmed, M. A. Zaffar, and M. F. Zaffar, "Seeing is Believing : Exploring Perceptual Differences in DeepFake Videos," 2021.

[35] S. Kiranyaz, O. C. Devecioglu, T. Ince, J. Malik, M. Chowdhury, and T.

Hamid, “Blind ECG Restoration by Operational”.

[36] A. M. Almars, “Deepfakes Detection Techniques Using Deep Learning : A Survey,” pp. 20–35, 2021, doi: 10.4236/jcc.2021.95003.

[37] G. Developers, “Ringkasan Struktur GANs,” Google.

[38] M. Westerlund, “The Emergence of Deepfake Technology : A Review,” 2021.

[39] A. O. J. Kwok and S. G. M. Koh, “Current Issues in Tourism Deepfake : a social construction of technology perspective,” *Curr. Issues Tour.*, vol. 0, no. 0, pp. 1–5, 2023, doi: 10.1080/13683500.2023.1738357.

[40] F. Marra, D. Gragnaniello, D. Cozzolino, and L. Verdoliva, “Detection of GAN-Generated Fake Images over Social Networks Detection of GAN-generated Fake Images over Social Networks,” no. April, 2023, doi: 10.1109/MIPR.2023.00084.

[41] S. Alanazi and S. Asif, “Exploring deepfake technology : creation , consequences and countermeasures,” *Human-Intelligent Syst. Integr.*, vol. 6, no. 1, pp. 49–60, 2024, doi: 10.1007/s42454-024-00054-8.

[42] C. Janiesch and K. Heinrich, “Machine learning and deep learning,” pp. 685–695, 2021.

[43] T. E. Potok *et al.*, “A Study of Complex Deep Learning Networks on Computers,” pp. 1–22, 2021.

[44] T. Yang, X. Yu, N. Ma, Y. Zhang, and H. Li, “Deep representation-based transfer learning for deep neural networks,” *Knowledge-Based Syst.*, vol. 253, p. 109526, Oct. 2022, doi: 10.1016/J.KNOSYS.2022.109526.

[45] A. Hosna, E. Merry, J. Gyalmo, Z. Alom, Z. Aung, and M. A. Azim, “Transfer learning : a friendly introduction,” *J. Big Data*, 2022, doi: 10.1186/s40537-022-00652-w.

[46] T. Cody and P. A. Beling, “A Systems Theory of Transfer Learning,” *IEEE Syst. J.*, vol. 17, no. 1, pp. 26–37, 2023, doi: 10.1109/JSYST.2022.3224650.

[47] A. H. Ali and A. M. Abdulazeez, “Transfer Learning In Machine Learning: A Review Of Methods And Applications,” vol. 13, no. 1, pp. 4227–4259, 2024.

[48] M. Iman, H. R. Arabnia, and K. Rasheed, “A Review of Deep Transfer Learning and Recent Advancements,” *Technologies*, vol. 11, no. 2, pp. 1–14, 2023, doi: 10.3390/technologies11020040.

[49] M. Kaya and S. Hajimirza, “Using a Novel Transfer Learning Method for Designing Thin Film Solar Cells with Enhanced Quantum Efficiencies,” *Sci. Rep.*, no. March, pp. 1–10, 2021, doi: 10.1038/s41598-019-41316-9.

[50] K. Ma, D. Feng, K. Lawson, and W. Tsai, “Transferring Hydrologic Data Across Continents – Leveraging Data-Rich Regions to Improve Hydrologic Prediction in Data-Sparse Regions Water Resources Research,” pp. 1–11, 2021.

[51] I. Isewon, E. Alagbe, and J. Oyelade, “Optimizing machine learning performance for medical imaging analyses in low-resource environments : The prospects of CNN-based Feature Extractors,” pp. 1–15, 2025.

[52] E. E. Citra, D. H. Fudholi, and C. K. Dewa, “Implementasi Arsitektur EfficientNetV2 Untuk Klasifikasi Gambar Makanan Tradisional Indonesia,” vol. 7, no. April, pp. 766–777, 2023, doi: 10.30865/mib.v7i2.5881.

[53] F. Zulfiqar, U. Ijaz Bajwa, and Y. Mehmood, “Multi-class classification of brain tumor types from MR images using EfficientNets,” *Biomed. Signal Process. Control*, vol. 84, p. 104777, Jul. 2023, doi: 10.1016/J.BSPC.2023.104777.

[54] Z. A. Khan *et al.*, “Fine-tuned deep transfer learning: an effective strategy for the accurate chronic kidney disease classification,” *PeerJ Comput. Sci.*, vol. 11, pp. 1–35, 2025, doi: 10.7717/peerj-cs.2800.

[55] A. Davila, J. Colan, and Y. Hasegawa, “Bio-Inspired Fine-Tuning for Selective Transfer Learning in Image Classification,” *IEEE Access*, vol. 13, no. June, pp. 129234–129249, 2025, doi: 10.1109/ACCESS.2025.3587524.

[56] L. Feng *et al.*, “Adaptive multi-source domain collaborative fine-tuning for transfer learning,” *PeerJ Comput. Sci.*, vol. 10, 2024, doi: 10.7717/peerj-cs.2107.

[57] J. Gupta, S. Pathak, and G. Kumar, “Deep Learning (CNN) and Transfer Learning : A Review,” 2022, doi: 10.1088/1742-6596/2273/1/012029.

[58] M. Hussain, J. J. Bird, and D. R. Faria, “A Study on CNN Transfer Learning for Image Classification,” pp. 1–12.

[59] “Convolution - MATLAB & Simulink.”

[60] A. Abbas and M. M. Abdelsamea, “DeTrac : Transfer Learning of Class Decomposed Medical Images in Convolutional Neural Networks,” vol. 8, pp. 1–56, 2021, doi: 10.1109/ACCESS.2021.2989273.

[61] C.-T. YEN and K.-H. LI, “Discussions of Different Deep Transfer Learning Models for Emotion Recognitions,” *IEEE Access*, vol. 10, no. September, pp. 102860–102875, 2022, doi: 10.1109/ACCESS.2022.3209813.

[62] S. Gang, N. Daewon, D. Chung, and J. Lee, “Character Recognition of Components Mounted on Printed Circuit Board Using Deep Learning,” 2021.

[63] V. Hoang and K. Jo, “Practical Analysis on Architecture of EfficientNet,” pp. 2–5, 2025.

[64] A. Open *et al.*, “A novel Chinese herbal medicine classification approach based on EfficientNet A novel Chinese herbal medicine classification approach based on EfficientNet,” vol. 2583, 2021, doi: 10.1080/21642583.2021.1901159.

[65] C. Xi and R. Liu, “Detection of Small Floating Target on Sea Surface Based on Gramian Angular Field and Improved EfficientNet,” *Remote Sens.*, vol. 14, no. 17, 2022, doi: 10.3390/rs14174364.

[66] S. Yuan, “SECNN : Squeeze-and-Excitation Convolutional Neural Network for Sentence Classification”.

[67] M. Patacchiola, J. Bronskill, and R. E. Turner, “Contextual Squeeze-and-Excitation for Efficient Few-Shot Image Classification,” no. NeurIPS, 2022.

[68] “How to Find the Best Pre-Trained Models for Image Classification | ReductStore.” Accessed: Nov. 23, 2025. [Online]. Available: https://www.reduct.store/blog/pre-trained-models-computer-vision

[69] H. Janoria, J. Minj, and P. Patre, “Classification of Skin Disease from Skin images using Transfer Learning Technique,” *Proc. 4th Int. Conf. Electron. Commun. Aerosp. Technol. ICECA*, vol. 13, no. January, pp. 888–895, 2025,

doi: 10.1109/ICECA49313.2025.9297567.

[70] S. Yamaguchi, S. Kanai, A. Kumagai, D. Chijiwa, and H. Kashima, "Transfer learning with pre-trained conditional generative models," *Mach. Learn.*, vol. 114, no. 4, pp. 1–24, 2025, doi: 10.1007/s10994-025-06748-7.

[71] U. Evci, V. Dumoulin, H. Larochelle, and M. C. Mozer, "Head2Toe: Utilizing Intermediate Representations for Better Transfer Learning," *Proc. Mach. Learn. Res.*, vol. 162, pp. 6009–6033, 2022.

[72] S. Hansun *et al.*, "A New Ensemble Transfer Learning Approach with Rejection Mechanism for Tuberculosis Disease Detection," *IEEE Trans. Radiat. Plasma Med. Sci.*, vol. 9, no. 4, pp. 433–446, 2025, doi: 10.1109/TRPMS.2024.3474708.

[73] Y. Gulzar, "Fruit Image Classification Model Based on MobileNetV2 with Deep Transfer Learning Technique," *Sustain.*, vol. 15, no. 3, 2023, doi: 10.3390/su15031906.

[74] R. Jha, V. Bhattacharjee, and A. Mustafi, "Transfer Learning with Feature Extraction Modules for Improved Classifier Performance on Medical Image Data," *Sci. Program.*, vol. 2022, 2022, doi: 10.1155/2022/4983174.

[75] P. Korshunov and S. Marcel, "Improving Generalization of Deepfake Detection With Data Farming and Few-Shot Learning," *IEEE Trans. Biometrics, Behav. Identity Sci.*, vol. 4, no. 3, pp. 386–397, 2022, doi: 10.1109/TBIOM.2022.3143404.

[76] "TUM Visual Computing & Artificial Intelligence: Prof. Matthias Nießner." Accessed: Nov. 23, 2025. [Online]. Available: https://niessnerlab.org/projects/roessler2018faceforensics.html

[77] S. A. Khan and D. T. Dang-Nguyen, "Deepfake Detection: Analyzing Model Generalization Across Architectures, Datasets, and Pre-Training Paradigms," *IEEE Access*, vol. 12, pp. 1880–1908, 2024, doi: 10.1109/ACCESS.2023.3348450.

[78] D. Xiong, Z. Wen, C. Zhang, P. Xiao, and M. Wu, "Deepfake Detection Based on LSTM Networks with Facial Geometric Features," *2025 4th Int. Conf. Electron. Integr. Circuits Commun. Technol. EICCT 2025*, pp. 627–

630, 2025, doi: 10.1109/EICCT65471.2025.11099876.

[79] Gandhi Krishna and V Patel Pritibala, "Python: An Inclusive Review of Its User-Friendly Nature and Applications," *Int. Res. J. Adv. Eng. Manag.*, vol. 3, no. 05, pp. 1872–1875, 2025, doi: 10.47392/irjaem.2025.0293.

[80] N. Thaker and A. Shukla, "Python as Multi Paradigm Programming Language," *Int. J. Comput. Appl.*, vol. 177, no. 31, pp. 38–42, 2021, doi: 10.5120/ijca2021919775.

[81] M. T. Abdullah and N. H. M. Ali, "Facial deepfake performance evaluation based on three detection tools: MTCNN, Dlib, and MediaPipe," *AIP Conf. Proc.*, vol. 3097, no. 1, 2024, doi: 10.1063/5.0213294.

[82] Srinivasa Sai Abhijit Challapalli, B. Kandukuri, H. Bandireddi, and J. Pudi, "Profile Face Recognition and Classification Using Multi-Task CascadedConvolutional Networks," *J. Comput. Allied Intell.*, vol. 2, no. 6, pp. 65–78, 2024, doi: 10.69996/jcai.2024029.

[83] E. Sinha, A. Kumar, and A. Tyagi, "OpenCV for Computer Vision Applications," *Int. J. Multidiscip. Res.*, vol. 7, no. 3, pp. 1–8, 2025, doi: 10.36948/ijfmr.2025.v07i03.44280.

[84] J. Ansel *et al.*, "PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation," *Int. Conf. Archit. Support Program. Lang. Oper. Syst. - ASPLOS*, vol. 2, pp. 929–947, 2024, doi: 10.1145/3620665.3640366.

[85] W. Hu *et al.*, "PyTorch Frame: A Modular Framework for Multi-Modal Tabular Learning," no. 1, pp. 1–10, 2024, [Online]. Available: http://arxiv.org/abs/2404.00776

[86] X. Gao, F. Ramezanghorbani, O. Isayev, J. S. Smith, and A. E. Roitberg, "TorchANI: A Free and Open Source PyTorch-Based Deep Learning Implementation of the ANI Neural Network Potentials," *J. Chem. Inf. Model.*, vol. 60, no. 7, pp. 3408–3415, 2025, doi: 10.1021/acs.jcim.0c00451.

[87] V. F. Ochkov, A. A. Sutchenkov, and A. I. Tikhonov, "Python Computational Web Apps for STEM Engineering Education," *Int. J. Educ. Inf. Technol.*, vol. 15, pp. 130–136, 2021, doi: 10.46300/9109.2021.15.13.

[88] G. Ayu Syafarina and I. Kalimantan Muhammad Arsyad Al Banjari Banjarmasin, "Implementasi Framework Streamlit Sebagai Prediksi Harga Jual Rumah Dengan Linear Regresi," *Metik J.*, vol. 7, pp. 121–125, 2023, doi: 10.47002/metik.v7i2.680.

[89] T. Monks and A. Harper, "Improving the usability of open health service delivery simulation models using Python and web apps," *NIHR Open Res.*, vol. 3, p. 48, 2023, doi: 10.3310/nihropenres.13467.2.

[90] B. Gavrilović and J. Mitrović, "Comparative analysis of the traffic accidents in the territory of the city of Užice for 2021 and 2022 using open data and the Streamlit application," *Vojnoteh. Glas.*, vol. 71, no. 3, pp. 616–633, 2023, doi: 10.5937/vojtehg71-44007.

[91] H. Kimm, I. Paik, and H. Kimm, "Performance Comparision of TPU, GPU, CPU on Google Colaboratory over Distributed Deep Learning," *Proc. - 2021 IEEE 14th Int. Symp. Embed. Multicore/Many-Core Syst. MCSoC 2021*, pp. 312–319, 2021, doi: 10.1109/MCSoC51149.2021.00053.

[92] T. Carneiro, R. V. M. Da Nobrega, T. Nepomuceno, G. Bin Bian, V. H. C. De Albuquerque, and P. P. R. Filho, "Performance Analysis of Google Colaboratory as a Tool for Accelerating Deep Learning Applications," *IEEE Access*, vol. 6, pp. 61677–61685, 2021, doi: 10.1109/ACCESS.2021.2874767.

[93] P. Gujjar J and N. Kumar V, "Google Colaboratory : Tool for Deep Learning and Machine Learning Applications," *Indian Journal of Computer Science*. 2021.

[94] A. Rössler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies, and M. Nießner, "FaceForensics: A Large-scale Video Dataset for Forgery Detection in Human Faces," 2018.

[95] J. Gu, Y. Xu, J. Sun, and W. Liu, "Exploiting Deepfakes by Analyzing Temporal Feature Inconsistency," vol. 14, no. 12, 2023.

LAMPIRAN

Lampiran 1. Model *Summary*

Layer (Type)	Output Shape	Param #
Conv2d-1	[-1, 32, 112, 112]	864
BatchNorm2d-2	[-1, 32, 112, 112]	64
SiLU-3	[-1, 32, 112, 112]	0
Conv2d-4	[-1, 32, 112, 112]	288
BatchNorm2d-5	[-1, 32, 112, 112]	64
SiLU-6	[-1, 32, 112, 112]	0
AdaptiveAvgPool2d-7	[-1, 32, 1, 1]	0
Conv2d-8	[-1, 8, 1, 1]	264
SiLU-9	[-1, 8, 1, 1]	0
Conv2d-10	[-1, 32, 1, 1]	288
Sigmoid-11	[-1, 32, 1, 1]	0
SqueezeExcitation-12	[-1, 32, 112, 112]	0
Conv2d-13	[-1, 16, 112, 112]	512
BatchNorm2d-14	[-1, 16, 112, 112]	32
MBConv-15	[-1, 16, 112, 112]	0
Conv2d-16	[-1, 96, 112, 112]	1,536
BatchNorm2d-17	[-1, 96, 112, 112]	192
SiLU-18	[-1, 96, 112, 112]	0
Conv2d-19	[-1, 96, 56, 56]	864
BatchNorm2d-20	[-1, 96, 56, 56]	192
SiLU-21	[-1, 96, 56, 56]	0
AdaptiveAvgPool2d-22	[-1, 96, 1, 1]	0
Conv2d-23	[-1, 4, 1, 1]	388
SiLU-24	[-1, 4, 1, 1]	0
Conv2d-25	[-1, 96, 1, 1]	480
Sigmoid-26	[-1, 96, 1, 1]	0
SqueezeExcitation-27	[-1, 96, 56, 56]	0
Conv2d-28	[-1, 24, 56, 56]	2,304
BatchNorm2d-29	[-1, 24, 56, 56]	48
MBConv-30	[-1, 24, 56, 56]	0
Conv2d-31	[-1, 144, 56, 56]	3,456
BatchNorm2d-32	[-1, 144, 56, 56]	288
SiLU-33	[-1, 144, 56, 56]	0
Conv2d-34	[-1, 144, 56, 56]	1,296
BatchNorm2d-35	[-1, 144, 56, 56]	288

SiLU-36	[-1, 144, 56, 56]	0
AdaptiveAvgPool2d-37	[-1, 144, 1, 1]	0
Conv2d-38	[-1, 6, 1, 1]	870
SiLU-39	[-1, 6, 1, 1]	0
Conv2d-40	[-1, 144, 1, 1]	1,008
Sigmoid-41	[-1, 144, 1, 1]	0
SqueezeExcitation-42	[-1, 144, 56, 56]	0
Conv2d-43	[-1, 24, 56, 56]	3,456
BatchNorm2d-44	[-1, 24, 56, 56]	48
StochasticDepth-45	[-1, 24, 56, 56]	0
MBConv-46	[-1, 24, 56, 56]	0
Conv2d-47	[-1, 144, 56, 56]	3,456
BatchNorm2d-48	[-1, 144, 56, 56]	288
SiLU-49	[-1, 144, 56, 56]	0
Conv2d-50	[-1, 144, 28, 28]	3,600
BatchNorm2d-51	[-1, 144, 28, 28]	288
SiLU-52	[-1, 144, 28, 28]	0
AdaptiveAvgPool2d-53	[-1, 144, 1, 1]	0
Conv2d-54	[-1, 6, 1, 1]	870
SiLU-55	[-1, 6, 1, 1]	0
Conv2d-56	[-1, 144, 1, 1]	1,008
Sigmoid-57	[-1, 144, 1, 1]	0
SqueezeExcitation-58	[-1, 144, 28, 28]	0
Conv2d-59	[-1, 40, 28, 28]	5,760
BatchNorm2d-60	[-1, 40, 28, 28]	80
MBConv-61	[-1, 40, 28, 28]	0
Conv2d-62	[-1, 240, 28, 28]	9,600
BatchNorm2d-63	[-1, 240, 28, 28]	480
SiLU-64	[-1, 240, 28, 28]	0
Conv2d-65	[-1, 240, 28, 28]	6,000
BatchNorm2d-66	[-1, 240, 28, 28]	480
SiLU-67	[-1, 240, 28, 28]	0
AdaptiveAvgPool2d-68	[-1, 240, 1, 1]	0
Conv2d-69	[-1, 10, 1, 1]	2,410
SiLU-70	[-1, 10, 1, 1]	0
Conv2d-71	[-1, 240, 1, 1]	2,640
Sigmoid-72	[-1, 240, 1, 1]	0
SqueezeExcitation-73	[-1, 240, 28, 28]	0
Conv2d-74	[-1, 40, 28, 28]	9,600

BatchNorm2d-75	[-1, 40, 28, 28]	80
StochasticDepth-76	[-1, 40, 28, 28]	0
MBConv-77	[-1, 40, 28, 28]	0
Conv2d-78	[-1, 240, 28, 28]	9,600
BatchNorm2d-79	[-1, 240, 28, 28]	480
SiLU-80	[-1, 240, 28, 28]	0
Conv2d-81	[-1, 240, 14, 14]	2,160
BatchNorm2d-82	[-1, 240, 14, 14]	480
SiLU-83	[-1, 240, 14, 14]	0
AdaptiveAvgPool2d-84	[-1, 240, 1, 1]	0
Conv2d-85	[-1, 10, 1, 1]	2,410
SiLU-86	[-1, 10, 1, 1]	0
Conv2d-87	[-1, 240, 1, 1]	2,640
Sigmoid-88	[-1, 240, 1, 1]	0
SqueezeExcitation-89	[-1, 240, 14, 14]	0
Conv2d-90	[-1, 80, 14, 14]	19,200
BatchNorm2d-91	[-1, 80, 14, 14]	160
MBConv-92	[-1, 80, 14, 14]	0
Conv2d-93	[-1, 480, 14, 14]	38,400
BatchNorm2d-94	[-1, 480, 14, 14]	960
SiLU-95	[-1, 480, 14, 14]	0
Conv2d-96	[-1, 480, 14, 14]	4,320
BatchNorm2d-97	[-1, 480, 14, 14]	960
SiLU-98	[-1, 480, 14, 14]	0
AdaptiveAvgPool2d-99	[-1, 480, 1, 1]	0
Conv2d-100	[-1, 20, 1, 1]	9,620
SiLU-101	[-1, 20, 1, 1]	0
Conv2d-102	[-1, 480, 1, 1]	10,080
Sigmoid-103	[-1, 480, 1, 1]	0
SqueezeExcitation-104	[-1, 480, 14, 14]	0
Conv2d-105	[-1, 80, 14, 14]	38,400
BatchNorm2d-106	[-1, 80, 14, 14]	160
StochasticDepth-107	[-1, 80, 14, 14]	0
MBConv-108	[-1, 80, 14, 14]	0
Conv2d-109	[-1, 480, 14, 14]	38,400
BatchNorm2d-110	[-1, 480, 14, 14]	960
SiLU-111	[-1, 480, 14, 14]	0
Conv2d-112	[-1, 480, 14, 14]	4,320
BatchNorm2d-113	[-1, 480, 14, 14]	960

SiLU-114	[-1, 480, 14, 14]	0
AdaptiveAvgPool2d-115	[-1, 480, 1, 1]	0
Conv2d-116	[-1, 20, 1, 1]	9,620
SiLU-117	[-1, 20, 1, 1]	0
Conv2d-118	[-1, 480, 1, 1]	10,080
Sigmoid-119	[-1, 480, 1, 1]	0
SqueezeExcitation-120	[-1, 480, 14, 14]	0
Conv2d-121	[-1, 80, 14, 14]	38,400
BatchNorm2d-122	[-1, 80, 14, 14]	160
StochasticDepth-123	[-1, 80, 14, 14]	0
MBConv-124	[-1, 80, 14, 14]	0
Conv2d-125	[-1, 480, 14, 14]	38,400
BatchNorm2d-126	[-1, 480, 14, 14]	960
SiLU-127	[-1, 480, 14, 14]	0
Conv2d-128	[-1, 480, 14, 14]	12,000
BatchNorm2d-129	[-1, 480, 14, 14]	960
SiLU-130	[-1, 480, 14, 14]	0
AdaptiveAvgPool2d-131	[-1, 480, 1, 1]	0
Conv2d-132	[-1, 20, 1, 1]	9,620
SiLU-133	[-1, 20, 1, 1]	0
Conv2d-134	[-1, 480, 1, 1]	10,080
Sigmoid-135	[-1, 480, 1, 1]	0
SqueezeExcitation-136	[-1, 480, 14, 14]	0
Conv2d-137	[-1, 112, 14, 14]	53,760
BatchNorm2d-138	[-1, 112, 14, 14]	224
MBConv-139	[-1, 112, 14, 14]	0
Conv2d-140	[-1, 672, 14, 14]	75,264
BatchNorm2d-141	[-1, 672, 14, 14]	1,344
SiLU-142	[-1, 672, 14, 14]	0
Conv2d-143	[-1, 672, 14, 14]	16,800
BatchNorm2d-144	[-1, 672, 14, 14]	1,344
SiLU-145	[-1, 672, 14, 14]	0
AdaptiveAvgPool2d-146	[-1, 672, 1, 1]	0
Conv2d-147	[-1, 28, 1, 1]	18,844
SiLU-148	[-1, 28, 1, 1]	0
Conv2d-149	[-1, 672, 1, 1]	19,488
Sigmoid-150	[-1, 672, 1, 1]	0
SqueezeExcitation-151	[-1, 672, 14, 14]	0
Conv2d-152	[-1, 112, 14, 14]	75,264

BatchNorm2d-153	[-1, 112, 14, 14]	224
StochasticDepth-154	[-1, 112, 14, 14]	0
MBConv-155	[-1, 112, 14, 14]	0
Conv2d-156	[-1, 672, 14, 14]	75,264
BatchNorm2d-157	[-1, 672, 14, 14]	1,344
SiLU-158	[-1, 672, 14, 14]	0
Conv2d-159	[-1, 672, 14, 14]	16,800
BatchNorm2d-160	[-1, 672, 14, 14]	1,344
SiLU-161	[-1, 672, 14, 14]	0
AdaptiveAvgPool2d-162	[-1, 672, 1, 1]	0
Conv2d-163	[-1, 28, 1, 1]	18,844
SiLU-164	[-1, 28, 1, 1]	0
Conv2d-165	[-1, 672, 1, 1]	19,488
Sigmoid-166	[-1, 672, 1, 1]	0
SqueezeExcitation-167	[-1, 672, 14, 14]	0
Conv2d-168	[-1, 112, 14, 14]	75,264
BatchNorm2d-169	[-1, 112, 14, 14]	224
StochasticDepth-170	[-1, 112, 14, 14]	0
MBConv-171	[-1, 112, 14, 14]	0
Conv2d-172	[-1, 672, 14, 14]	75,264
BatchNorm2d-173	[-1, 672, 14, 14]	1,344
SiLU-174	[-1, 672, 14, 14]	0
Conv2d-175	[-1, 672, 7, 7]	16,800
BatchNorm2d-176	[-1, 672, 7, 7]	1,344
SiLU-177	[-1, 672, 7, 7]	0
AdaptiveAvgPool2d-178	[-1, 672, 1, 1]	0
Conv2d-179	[-1, 28, 1, 1]	18,844
SiLU-180	[-1, 28, 1, 1]	0
Conv2d-181	[-1, 672, 1, 1]	19,488
Sigmoid-182	[-1, 672, 1, 1]	0
SqueezeExcitation-183	[-1, 672, 7, 7]	0
Conv2d-184	[-1, 192, 7, 7]	129,024
BatchNorm2d-185	[-1, 192, 7, 7]	384
MBConv-186	[-1, 192, 7, 7]	0
Conv2d-187	[-1, 1152, 7, 7]	221,184
BatchNorm2d-188	[-1, 1152, 7, 7]	2,304
SiLU-189	[-1, 1152, 7, 7]	0
Conv2d-190	[-1, 1152, 7, 7]	28,800
BatchNorm2d-191	[-1, 1152, 7, 7]	2,304

SiLU-192	[-1, 1152, 7, 7]	0
AdaptiveAvgPool2d-193	[-1, 1152, 1, 1]	0
Conv2d-194	[-1, 48, 1, 1]	55,344
SiLU-195	[-1, 48, 1, 1]	0
Conv2d-196	[-1, 1152, 1, 1]	56,448
Sigmoid-197	[-1, 1152, 1, 1]	0
SqueezeExcitation-198	[-1, 1152, 7, 7]	0
Conv2d-199	[-1, 192, 7, 7]	221,184
BatchNorm2d-200	[-1, 192, 7, 7]	384
StochasticDepth-201	[-1, 192, 7, 7]	0
MBConv-202	[-1, 192, 7, 7]	0
Conv2d-203	[-1, 1152, 7, 7]	221,184
BatchNorm2d-204	[-1, 1152, 7, 7]	2,304
SiLU-205	[-1, 1152, 7, 7]	0
Conv2d-206	[-1, 1152, 7, 7]	28,800
BatchNorm2d-207	[-1, 1152, 7, 7]	2,304
SiLU-208	[-1, 1152, 7, 7]	0
AdaptiveAvgPool2d-209	[-1, 1152, 1, 1]	0
Conv2d-210	[-1, 48, 1, 1]	55,344
SiLU-211	[-1, 48, 1, 1]	0
Conv2d-212	[-1, 1152, 1, 1]	56,448
Sigmoid-213	[-1, 1152, 1, 1]	0
SqueezeExcitation-214	[-1, 1152, 7, 7]	0
Conv2d-215	[-1, 192, 7, 7]	221,184
BatchNorm2d-216	[-1, 192, 7, 7]	384
StochasticDepth-217	[-1, 192, 7, 7]	0
MBConv-218	[-1, 192, 7, 7]	0
Conv2d-219	[-1, 1152, 7, 7]	221,184
BatchNorm2d-220	[-1, 1152, 7, 7]	2,304
SiLU-221	[-1, 1152, 7, 7]	0
Conv2d-222	[-1, 1152, 7, 7]	28,800
BatchNorm2d-223	[-1, 1152, 7, 7]	2,304
SiLU-224	[-1, 1152, 7, 7]	0
AdaptiveAvgPool2d-225	[-1, 1152, 1, 1]	0
Conv2d-226	[-1, 48, 1, 1]	55,344
SiLU-227	[-1, 48, 1, 1]	0
Conv2d-228	[-1, 1152, 1, 1]	56,448
Sigmoid-229	[-1, 1152, 1, 1]	0
SqueezeExcitation-230	[-1, 1152, 7, 7]	0

Conv2d-231	[-1, 192, 7, 7]	221,184
BatchNorm2d-232	[-1, 192, 7, 7]	384
StochasticDepth-233	[-1, 192, 7, 7]	0
MBConv-234	[-1, 192, 7, 7]	0
Conv2d-235	[-1, 1152, 7, 7]	221,184
BatchNorm2d-236	[-1, 1152, 7, 7]	2,304
SiLU-237	[-1, 1152, 7, 7]	0
Conv2d-238	[-1, 1152, 7, 7]	10,368
BatchNorm2d-239	[-1, 1152, 7, 7]	2,304
SiLU-240	[-1, 1152, 7, 7]	0
AdaptiveAvgPool2d-241	[-1, 1152, 1, 1]	0
Conv2d-242	[-1, 48, 1, 1]	55,344
SiLU-243	[-1, 48, 1, 1]	0
Conv2d-244	[-1, 1152, 1, 1]	56,448
Sigmoid-245	[-1, 1152, 1, 1]	0
SqueezeExcitation-246	[-1, 1152, 7, 7]	0
Conv2d-247	[-1, 320, 7, 7]	368,640
BatchNorm2d-248	[-1, 320, 7, 7]	640
MBConv-249	[-1, 320, 7, 7]	0
Conv2d-250	[-1, 1280, 7, 7]	409,600
BatchNorm2d-251	[-1, 1280, 7, 7]	2,560
SiLU-252	[-1, 1280, 7, 7]	0
AdaptiveAvgPool2d-253	[-1, 1280, 1, 1]	0
Dropout-254	[-1, 1280]	0
Dropout-255	[-1, 1280]	0
Linear-256	[-1, 1]	1,281
Total params: 4,008,829 Trainable params: 1,281 Non-trainable params: 4,007,548		
Input size (MB): 0.57 Forward/backward pass size (MB): 173.65 Params size (MB): 15.29 Estimated Total Size (MB): 189.52		

Struktur Model *EfficientNet Transfer Learning*

Lampiran 2. Link ke Dataset yang digunakan

Lampiran 3. Ringkasan Parameter yang digunakan

Parameter	Nilai	Justifikasi Teknis
Backbone Model	EfficientNet-B0	Keseimbangan terbaik akurasi/komputasi (5.3M param).
Input Size	224 x 224 px	Standar arsitektur, optimal untuk fitur spasial.
Dataset	300 Video	Presisi tertinggi (88.5%), minim *False Positive*.
Sampling	20 Frame (Linspace)	Representasi temporal merata, tahan variasi FPS.
Face Limit	3 Wajah	Efisiensi inferensi, fokus pada subjek utama.
Min Face Size	20 piksel	Mencegah analisis pada citra yang tidak layak (*upscaled noise*).
Threshold Frame	0.50	Cutoff probabilitas biner standar.
Threshold Video	0.30	*Safety margin* untuk manipulasi temporal/parsial.

Lampiran 4. Tabel Pengujian Parameter *Minimal Face Size*

Tabel 33.Hasil Pengujian pada Parameter MFS 10x10 (Asli)

No	Nama Video	Hasil Klasifikasi	Efisiensi (waktu)	Akurasi	Keterangan
1	Subject1.mp4	Asli	21.78 detik	96%	Benar
2	Subject2.mp4	*Deepfake*	72.05 detik	76%	Salah
3	Subject3.mp4	Asli	82.18 detik	85%	Benar
4	Subject4.mp4	Asli	77.27 detik	74%	Benar
5	Subject5.mp4	Asli	70.53 detik	74%	Benar
6	Subject6.mp4	Asli	94.23 detik	97%	Benar
7	Subject7.mp4	Asli	12.27 detik	91%	Benar
8	Subject8.mp4	Asli	75.56 detik	95%	Benar
9	Subject9.mp4	Asli	12.78 detik	94%	Benar
10	Subject10.mp4	Asli	14.23 detik	98%	Benar
11	990.mp4	Asli	24.12 detik	84%	Benar
12	991.mp4	Asli	18.36 detik	90%	Benar
13	992.mp4	Asli	18.87 detik	97%	Benar
14	993.mp4	Asli	18.44 detik	99%	Benar
15	994.mp4	Asli	22.81 detik	91%	Benar
16	995.mp4	Asli	17.84 detik	99%	Benar
17	996.mp4	Asli	20.72 detik	97%	Benar
18	997.mp4	Asli	22.62 detik	97%	Benar
19	998.mp4	*Deepfake*	29.40 detik	68%	Salah
20	999.mp4	Asli	17.38 detik	97%	Benar

Tabel 34.Hasil Pengujian pada Parameter MFS 5x5 (Asli)

No	Nama Video	Hasil Klasifikasi	Efisiensi (waktu)	Akurasi	Keterangan
1	Subject1.mp4	Asli	21.41 detik	96%	Benar
2	Subject2.mp4	*Deepfake*	68.96 detik	76%	Salah
3	Subject3.mp4	Asli	79.46 detik	85%	Benar
4	Subject4.mp4	Asli	74.77 detik	74%	Benar
5	Subject5.mp4	Asli	67.92 detik	74%	Benar
6	Subject6.mp4	Asli	76.06 detik	97%	Benar
7	Subject7.mp4	Asli	13.86 detik	91%	Benar
8	Subject8.mp4	Asli	70.67 detik	95%	Benar
9	Subject9.mp4	Asli	12.95 detik	94%	Benar
10	Subject10.mp4	Asli	14.16 detik	98%	Benar
11	990.mp4	Asli	24.10 detik	84%	Benar
12	991.mp4	Asli	20.06 detik	90%	Benar
13	992.mp4	Asli	19.48 detik	97%	Benar
14	993.mp4	Asli	19.32 detik	99%	Benar
15	994.mp4	Asli	24.09 detik	91%	Benar
16	995.mp4	Asli	18.16 detik	99%	Benar
17	996.mp4	Asli	20.55 detik	97%	Benar
18	997.mp4	Asli	23.62 detik	97%	Benar
19	998.mp4	*Deepfake*	30 detik	68%	Salah
20	999.mp4	Asli	18.06 detik	97%	Benar

Tabel 35.Hasil Pengujian pada Parameter MFS 0x0 (Asli)

No	Nama Video	Hasil Klasifikasi	Efisiensi (waktu)	Akurasi	Keterangan
1	Subject1.mp4	Asli	18.58 detik	96%	Benar
2	Subject2.mp4	*Deepfake*	61.20 detik	76%	Salah
3	Subject3.mp4	Asli	67.97 detik	85%	Benar
4	Subject4.mp4	Asli	64.55 detik	74%	Benar
5	Subject5.mp4	Asli	60.20 detik	74%	Benar
6	Subject6.mp4	Asli	68.57 detik	97%	Benar
7	Subject7.mp4	Asli	9.51 detik	91%	Benar
8	Subject8.mp4	Asli	63.68 detik	95%	Benar
9	Subject9.mp4	Asli	10.87 detik	94%	Benar
10	Subject10.mp4	Asli	11.28 detik	98%	Benar
11	990.mp4	Asli	22.84 detik	84%	Benar
12	991.mp4	Asli	18.36 detik	90%	Benar
13	992.mp4	Asli	18.87 detik	97%	Benar
14	993.mp4	Asli	15.19 detik	99%	Benar
15	994.mp4	Asli	23.54 detik	91%	Benar
16	995.mp4	Asli	18.49 detik	99%	Benar
17	996.mp4	Asli	21.50 detik	97%	Benar
18	997.mp4	Asli	23.48 detik	97%	Benar
19	998.mp4	*Deepfake*	30 detik	68%	Salah
20	999.mp4	Asli	18.77 detik	97%	Benar

Tabel 36. Hasil Pengujian pada Parameter MFS 10x10 (*Deepfake*)

No	Nama Video	Hasil Klasifikasi	Efisiensi (waktu)	Akurasi	Keterangan
1	Subject001.mp4	*Deepfake*	12.65 detik	71%	Benar
2	Subject002.mp4	*Deepfake*	21.21 detik	81%	Benar
3	Subject003.mp4	*Deepfake*	11.37 detik	90%	Benar
4	Subject004.mp4	Asli	11.31 detik	69%	Salah
5	Subject005.mp4	Asli	11.02 detik	83%	Salah
6	Subject006.mp4	*Deepfake*	13 detik	87%	Benar
7	Subject007.mp4	*Deepfake*	13.32 detik	76%	Benar
8	Subject008.mp4	*Deepfake*	10.84 detik	76%	Benar
9	Subject009.mp4	*Deepfake*	10.38 detik	91%	Benar
10	Subject010.mp4	*Deepfake*	8.28 detik	74%	Benar
11	990_008.mp4	Asli	20.52 detik	73%	Salah
12	991_064.mp4	*Deepfake*	19.28 detik	87%	Benar
13	992_980.mp4	*Deepfake*	18.71 detik	79%	Benar
14	993_989.mp4	*Deepfake*	18.48 detik	91%	Benar
15	994_986.mp4	Asli	22.50 detik	79%	Salah
16	995_233.mp4	*Deepfake*	18.89 detik	81%	Benar
17	996_056.mp4	Asli	20.12 detik	88%	Salah
18	997_040.mp4	*Deepfake*	20.40 detik	85%	Benar
19	998_561.mp4	*Deepfake*	21.90 detik	93%	Benar
20	999_960.mp4	*Deepfake*	18.55 detik	79%	Benar

Tabel 37.Hasil Pengujian pada Parameter MFS 5x5 (*Deepfake*)

No	Nama Video	Hasil Klasifikasi	Efisiensi (waktu)	Akurasi	Keterangan
1	Subject001.mp4	*Deepfake*	10.14 detik	71%	Benar
2	Subject002.mp4	*Deepfake*	17.71 detik	81%	Benar
3	Subject003.mp4	*Deepfake*	9.15 detik	90%	Benar
4	Subject004.mp4	Asli	8.86 detik	69%	Salah
5	Subject005.mp4	Asli	9.27 detik	83%	Salah
6	Subject006.mp4	*Deepfake*	10.56 detik	87%	Benar
7	Subject007.mp4	*Deepfake*	10.69 detik	76%	Benar
8	Subject008.mp4	*Deepfake*	8.66 detik	76%	Benar
9	Subject009.mp4	*Deepfake*	7.97 detik	91%	Benar
10	Subject010.mp4	*Deepfake*	6.51 detik	74%	Benar
11	990_008.mp4	Asli	11.36 detik	73%	Salah
12	991_064.mp4	*Deepfake*	10.96 detik	87%	Benar
13	992_980.mp4	*Deepfake*	18.53 detik	79%	Benar
14	993_989.mp4	*Deepfake*	18.80 detik	91%	Benar
15	994_986.mp4	Asli	22.55 detik	79%	Salah
16	995_233.mp4	*Deepfake*	18.24 detik	81%	Benar
17	996_056.mp4	Asli	20.22 detik	88%	Salah
18	997_040.mp4	*Deepfake*	22.21 detik	85%	Benar
19	998_561.mp4	*Deepfake*	29.55 detik	93%	Benar
20	999_960.mp4	*Deepfake*	17.51 detik	79%	Benar

Tabel 38.Hasil Pengujian pada Parameter MFS 0x0 (Asli)

No	Nama Video	Hasil Klasifikasi	Efisiensi (waktu)	Akurasi	Keterangan
1	Subject001.mp4	*Deepfake*	10.05 detik	71%	Benar
2	Subject002.mp4	*Deepfake*	17.34 detik	81%	Benar
3	Subject003.mp4	*Deepfake*	8.98 detik	90%	Benar
4	Subject004.mp4	Asli	8.73 detik	69%	Salah
5	Subject005.mp4	Asli	8.84 detik	83%	Salah
6	Subject006.mp4	*Deepfake*	10.20 detik	87%	Benar
7	Subject007.mp4	*Deepfake*	11.24 detik	76%	Benar
8	Subject008.mp4	*Deepfake*	8.72 detik	76%	Benar
9	Subject009.mp4	*Deepfake*	7.88 detik	91%	Benar
10	Subject010.mp4	*Deepfake*	6.29 detik	74%	Benar
11	990_008.mp4	Asli	8.46 detik	73%	Salah
12	991_064.mp4	*Deepfake*	9.24 detik	87%	Benar
13	992_980.mp4	*Deepfake*	7.14 detik	79%	Benar
14	993_989.mp4	*Deepfake*	6.92 detik	91%	Benar
15	994_986.mp4	Asli	8.24 detik	79%	Salah
16	995_233.mp4	*Deepfake*	6.53 detik	81%	Benar
17	996_056.mp4	Asli	7.60 detik	88%	Salah
18	997_040.mp4	*Deepfake*	9.36 detik	85%	Benar
19	998_561.mp4	*Deepfake*	11.90 detik	93%	Benar
20	999_960.mp4	*Deepfake*	6.39 detik	79%	Benar